18 Direct Memory Access (DMA)

The DMA (at **Figure 190**) shall move data from a source module to a destination module without CPU intervention.

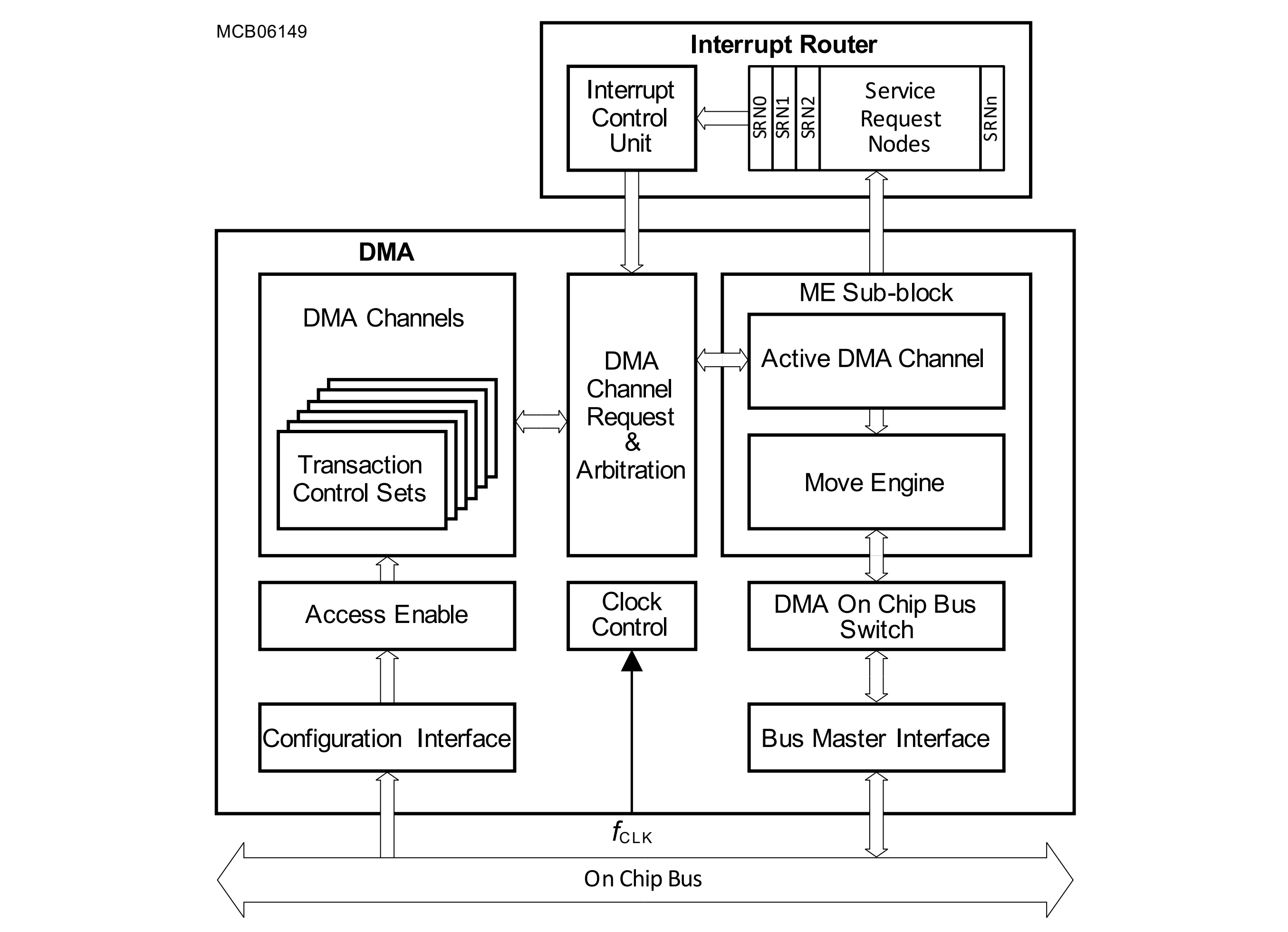

Figure 190 Block Diagram of DMA

DMA Glossary

Table 564 DMA Acronyms

Acronym	Description
ACCLL	Accumulated Linked List
CONLL	Conditional Linked List
CH	Channel
DER	Destination Error
DLLER	Linked List Error
DMA	Direct Memory Access
DMALL	Direct Memory Access Linked List
DMARAM	Direct Memory Access Random Access Memory
EH	Error Handler

Table 564 DMA Acronyms (cont'd)

Acronym	Description
ME	Move Engine
PATDET	Pattern Detection
RAMER	RAM Error
RDCRC	Read Data Cyclic Redundancy Check
RP	Resource Partition
RROAT	Reset Request Only After Transaction
SAFLL	Safe Linked List
SDCRC	Source Destination Cyclic Redundancy Check
SER	Source Error
SLLER	Safe Linked List Error
TCS	Transaction Control Set
TRL	Transaction/Transfer Request Lost
TS	Trigger Set

Table 565 DMA Terms

Term	Description
DMA Configuration Data	The DMA configuration data configures the DMA configuration structure.
DMA Configuration Structure	The DMA configuration structure is used to configure CH, RP and ME. The DMA configuration structure is defined by the DMA address map. *Note: The DMA configuration structure does not include IR, SMU, etc. which is needed to make the DMA function within the system.* *3. The DMA configuration structure and DMA configuration data defines all DMA move functions.*
DMA Software Request	The CPU initiates a DMA transfer or a DMA transaction in a DMA channel.
DMA Hardware Request	The Interrupt Router initiates a DMA transfer or a DMA transaction in a DMA channel.
DMA Daisy Chain Request	As soon as the current DMA transaction completes, the DMA channel initiates a DMA transfer or a DMA transaction in the next lower priority DMA channel.
DMA Auto Start Request	As soon as the current DMA transaction completes, a linked list operation loads a new TCS and initiates a DMA transfer or a DMA transaction.
DMA Request	DMA software request, DMA hardware request, DMA daisy chain request or DMA auto start request.
DMA Read Move Data	Data read from the source module.
DMA Write Move Data	Data written to the destination module.
DMA Channel Interrupt Triggers	DMA signalling of the successful completion of a DMA move function.
DMA RP Error Interrupt Triggers	DMA signalling of a DMA move function error.
DMA Alarms	DMA signalling of a DMA safety error.

Table 565 DMA Terms (cont'd)

Term	Description
DMA Address Checksum	Cyclic redundancy checksum calculated according to the IEEE 802.3 standard for addresses generated during a DMA transaction.
DMA Data Checksum	Cyclic redundancy checksum calculated according to the IEEE 802.3 standard for data moved during a DMA transaction.
DMA Timestamp	Appendage of timestamp to a DMA transaction.

The terms (at **Table 566**) are used to define the structure of the data move function.

Table 566 Structure of Data Move Function

Term	Description
DMA Move	A DMA move is an operation that always consists of two parts: 1. A DMA read move that loads DMA read move data from a source module to the DMA. 2. A DMA write move that stores DMA write move data from the DMA to a destination module.
DMA Transfer	A DMA transfer shall be composed of 1, 2, 3, 4, 5, 8, 9 or 16 DMA moves.
DMA Transaction	A DMA transaction shall be composed of at least one DMA transfer.
Linked List	A linked list is a series of DMA transactions executed in the same DMA channel.

18.1 Feature List

The DMA is a fast and flexible DMA controller that has the following features:

- **Resource Partitions**
  - An application running one set of defined move data functions shall be free from interference by another application running another set of defined move data functions.
  - Each RP has independent **Access Enable** control via enabling individual Master TAG identifiers to have write access enable to the RP and assigned DMA channels.
  - Each RP has a unique master tag identifier driven onto the on chip bus during a DMA move.
  - Each RP executes on chip bus accesses in supervisor or user mode.
- **DMA Channels**
  - The DMA supports multiple independent DMA channels.
  - Each DMA channel shall be assigned to a RP.
  - Each DMA channel shall be individually programmable.
  - Each DMA channel TCS shall be stored in DMARAM.
  - The DMA channel source and destination address pointers shall be 32-bit wide address counters.
  - Wrap buffer addressing mode with flexible circular buffer sizes.
  - The DMA channel source and destination wrap buffers shall be selectable.
  - Programmable data width of DMA moves.
- **Double Buffering Operations**
  - The DMA transaction can execute read or fill DMA transfers from one of two source or destination buffers.
  - A control bit allows the re-direction of DMA transfers from the one buffer to the other buffer.
- **DMA Linked List (DMALL)**
  - The current DMA transaction can load the next DMA channel TCS into the DMARAM by overwriting the existing DMA channel TCS.

  - The next DMA transaction may be auto started.
- **DMA Channel Request Control**
  - **DMA Software Request**
  - **DMA Hardware Request**
  - **DMA Daisy Chain Request**
  - **DMA Auto Start Request**
- **Move Engine**
  - Any ME shall service a DMA request from any DMA channel.
  - DMA requests from the highest number DMA channel are serviced first by a ME.
  - Multiple MEs support the parallel servicing of DMA requests.
  - SRI-source to SRI-destination data block move throughput <8 Mbyte DMA moves per DMA transaction.
  - SPB-source to SPB-destination data block move throughput <1 Mbyte DMA moves per DMA transaction.
- **DMA On Chip Bus Switch**
  - DMA read moves and DMA write moves are directed by the DMA on chip bus switch to different sources and destinations depending on the source or destination address.
  - Buffer capability for move actions on the buses (at least 1 x DMA move per bus is buffered).
- Interrupt Triggers
  - Each DMA channel generates one interrupt trigger with an unique interrupt vector and priority level.
  - Each DMA RP generates one error interrupt trigger with an unique interrupt vector and priority level.
- Operating frequencies
  - The DMA configuration and request control function works at the SPB clock frequency.
  - The ME function works at the SRI clock frequency in order to maximise the data throughput for DMA moves from SRI source addresses to SRI destination addresses.

18.2 Overview

The DMA moves data from source locations to destination locations without the intervention of the CPU or other on chip devices. A data move is controlled by the TCS of an active DMA channel executed by a ME. A DMA channel is activated by a DMA request.

18.3 Functional Description

18.3.1 Configuration Interface

The DMA implements a standard FPI slave interface compliant with the FPI bus protocol on the SPB bus. The DMA configuration interface supports single data transfers and does not support block transfers.

18.3.2 Resource Partitions

During DMA configuration, each DMA channel shall be assigned to a RP.

18.3.2.1 Access Enable

Each RP has its own access enable protection. A slave destination controls access to its bus peripheral interfaces and kernel address space. Each on chip resource with bus master capability has a unique master tag identifier that is used to identify the source of an on chip bus transaction. The master tag identifier based access protection is used to enable write accesses to individual slave address ranges.

18.3.2.2 DMA Moves

Each RP has a unique master tag identifier driven onto the on chip bus during a DMA read move or DMA write move. Mode control selects if a DMA move on chip bus access is made in supervisor mode or user mode.

Note: See On-Chip System Connectivity chapter for on chip bus master TAG identifier assignments.

18.3.2.3 DMA RP Error Interrupt Service Request

Each RP generates one error interrupt service request to cover all error events for DMA channels assigned to that RP including servicing of DMA requests by the ME:

- DMA channel **TRL** interrupt service request.
- ME SER and DER error interrupt service request.
- ME **DMARAM Integrity Error** error interrupt service request.
- ME **Linked List Operation TCS Load Error** error interrupt service request.
- ME **SAFLL DMA Address Checksum Error** error interrupt service request.

If a DMA RP error interrupt service request is triggered then the software RP application EH shall read the contents of the error status registers to identify the origin of the error.

18.3.3 DMA Channels

Each DMA channel is assigned to a RP and stores the context of an independent DMA operation.

18.3.3.1 DMA Channel Request Control

The DMA channel request control (at **Figure 191**) is implemented for each DMA channel.

The DMA channel operation is individually programmable. The following types of DMA requests are possible:

- **DMA Software Request** initiated by a CPU.
- **DMA Hardware Request** initiated by the Interrupt Router (IR) Interrupt Control Unit (ICU).
- **DMA Daisy Chain Request** initiated by the next higher priority DMA channel.

- **DMA Auto Start Request** initiated by the loading of the next TCS during a **DMA Linked List (DMALL)**, **Accumulated Linked List (ACCLL)**, **Safe Linked List (SAFLL)** or **Conditional Linked List (CONLL)** operation.

The DMA channel status flag TSR.CH indicates if a DMA request is pending. TSR.CH may be cleared at the start of a DMA transfer or at the end of a DMA transaction. It follows that a DMA request may trigger a single **DMA Transfer** or one complete **DMA Transaction**.

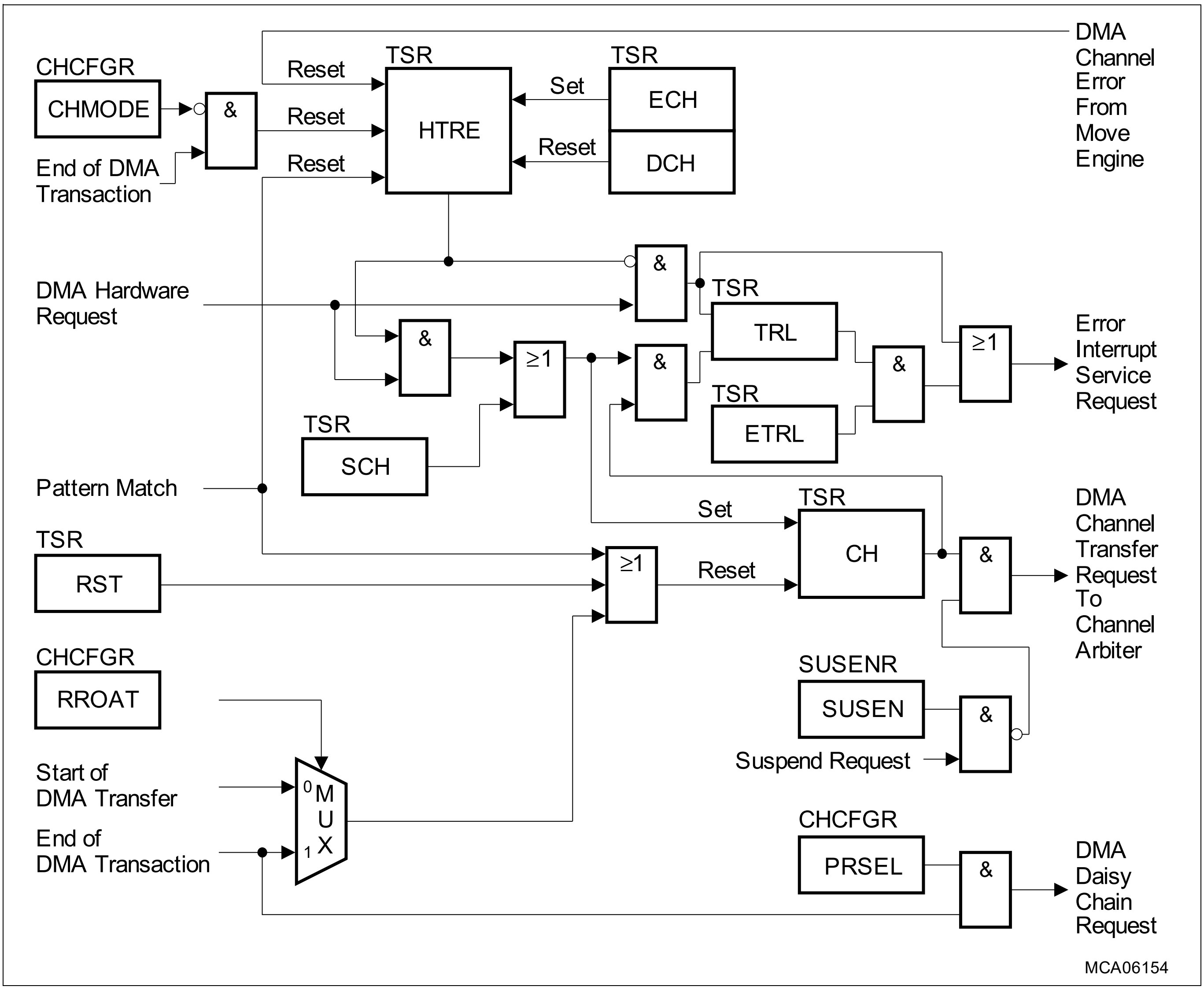

Figure 191 DMA Channel Request Control

18.3.3.1.1 DMA Channel States

Table 567 DMA Channel States

State	Status	Comments
Idle State	DMA channel TSR.CH = 0_B	No **DMA Request** is pending.
Reset State	DMA channel TSR.CH = 0_B	No **DMA Request** is pending. DMA channel TCS bits cleared.
Halt State	DMA channel TSR.HLTACK = 1_B	A **DMA Request** shall not be serviced by a ME.
Pending State	DMA channel TSR.CH = 1_B	**DMA Request** is waiting to be serviced by a ME.
Active State		ME is servicing a **DMA Request**.

18.3.3.1.2 Reset Request Only After Transaction (RROAT)

The clearing of the DMA channel TSR.CH bit is controlled as follows:

- CHCFGR.RROAT = 0_B
  - DMA channel TSR.CH is cleared at the start of each DMA transfer.
  - One **DMA Request** starts one single **DMA Transfer**.
- CHCFGR.RROAT = 1_B
  - DMA channel TSR.CH is cleared at the end of each DMA transaction.
  - One **DMA Request** starts one complete **DMA Transaction**.

18.3.3.2 DMA Software Request

One **DMA Software Request** may start one complete DMA transaction or one single DMA transfer.
If the DMA channel is triggered only by software then a **DMA Hardware Request** should be disabled.
Software must set TSR.DCH to 1_B to disable a **DMA Hardware Request** (TSR.HTRE = 0_B).

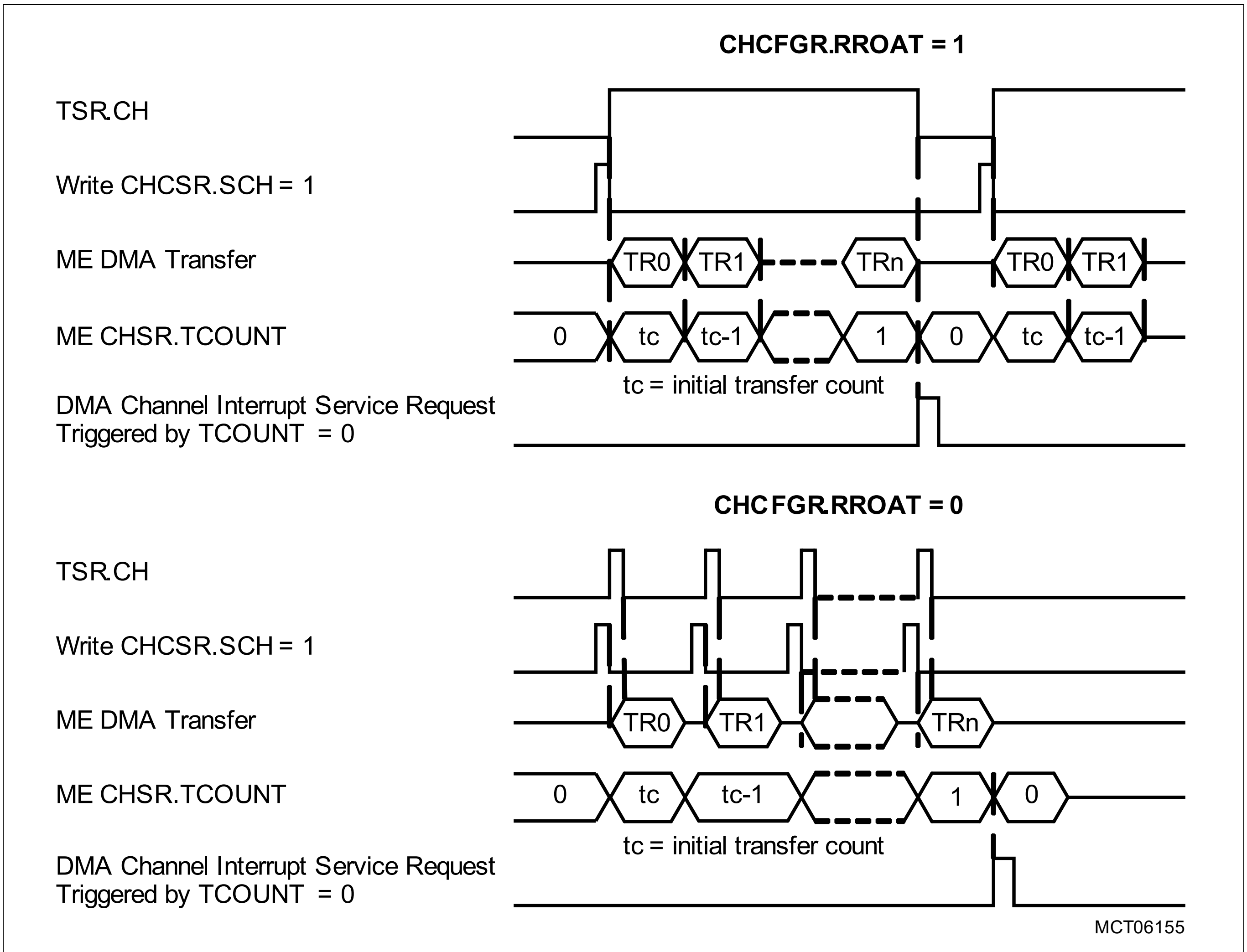

Figure 192 Software Control

The following DMA channel configuration is required to initiate one complete **DMA Transaction** under software control:

- DMA channel CHCFGR.RROAT = 1_B

A **DMA Software Request** is initiated by setting the DMA channel CHCSR.SCH to 1_B with the result that TSR.CH is set to 1_B. At the start of the DMA transaction, the value of the DMA channel reload value CHCFGR.TREL is loaded into ME CHSR.TCOUNT and DMA transfers are executed. After each DMA transfer, ME CHSR.TCOUNT is decremented and the next source and destination addresses are calculated. When TCOUNT decrements to 0_D, the DMA channel status flag TSR.CH is reset. Setting the DMA channel CHCSR.SCH = 1_B again starts a new DMA transaction of the DMA channel with the stored TCS parameters.

The following DMA channel configuration is required to initiate each single **DMA Transfer** under software control:

- DMA channel CHCFGR.RROAT = 0_B

A **DMA Software Request** must be initiated for each DMA transfer by setting the DMA channel CHCSR.SCH to 1_B.

18.3.3.3 DMA Hardware Request

A **DMA Hardware Request** is enabled/disabled by the DMA channel Hardware Transaction/Transfer Request Enable (HTRE) bit TSR.HTRE. The HTRE functionality is as follows:

- Software may set (TSR.ECH = 1_B) or clear (TSR.DCH = 1_B) DMA channel TSR.HTRE.
- Cleared as a result of the ME reporting an error for the DMA channel.
- Single Mode: at the start of the last DMA transfer of a DMA transaction.

Table 568 Conditions to Set/Reset DMA channel TSR.HTRE

DMA channel TSR.ECH	DMA channel TSR.DCH	Start of last DMA transfer of a DMA transaction[1]	Modification of DMA channel TSR.HTRE
0	0	0	Unchanged
1	0	0	Set
X	1	X	Reset
X	X	1	Reset

1) In **Single Mode** only. In **Continuous Mode**, the end of a DMA transaction has no impact.

DMA Channel Mode

The DMA channel mode bit CHCFGR.CHMODE controls the DMA channel mode as follows:

- **Single Mode** (DMA channel CHCFGR.CHMODE = 0_B)
  - **DMA Hardware Request** is disabled by hardware on completion of a DMA transaction.
- **Continuous Mode** (DMA channel CHCFGR.CHMODE = 1_B)
  - **DMA Hardware Request** is not disabled by hardware on completion of a DMA transaction.

Single Mode

The following DMA channel configuration is required to initiate one complete **DMA Transaction** under hardware control in Single Mode:

- DMA channel CHCFGR.CHMODE = 0_B
- DMA channel CHCFGR.RROAT = 1_B
- DMA channel CHCFGR.PRSEL = 0_B
- DMA channel TSR.ECH = 1_B

Setting DMA channel TSR.ECH to 1_B enables a **DMA Hardware Request** (TSR.HTRE = 1_B) for the DMA channel. When the ICU generates a **DMA Hardware Request**, TSR.CH is set high. If the DMA channel wins channel arbitration then the DMA channel transitions to the active state. The value of CHCFGR.TREL is loaded into the ME CHSR.TCOUNT and the DMA transaction is started by executing the first DMA transfer. After each DMA transfer, ME

CHSR.TCOUNT is decremented and the next source and destination addresses are calculated. When TCOUNT decrements to 0_D, **DMA Hardware Request** is disabled and status flags TSR.CH and TSR.HTRE are reset. In order to start a new hardware-controlled DMA transaction, a **DMA Hardware Request** must be enabled again by software writing TSR.ECH = 1_B to set TSR.HTRE. The hardware request disable function in Single Mode is typically needed to reprogram a DMARAM channel TCS before the next **DMA Hardware Request** initiates a DMA transaction.

The following DMA channel configuration is required to initiate each single **DMA Transfer** under hardware control in Single Mode:

- DMA channel CHCFGR.RROAT = 0_B

In this DMA channel configuration, TSR.CH is cleared at the start of each DMA transfer and a new **DMA Hardware Request** must be generated to start the next DMA transfer.

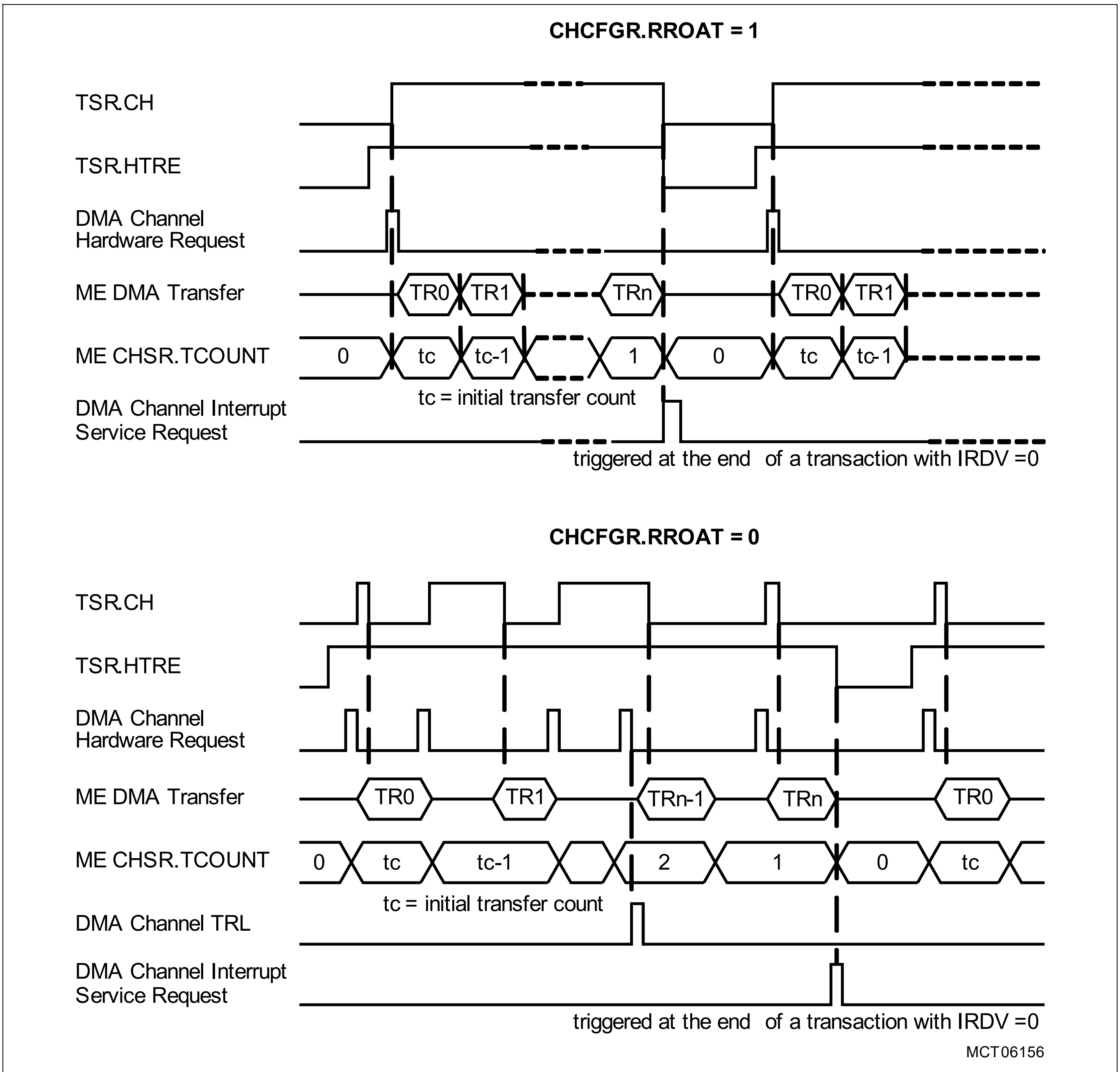

Figure 193 Hardware Control in Single Mode

Continuous Mode

If the DMA channel is configured for Continuous Mode (CHCFGR.CHMODE = 1_B), then TSR.HTRE is not reset at the end of a DMA transaction. On completion of the current DMA transaction (ME CHSR.TCOUNT = 0_D) each new **DMA Hardware Request** will start a new DMA transaction with the stored DMA channel TCS.

18.3.3.4 Combined DMA Software Request and DMA Hardware Request

A DMA channel may be operated under combined software and hardware control. The example (at **Figure 194**) demonstrates combined control:

- The first DMA transfer is triggered by a **DMA Software Request** setting the DMA channel CHCSR.SCH = 1_B.
- A **DMA Hardware Request** is still disabled i.e. DMA channel TSR.HTRE = 0_B.
- Software enables a **DMA Hardware Request** by setting the DMA channel TSR.ECH = 1_B.
- Each subsequent DMA transfer is triggered by a **DMA Hardware Request** from the ICU.

In the example, the DMA channel operates in Single Mode (DMA channel CHCFGR.CHMODE = 0_B). In single mode, the DMA channel TSR.HTRE is reset by hardware when ME CHSR.TCOUNT = 0_D at the end of the DMA transaction.

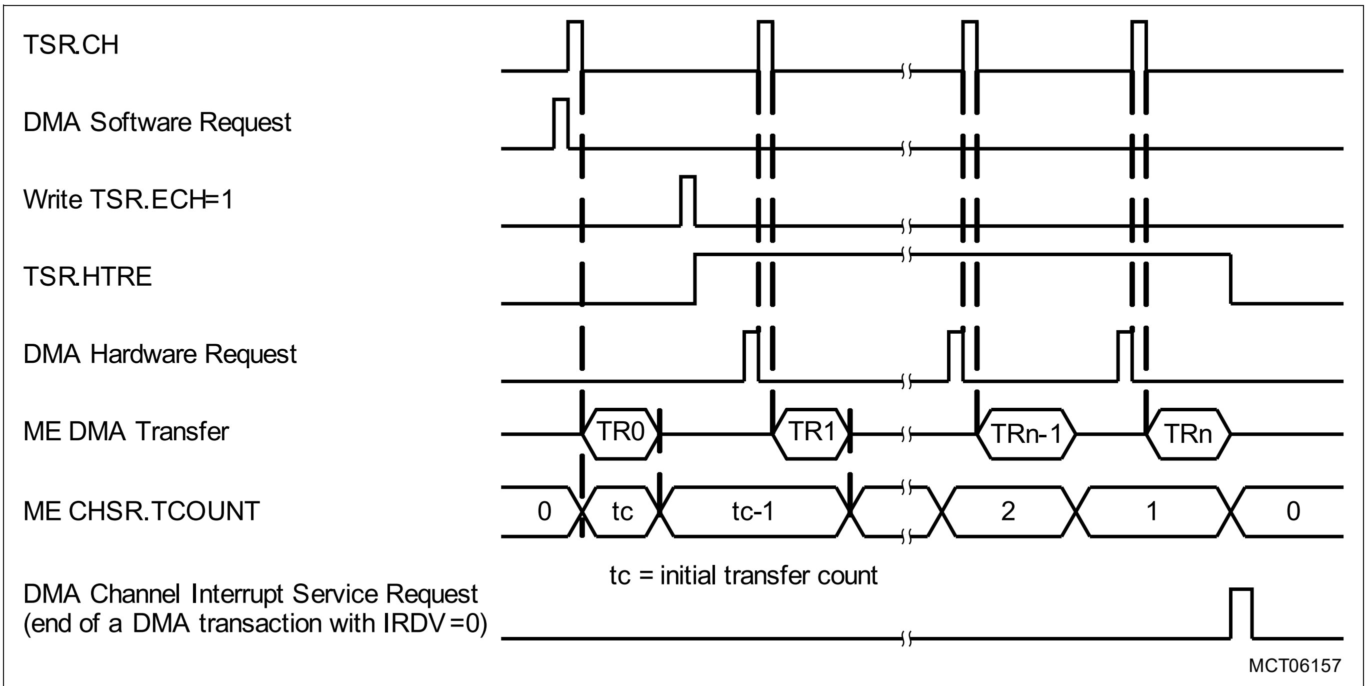

Figure 194 Transaction Start by Software, Continuation by Hardware

Transaction Request Lost (TRL)

When a DMA channel is configured to be triggered by parallel hardware and software requests, if a **DMA Software Request** and a **DMA Hardware Request** collide in the same clock cycle then a **TRL** event shall be flagged.

18.3.3.5 DMA Daisy Chain Request

DMA channels shall be configured for DMA daisy chain request by setting DMA channel CHCFGR.PRSEL.

When a higher priority DMA channel completes a DMA transaction it will initiate a DMA transaction in the next lower priority DMA channel by setting the access pending bit TSR.CH bit. **DMA Daisy Chain Request** is limited to a higher priority DMA channel initiating a DMA request in the next lower priority DMA Channel.

Enabling the daisy chain disables the **DMA Channel Interrupt Service Request** trigger in the next higher priority DMA channel. In a typical DMA daisy chain application only the lowest priority DMA channel is required to generate a **DMA Channel Interrupt Service Request**. When the sequence of DMA transactions from the highest to lowest priority DMA channel has completed then the lowest priority DMA channel in the daisy chain will generate a **DMA Channel Interrupt Service Request** to signal the end of the DMA operation.

If DMA channels are configured in a daisy chain, a DMA transfer or DMA transaction in the highest priority DMA channel is initiated by a **DMA Software Request** or a **DMA Hardware Request**. DMA transactions in the lower priority DMA channels are triggered by a **DMA Daisy Chain Request** in order to improve DMA latency.

18.3.3.6 DMA Channel Transaction Request Lost Interrupt Service Request

A DMA channel **TRL** event occurs for the following conditions:

- If a **DMA Request** is detected and the DMA channel TSR.CH is set, the DMA channel shall set the DMA channel TRL bit (TSR.TRL = 1_B). If the DMA channel enable TRL bit is set (TSR.ETRL = 1_B), the DMA shall trigger a **DMA RP Error Interrupt Service Request** (at **Figure 195**).
- If the DMA channel is disabled for hardware requests and a **DMA Hardware Request** is detected, the DMA shall set the DMA channel TRL bit (TSR.TRL = 1_B) and trigger a **DMA RP Error Interrupt Service Request**.

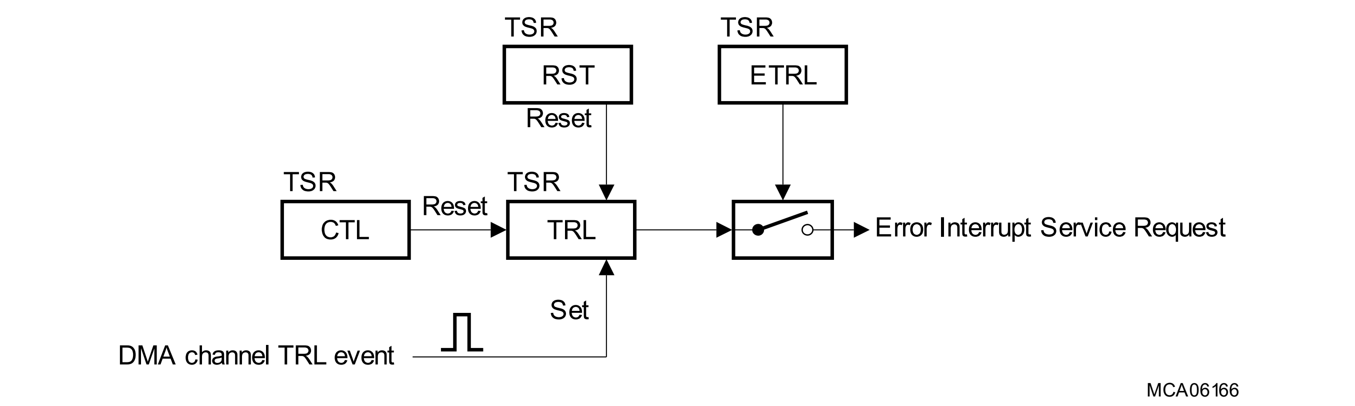

Figure 195 DMA Channel Transaction Request Lost Interrupt Service Request

An **Error Handler** shall interrogate the DMA channels to identify the source of the error. Software may clear TSR.TRL by setting the DMA channel TSR.CTL or TSR.RST.

18.3.3.7 DMA Service Requests

Interrupt Requests are prioritized by the Interrupt Router and processed by one of the Service Providers (CPU or DMA). The DMA interfaces to an Interrupt Control Unit (ICU) instantiated in the Interrupt Router (IR).

DMA channels are associated with the Service Request Priority Number (SRPN) bit field programmed in the Service Request Control (SRC) Register SRC.SRPN. For example:

- DMA channel 000 equates to SRC.SRPN = 0_D programmed in IR.
- DMA channel 001 equates to SRC.SRPN = 1_D programmed in IR.
- DMA channel 002 equates to SRC.SPRN = 2_D programmed in IR.
- DMA channel 003 equates to SRC.SRPN = 3_D programmed in IR.
- DMA channel 004 equates to SRC.SPRN = 4_D programmed in IR.

The routing of a hardware service request to a service provider destination is determined by the IR Type Of Service (TOS) control bit field SRC.TOS. The DMA will acknowledge all service requests. If the value programmed in the SRC.SRPN is for an invalid DMA channel then the DMA will take no action. The user must programme valid SRC.SRPN values for the DMA.

18.3.3.8 DMA Request Arbitration

The DMA arbiter continuously monitors all DMA channels for a pending **DMA Request** when the DMA channel is not in the halt state (TSR.HLTREQ = 1_B) and/or the suspend state (SUSASR.SUSAC = 1_B).

The highest number DMA channel with a pending **DMA Request** wins the DMA channel arbitration. The pending DMA request is forwarded to the highest number available ME. The ME reads the DMA channel TCS from the DMARAM and loads the TCS into the ME active channel register set and executes a DMA transfer. On completion of each DMA transfer the DMA performs an **Arbitration Sequence**.

Arbitration Sequence

- **Rule 1:** If there is a higher priority DMA channel with a pending **DMA Request** then
  - **Rule 1.1**: The ME shall write back the active DMA channel updated TCS to the DMARAM.
  - **Rule 1.2**: The ME reads the higher priority DMA channel TCS from the DMARAM to the ME.
  - **Rule 1.3**: The ME shall execute a DMA transfer.
  - **Rule 1.4**: On completion of the DMA transfer, the DMA performs an **Arbitration Sequence**.
- **Rule 2**: If there is not a higher priority DMA channel pending **DMA Request** and for the ME active DMA channel CHCFGR.RROAT = 0_B then
  - **Rule 2.1**: The updated TCS shall be written back from the ME to the DMARAM.
  - **Rule 2.2**: During each clock cycle the DMA shall perform an **Arbitration Sequence**.
  - **Rule 2.3**: If a **DMA Request** is received then the ME shall execute a DMA transfer.
  - **Rule 2.4**: On completion of the DMA transfer, the DMA shall perform an **Arbitration Sequence**.
- **Rule 3**: If there is not a higher priority DMA channel pending **DMA Request** and for the ME active DMA channel CHCFGR.RROAT = 1_B then
  - **Rule 3.1**: The ME shall execute a DMA transfer.
  - **Rule 3.2**: On completion of the DMA transfer, the DMA shall perform an **Arbitration Sequence**.
- **Rule 4**: On completion of the last DMA transfer, the DMA channel request bit is cleared (TSR.CH = 0_B).
- **Rule 5**: The **Arbitration Sequence** continues until all DMA channel access pending requests are serviced by ME.

18.3.3.9 DMA Channel Reset

Software shall reset an individual DMA channel by setting the DMA channel reset bit (TSR.RST = 1_B). When a **DMA Channel Reset** is applied to a DMA channel, the transition to the **Reset State** (TSR.RST = 0_B) is as follows:

- **Idle State** and **Pending State**: the DMA channel shall transition to the **Reset State**.
- **Active State**: on completion of the current DMA transfer, the DMA channel transitions to the **Reset State**.

Reset State

On completion of a **DMA Channel Reset** the DMA channel enters the **Reset State** defined as:

- The following DMA channel bits are reset:
  - DMA Transaction State Register: TSR.HLTREQ, TSR.HLTACK, TSR.HTRE, TSR.CH and TSR.TRL.
  - DMARAM TCS: CHCFGR.PRSEL, CHCSR.ICH, CHCSR.IPM, CHCSR.WRPD, CHCSR.WRPS, CHCSR.FROZEN, CHCSR.BUFFER, CHCSR.LXO and CHCHSR.TCOUNT.
- If a circular buffer ADICR.SCBE and/or ADICR.DCBE is enabled for the DMA channel then the source and/or destination address register will be set to the wrap boundary else the address registers are cleared.
- DMA channel shadow address register (SHADR) shall be cleared.

Resetting a DMA Channel

A user program must execute the following steps to reset a DMA channel:

1. If a **DMA Hardware Request** is enabled then disable hardware requests (TSR.DCH = 1_B).
2. Software requests a DMA channel reset (TSR.RST = 1_B).
3. Software shall monitor the DMA channel reset and the DMA channel SADR, DADR and SHADR registers.
4. As soon as the DMA has cleared the DMA channel reset (TSR.RST = 0_B) and the DMA has reset the DMA channel SADR, DADR and SHADR registers, the **DMA Channel Reset** has completed.

During a **DMA Channel Reset** operation, a **DMA Software Request** must not be initiated by setting CHCSR.SCH = 1_B.

Restarting a DMA Channel

A user program must execute the following steps to restart a DMA channel after a **DMA Channel Reset**:

1. Configure the DMA channel TCS including:
   a) DMA double buffering: program address pointers to the base address.
2. The DMA channel shall be restarted as follows:
   a) **DMA Software Request**: software initiates a DMA request (DMA channel CHCSR.SCH = 1_B).
   b) **DMA Hardware Request**: software enables a DMA request (DMA channel TSR.ECH = 1_B).

18.3.3.10 DMA Channel Halt

A DMA channel may be halted during a DMA transaction and the state frozen to allow a background RAM test to be run over a destination memory in order to detect stuck bits and distinguish between static errors and transient errors. On completion of the RAM test the DMA channel may be re-started and the DMA transaction completed.

The DMA channel halt logic (at **Figure 196**) utilizes a set/clear mechanism to request the DMA channel transitions to and from the HALT state on completion of the current DMA transfer. Only writing a logic '1' to set or clear a halt request (at **Figure 197**) to a DMA channel has an effect. The status of other DMA channels shall be ignored.

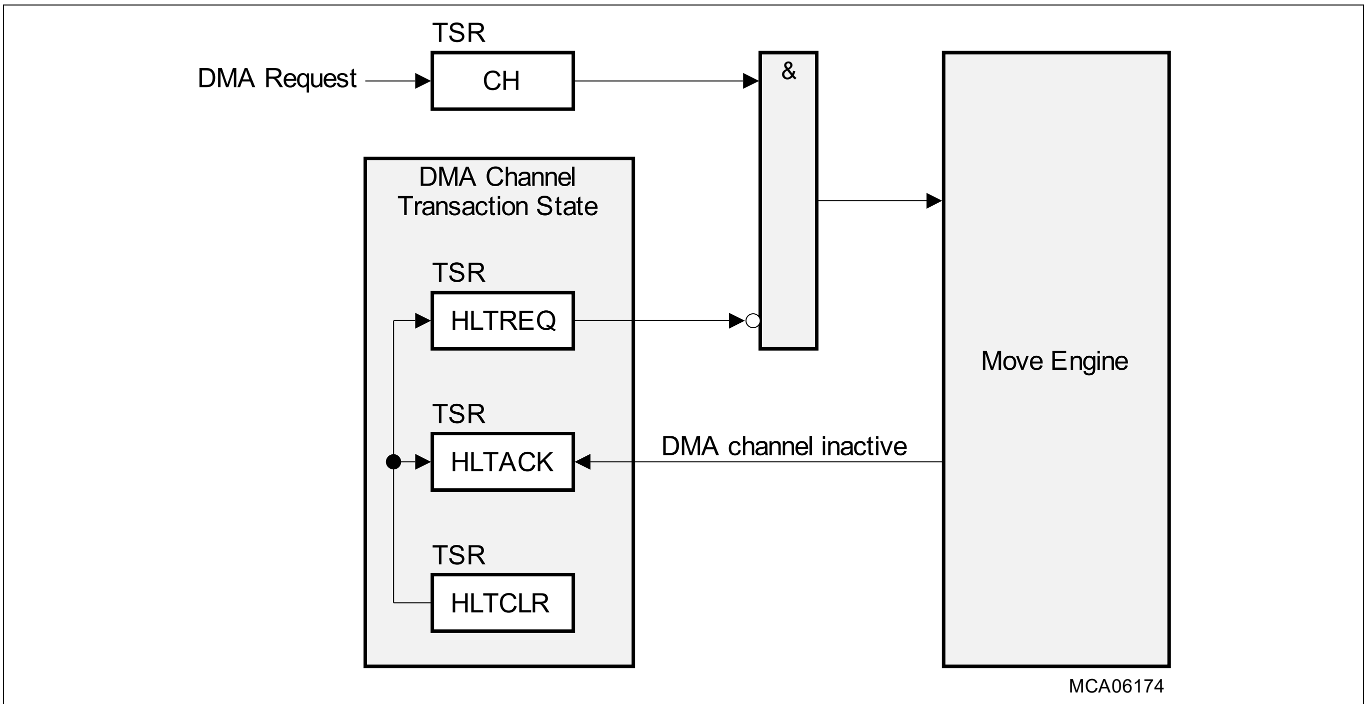

Figure 196 DMA Channel Halt Logic

Halt State

A DMA channel is defined to be in the **Halt State** when DMA channel TSR.HLTACK = 1_B.

Entering DMA Channel Halt

A DMA channel shall be halted by software writing the DMA channel halt request bit (TSR.HLTREQ = 1_B). The DMA channel enters the **Halt State** as follows:

- **Idle State**, **Reset State** and **Pending State**: as soon as the DMA channel receives the halt request, the DMA channel shall transition to the **Halt State**.
- **Active State**: on completion of the current DMA transfer, the DMA channel shall transition to the **Halt State**.

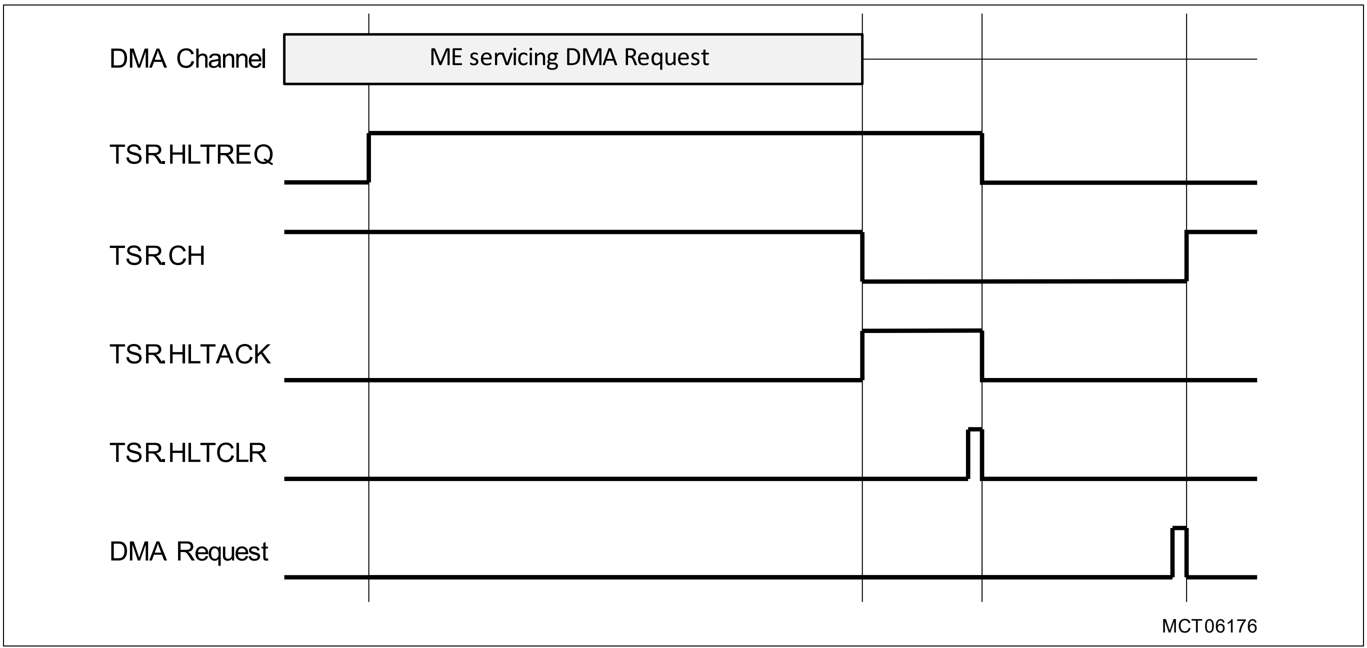

Figure 197 DMA Channel Halt Operation

Exiting DMA Channel Halt

A DMA channel is released from the **Halt State** by software writing the DMA channel halt clear bit (TSR.HLTCLR = 1_B). The DMA operation is resumed. If the halt request is cleared before it is acknowledged then there is no effect on DMA operation.

DMA Channel Hardware Request during DMA Channel Halt

If a DMA channel is in the **Halt State** and Hardware Transaction Request is enabled (TSR.HTRE = 1_B) then the DMA channel will respond to a **DMA Hardware Request** as follows:

- No DMA request pending (TSR.CH = 0_B): the DMA channel shall set the access pending bit (TSR.CH = 1_B). The **DMA Hardware Request** shall be serviced when DMA channel exits the **Halt State**.
- DMA request pending (TSR.CH = 1_B): the DMA channel shall record a TRL event. If the DMA channel TRL enable bit is set (TSR.ETRL = 1_B), then the DMA shall trigger a **DMA RP Error Interrupt Service Request**.

18.3.4 DMA Random Access Memory

Software stores a DMA channel TCS (at **Figure 198**) in the DMARAM to define the move function of each DMA channel.

Bit	WORD 0	WORD 1	WORD 2	WORD 3	WORD 4	WORD 5	WORD 6	WORD 7
	RDCRCR	SDCRCR	SADR	DADR	ADICR	CHCFGR	SHADR	CHCSR
31	RDCRC[31]	SDCRC[31]	SADR[31]	DADR[31]	IRDV[3]	Reserved	SHADR[31]	SCH
30	RDCRC[30]	SDCRC[30]	SADR[30]	DADR[30]	IRDV[2]	Reserved	SHADR[30]	Reserved
29	RDCRC[29]	SDCRC[29]	SADR[29]	DADR[29]	IRDV[1]	Reserved	SHADR[29]	Reserved
28	RDCRC[28]	SDCRC[28]	SADR[28]	DADR[28]	IRDV[0]	PRSEL	SHADR[28]	Reserved
27	RDCRC[27]	SDCRC[27]	SADR[27]	DADR[27]	INTCT[1]	SWAP	SHADR[27]	SIT
26	RDCRC[26]	SDCRC[26]	SADR[26]	DADR[26]	INTCT[0]	PATSEL[2]	SHADR[26]	CICH
25	RDCRC[25]	SDCRC[29]	SADR[25]	DADR[25]	WRPDE	PATSEL[1]	SHADR[25]	CWRP
24	RDCRC[24]	SDCRC[28]	SADR[24]	DADR[24]	WRPSE	PATSEL[0]	SHADR[24]	SWB
23	RDCRC[23]	SDCRC[27]	SADR[23]	DADR[23]	Reserved	CHDW[2]	SHADR[23]	FROZEN
22	RDCRC[22]	SDCRC[26]	SADR[22]	DADR[22]	STAMP	CHDW[1]	SHADR[22]	BUFFER
21	RDCRC[21]	SDCRC[21]	SADR[21]	DADR[21]	DCBE	CHDW[0]	SHADR[21]	Reserved
20	RDCRC[20]	SDCRC[20]	SADR[20]	DADR[20]	SCBE	CHMODE	SHADR[20]	Reserved
19	RDCRC[19]	SDCRC[19]	SADR[19]	DADR[19]	SHCT[3]	RROAT	SHADR[19]	IPM
18	RDCRC[18]	SDCRC[18]	SADR[18]	DADR[18]	SHCT[2]	BLKM[2]	SHADR[18]	ICH
17	RDCRC[17]	SDCRC[17]	SADR[17]	DADR[17]	SHCT[1]	BLKM[1]	SHADR[17]	WRPD
16	RDCRC[16]	SDCRC[16]	SADR[16]	DADR[16]	SHCT[0]	BLKM[0]	SHADR[16]	WRPS
15	RDCRC[15]	SDCRC[15]	SADR[15]	DADR[15]	CLBD[3]	Reserved	SHADR[15]	LXO
14	RDCRC[14]	SDCRC[14]	SADR[14]	DADR[14]	CLBD[2]	Reserved	SHADR[14]	Reserved
13	RDCRC[13]	SDCRC[13]	SADR[13]	DADR[13]	CLBD[1]	TREL[13]	SHADR[13]	TCOUNT[13]
12	RDCRC[12]	SDCRC[12]	SADR[12]	DADR[12]	CLBD[0]	TREL[12]	SHADR[12]	TCOUNT[12]
11	RDCRC[11]	SDCRC[11]	SADR[11]	DADR[11]	CLBS[3]	TREL[11]	SHADR[11]	TCOUNT[11]
10	RDCRC[10]	SDCRC[10]	SADR[10]	DADR[10]	CLBS[2]	TREL[10]	SHADR[10]	TCOUNT[10]
9	RDCRC[9]	SDCRC[9]	SADR[9]	DADR[9]	CLBS[1]	TREL[9]	SHADR[9]	TCOUNT[9]
8	RDCRC[8]	SDCRC[8]	SADR[8]	DADR[8]	CLBS[0]	TREL[8]	SHADR[8]	TCOUNT[8]
7	RDCRC[7]	SDCRC[7]	SADR[7]	DADR[7]	INCD	TREL[7]	SHADR[7]	TCOUNT[7]
6	RDCRC[6]	SDCRC[6]	SADR[6]	DADR[6]	DMF[2]	TREL[6]	SHADR[6]	TCOUNT[6]
5	RDCRC[5]	SDCRC[5]	SADR[5]	DADR[5]	DMF[1]	TREL[5]	SHADR[5]	TCOUNT[5]
4	RDCRC[4]	SDCRC[4]	SADR[4]	DADR[4]	DMF[0]	TREL[4]	SHADR[4]	TCOUNT[4]
3	RDCRC[3]	SDCRC[3]	SADR[3]	DADR[3]	INCS	TREL[3]	SHADR[3]	TCOUNT[3]
2	RDCRC[2]	SDCRC[2]	SADR[2]	DADR[2]	SMF[2]	TREL[2]	SHADR[2]	TCOUNT[2]
1	RDCRC[1]	SDCRC[1]	SADR[1]	DADR[1]	SMF[1]	TREL[1]	SHADR[1]	TCOUNT[1]
0	RDCRC[0]	SDCRC[0]	SADR[0]	DADR[0]	SMF[0]	TREL[0]	SHADR[0]	TCOUNT[0]

Key: Read/Write | Read Only | Write Only

MCA06180

Figure 198 DMARAM TCS Organization

18.3.4.1 DMA Channel Operation

The DMARAM TCS defines the type of DMA channel operation and the supported on chip bus access size.

Table 569 DMA Channel Operation

PATSEL	STAMP	SHCT	Description	BTR4	BTR2	SDTD	SDTW	SDTH	SDTB
000_B	0_B	0000_B	Move Operation	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	0001_B	Shadow Operation Read Only Mode Source Address	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	0010_B	Shadow Operation Read Only Mode Destination Address	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	0101_B	Shadow Operation Direct Write Mode Source Address	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	0110_B	Shadow Operation Direct Write Mode Destination Address	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	1000_B	DMA Double Source Buffering Software Switch Only	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	1001_B	DMA Double Source Buffering Software Switch and Automatic Hardware Switch	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	1010_B	DMA Double Destination Buffering Software Switch Only	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	1011_B	DMA Double Destination Buffering Software Switch and Automatic Hardware Switch	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	1100_B	DMA Linked List (DMALL)	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	1101_B	Accumulated Linked List (ACCLL)	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	1110_B	Safe Linked List (SAFLL)	Yes	Yes	Yes	Yes	Yes	Yes
000_B	0_B	1111_B	Conditional Linked List (CONLL) DMA read move data compared to PRR0	No	No	No	No	No	Yes
100_B	0_B	1111_B	Conditional Linked List (CONLL) DMA read move data compared to PRR1	No	No	No	No	No	Yes
000_B	1_B	0000_B	Move Operation with DMA Timestamp	Yes	Yes	Yes	Yes	Yes	Yes
000_B	1_B	0001_B	Shadow Operation Read Only Mode Source Address with DMA Timestamp	Yes	Yes	Yes	Yes	Yes	Yes
000_B	1_B	0010_B	Shadow Operation Read Only Mode Destination Address with DMA Timestamp	Yes	Yes	Yes	Yes	Yes	Yes
000_B	1_B	0101_B	Shadow Operation Direct Write Mode Source Address with DMA Timestamp	Yes	Yes	Yes	Yes	Yes	Yes
000_B	1_B	0110_B	Shadow Operation Direct Write Mode Destination Address with DMA Timestamp	Yes	Yes	Yes	Yes	Yes	Yes
000_B	1_B	1001_B	DMA Double Source Buffering with DMA Timestamp[1)] Software Switch and Automatic Hardware Switch	Yes	Yes	Yes	Yes	Yes	Yes
000_B	1_B	1011_B	DMA Double Destination Buffering with DMA Timestamp[1)] Software Switch and Automatic Hardware Switch	Yes	Yes	Yes	Yes	Yes	Yes

Table 569 DMA Channel Operation (cont'd)

PATSEL	STAMP	SHCT	Description	BTR4	BTR2	SDTD	SDTW	SDTH	SDTB
000_B	1_B	1100_B	**DMA Linked List (DMALL) with DMA Timestamp**	Yes	Yes	Yes	Yes	Yes	Yes
000_B	1_B	1101_B	**Accumulated Linked List (ACCLL) with DMA Timestamp**	Yes	Yes	Yes	Yes	Yes	Yes
000_B	1_B	1110_B	**Safe Linked List (SAFLL) with DMA Timestamp**	Yes	Yes	Yes	Yes	Yes	Yes
001_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 8-bit Channel Data Width** **Lower Pattern Compare to PRR0**	No	No	No	No	No	Yes
010_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 8-bit Channel Data Width** **Upper Pattern Compare to PRR0**	No	No	No	No	No	Yes
011_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 8-bit Channel Data Width** **Two Byte Pattern Detection Sequence compared to PRR0**	No	No	No	No	No	Yes
001_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 16-bit Channel Data Width** **Aligned Mode compared to PRR0**	No	No	No	No	Yes	No
010_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 16-bit Channel Data Width** **Unaligned Mode 1 or Unaligned Mode 2 compared to PRR0**	No	No	No	No	Yes	No
011_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 16-bit Channel Data Width** **Combined Mode compared to PRR0**	No	No	No	No	Yes	No
001_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 32-bit Channel Data Width** **Lower Data Half Word compared to PRR0**	No	No	No	Yes	No	No
010_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 32-bit Channel Data Width** **Upper Data Half Word compared to PRR0**	No	No	No	Yes	No	No
011_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 32-bit Channel Data Width** **Complete Data Word compared to PRR0**	No	No	No	Yes	No	No
101_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 8-bit Channel Data Width** **Lower Pattern Compare to PRR1**	No	No	No	No	No	Yes
110_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 8-bit Channel Data Width** **Upper Pattern Compare to PRR1**	No	No	No	No	No	Yes
111_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 8-bit Channel Data Width** **Two Byte Pattern Detection Sequence compared to PRR1**	No	No	No	No	No	Yes
101_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 16-bit Channel Data Width** **Aligned Mode compared to PRR1**	No	No	No	No	Yes	No

Table 569 DMA Channel Operation (cont'd)

PATSEL	STAMP	SHCT	Description	BTR4	BTR2	SDTD	SDTW	SDTH	SDTB
110_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 16-bit Channel Data Width** **Unaligned Mode 1 or Unaligned Mode 2 compared to PRR1**	No	No	No	No	Yes	No
111_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 16-bit Channel Data Width** **Combined Mode compared to PRR1**	No	No	No	No	Yes	No
101_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 32-bit Channel Data Width** **Lower Data Half Word compared to PRR1**	No	No	No	Yes	No	No
110_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 32-bit Channel Data Width** **Upper Data Half Word compared to PRR1**	No	No	No	Yes	No	No
111_B	0_B	0000_B	**Move Operation**[2)] **Pattern Detection for 32-bit Channel Data Width** **Complete Data Word compared to PRR1**	No	No	No	Yes	No	No
Others			**Reserved**[3)]	N/A	N/A	N/A	N/A	N/A	N/A

1) DMA timestamp shall only be appended on completion of a DMA transaction.

2) DMA channel operation "Pattern Detection" shall function for all types of DMA channel Shadow Operation.

3) If a DMA channel is configured for a reserved value of DMA channel ADICR.SHCT and a DMA request is received, the DMA clears DMA channel TSR.CH and performs no DMA moves.

18.3.4.2 DMA Channel Updates

Software shall only configure the DMARAM channel TCS when the DMA channel is in the **Idle State** (DMA channel CHCSR.TCOUNT=0_D) or **Reset State**. If the DMA channel is in the **Idle State** (DMA channel CHCSR.TCOUNT!=0_D), **Pending State** or **Active State**, software shall not update the DMARAM channel TCS. The following DMA channel update exceptions apply for **Shadow Operations** and **Double Buffering Operations**:

18.3.4.2.1 Shadow Operations

If the DMARAM channel TCS is configured for **Shadow Operation** and the DMA channel is in the **Idle State** (DMA channel CHCSR.TCOUNT!=0_D), **Pending State** or **Active State**, software shall be restricted to updating the DMARAM channel TCS words marked 'X':

Table 570 DMARAM channel TCS updates during Shadow Operation

SHCT	Description	CHCSR	SHADR	CHCFGR	ADICR	DADR	SADR	SDCRCR	RDCRCR
0001_B	**Shadow Operation Read Only Mode** **Source Address**						X		
0010_B	**Shadow Operation Read Only Mode** **Destination Address**					X			

Table 570 DMARAM channel TCS updates during Shadow Operation (cont'd)

SHCT	Description	CHCSR	SHADR	CHCFGR	ADICR	DADR	SADR	SDCRCR	RDCRCR
0101_B	**Shadow Operation Direct Write Mode Source Address**		X						
0110_B	**Shadow Operation Direct Write Mode Destination Address**		X						

18.3.4.2.2 Double Buffering Operations

If the DMARAM channel TCS is configured for **Double Buffering Operations** and the DMA channel is in the **Idle State** (DMA channel CHCSR.TCOUNT!=0_D), **Pending State** or **Active State**, software shall be restricted to updating the DMARAM channel TCS words marked 'X':

Table 571 DMARAM channel updates during Double Buffering Operations

SHCT	Description	CHCSR	SHADR	CHCFGR	ADICR	DADR	SADR	SDCRCR	RDCRCR
1000_B	**DMA Double Source Buffering Software Switch Only DMA channel CHCSR.BUFFER = 0_B**	X	X						
1000_B	**DMA Double Source Buffering Software Switch Only DMA channel CHCSR.BUFFER = 1_B**	X					X		
1001_B	**DMA Double Source Buffering Software Switch and Automatic Hardware Switch DMA channel CHCSR.BUFFER = 0_B**	X	X						
1001_B	**DMA Double Source Buffering Software Switch and Automatic Hardware Switch DMA channel CHCSR.BUFFER = 1_B**	X					X		
1010_B	**DMA Double Destination Buffering Software Switch Only DMA channel CHCSR.BUFFER = 0_B**	X	X						
1010_B	**DMA Double Destination Buffering Software Switch Only DMA channel CHCSR.BUFFER = 1_B**	X				X			
1011_B	**DMA Double Destination Buffering Software Switch and Automatic Hardware Switch DMA channel CHCSR.BUFFER = 0_B**	X	X						
1011_B	**DMA Double Destination Buffering Software Switch and Automatic Hardware Switch DMA channel CHCSR.BUFFER = 1_B**	X				X			

18.3.4.3 DMA Channel Reconfiguration

If a DMA channel is to be configured, software shall apply a **DMA Channel Reset** to initialize the DMA channel.

After the DMA channel has entered **Reset State**, software shall configure the DMA channel for a DMA operation.

18.3.4.4 Move Operation

The number of DMA moves (at **Figure 199**) may be calculated from the following:

- Block Mode (CHCFGR.BLKM) defines the number of DMA moves in a DMA transfer.
- Transfer Reload (CHCFGR.TREL) defines the number of DMA transfers in a DMA transaction.

After a DMA move, the next source and destination addresses are calculated. Source and destination addresses are calculated independently of each other. The following address calculation parameters may be selected:

- The address offset, which is a multiple of the selected data width.
- The offset direction: addition, subtraction, or none (unchanged address).

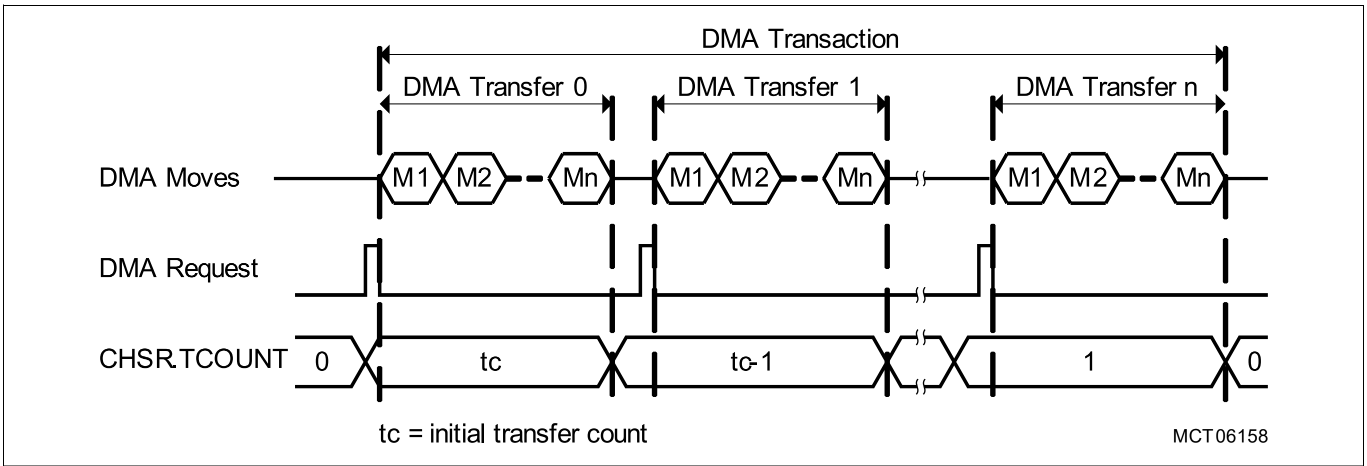

Figure 199 Block Mode and Transfer Count

18.3.4.4.1 Address Generation

Address control bits (in DMA channel ADICR) determine how the addresses increment or decrement. The data width is defined in CHCFGR.CHDW and is taken into account during the address calculation. The Address Offset Calculation Tables (at **Table 572** and **Table 573**) show the offset values that are added or subtracted to/from the source address (SMF and INCS parameters) and destination address (DMF and INCD parameters) after each DMA move.

Table 572 Address Offset Calculation Table (8-bit, 16-bit and 32-bit)

CHCFGR.CHDW = 000_B (8-bit Data Width)			CHCFGR.CHDW = 001_B (16-bit Data Width)			CHCFGR.CHDW = 010_B (32-bit Data Width)		
SMF DMF	**INCS INCD**	**Address Offset**	**SMF DMF**	**INCS INCD**	**Address Offset**	**SMF DMF**	**INCS INCD**	**Address Offset**
000_B	0_B	-1	000_B	0_B	-2	000_B	0_B	-4
	1_B	+1		1_B	+2		1_B	+4
001_B	0_B	-2	001_B	0_B	-4	001_B	0_B	-8
	1_B	+2		1_B	+4		1_B	+8
010_B	0_B	-4	010_B	0_B	-8	010_B	0_B	-16
	1_B	+4		1_B	+8		1_B	+16

Table 572 Address Offset Calculation Table (8-bit, 16-bit and 32-bit) (cont'd)

CHCFGR.CHDW = 000_B (8-bit Data Width)			CHCFGR.CHDW = 001_B (16-bit Data Width)			CHCFGR.CHDW = 010_B (32-bit Data Width)		
SMF DMF	**INCS INCD**	**Address Offset**	**SMF DMF**	**INCS INCD**	**Address Offset**	**SMF DMF**	**INCS INCD**	**Address Offset**
011_B	0_B	-8	011_B	0_B	-16	011_B	0_B	-32
	1_B	+8		1_B	+16		1_B	+32
100_B	0_B	-16	100_B	0_B	-32	100_B	0_B	-64
	1_B	+16		1_B	+32		1_B	+64
101_B	0_B	-32	101_B	0_B	-64	101_B	0_B	-128
	1_B	+32		1_B	+64		1_B	+128
110_B	0_B	-64	110_B	0_B	-128	110_B	0_B	-256
	1_B	+64		1_B	+128		1_B	+256
111_B	0_B	-128	111_B	0_B	-256	111_B	0_B	-512
	1_B	+128		1_B	+256		1_B	+512

Table 573 Address Offset Calculation Table (64-bit, 128-bit and 256-bit)

CHCFGR.CHDW = 011_B (64-bit Data Width)			CHCFGR.CHDW = 100_B (128-bit Data Width)			CHCFGR.CHDW = 101_B (256-bit Data Width)		
SMF DMF	**INCS INCD**	**Address Offset**	**SMF DMF**	**INCS INCD**	**Address Offset**	**SMF DMF**	**INCS INCD**	**Address Offset**
000_B	0_B	-8	000_B	0_B	-16	000_B	0_B	-32
	1_B	+8		1_B	+16		1_B	+32
001_B	0_B	-16	001_B	0_B	-32	001_B	0_B	-64
	1_B	+16		1_B	+32		1_B	+64
010_B	0_B	-32	010_B	0_B	-64	010_B	0_B	-128
	1_B	+32		1_B	+64		1_B	+128
011_B	0_B	-64	011_B	0_B	-128	011_B	0_B	-256
	1_B	+64		1_B	+128		1_B	+256
100_B	0_B	-128	100_B	0_B	-256	100_B	0_B	-512
	1_B	+128		1_B	+256		1_B	+512
101_B	0_B	-256	101_B	0_B	-512	101_B	0_B	-1024
	1_B	+256		1_B	+512		1_B	+1024
110_B	0_B	-512	110_B	0_B	-1024	110_B	0_B	-2048
	1_B	+512		1_B	+1024		1_B	+2048
111_B	0_B	-1024	111_B	0_B	-2048	111_B	0_B	-4096
	1_B	+1024		1_B	+2048		1_B	+4096

18.3.4.4.2 Address Calculation Examples

The following example demonstrate address generation for a data width of 16-bit (CHCFGR.CHDW = 001_B):

Programmable Address Generation - Example 1

16-bit half-words are moved from a source memory with an incrementing source address offset of 10_H to a destination memory with a decrementing destination address offset of 08_H.

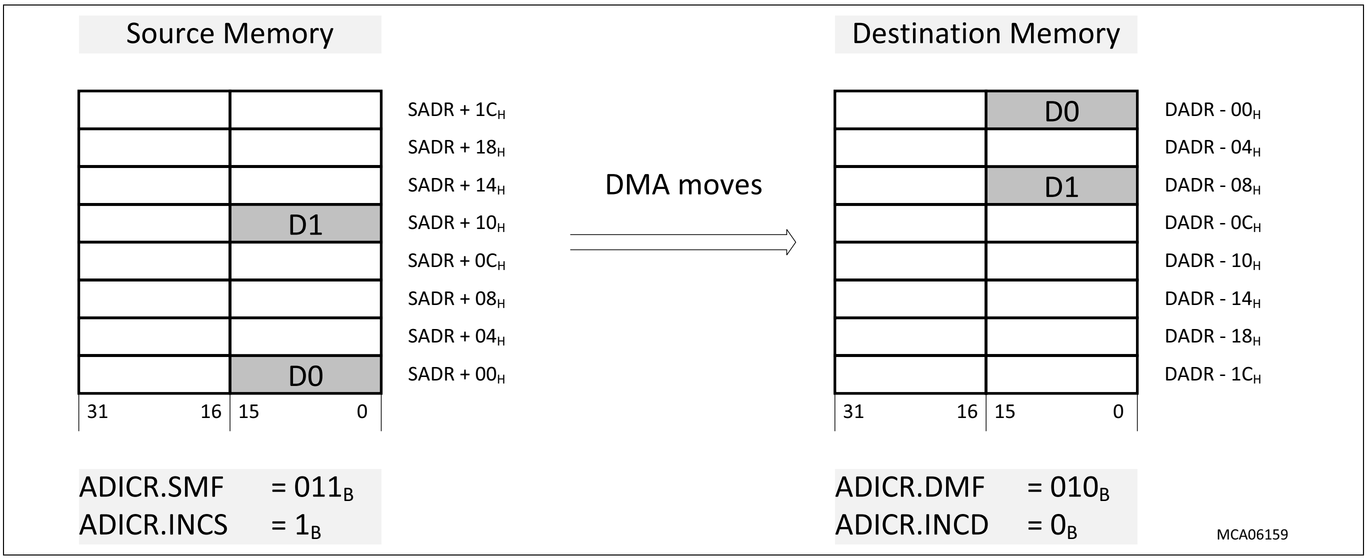

Figure 200 Programmable Address Generation - Example 1

Programmable Address Generation - Example 2

16-bit half-words are moved from a source memory with an incrementing source address offset of 02_H to a destination memory with an incrementing destination address offset of 04_H.

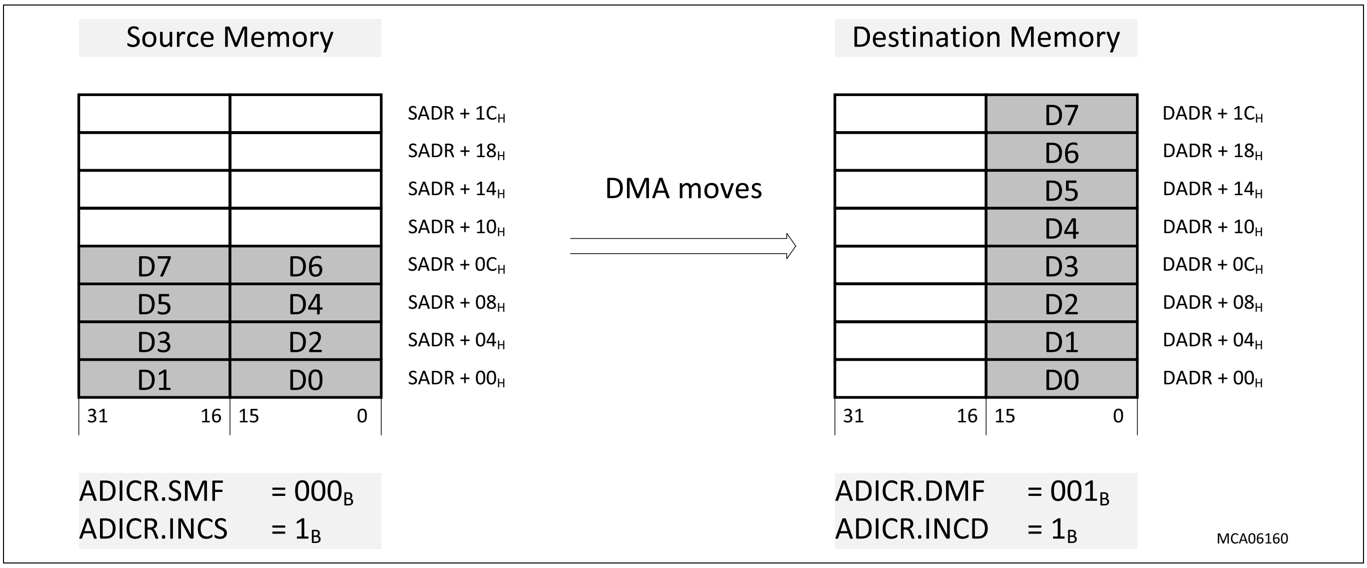

Figure 201 Programmable Address Generation - Example 2

18.3.4.4.3 Circular Buffer

The source and destination circular buffers are enabled by setting the DMA channel circular buffer enable bits ADICR.SCBE and ADICR.DCBE respectively. The source address and destination address may be configured to build a circular buffer separately for source and destination data. Within a circular buffer, the addresses are updated within the circular buffer wrap-around limits. The circular buffer length is determined by bit fields ADICR.CBLS (for the source buffer) and ADICR.CBLD (for the destination buffer). These 4-bit wide bit fields determine which bits of the 32-bit address remain unchanged during an address update. Possible buffer sizes of the circular buffers can be 2^{CBLS} or 2^{CBLD} bytes (= 1, 2, 4, 8, 16, … up to 64k bytes). Source and destination addresses are incremented or decremented during a DMA move, all upper bits [31:CBLS] of source address and [31:CBLD] of destination address are frozen and remain unchanged, even if a wrap-around from the lower address bits [CBLS-1:0] or [CBLD-1:0] occurred. This address-freezing mechanism always causes the circular buffers to be aligned to a multiple integer value of its size. If the circular buffer size is less than or equal to the selected address offset then the same circular buffer address will always be accessed.

18.3.4.4.4 Address Alignment

The DMA shall comply with the SRI bus protocol. All source addresses and destination addresses shall be aligned to the correct address boundary defined by the channel data width (CHCFGR.CHDW) as follows:

- Single transfers: the source address and destination address boundaries shall align with the channel data width (byte, half-word, word or double-word).
- Block transfers: the source address and destination address boundaries shall align to a double-word boundary. The SRI bus protocol defines a wrap around addressing scheme for block transfers not starting with a block transfer sized start address. The effect on a DMA transaction is demonstrated by different scenarios for BTR2 (at **Figure 202**) and BTR4 (at **Figure 203**).

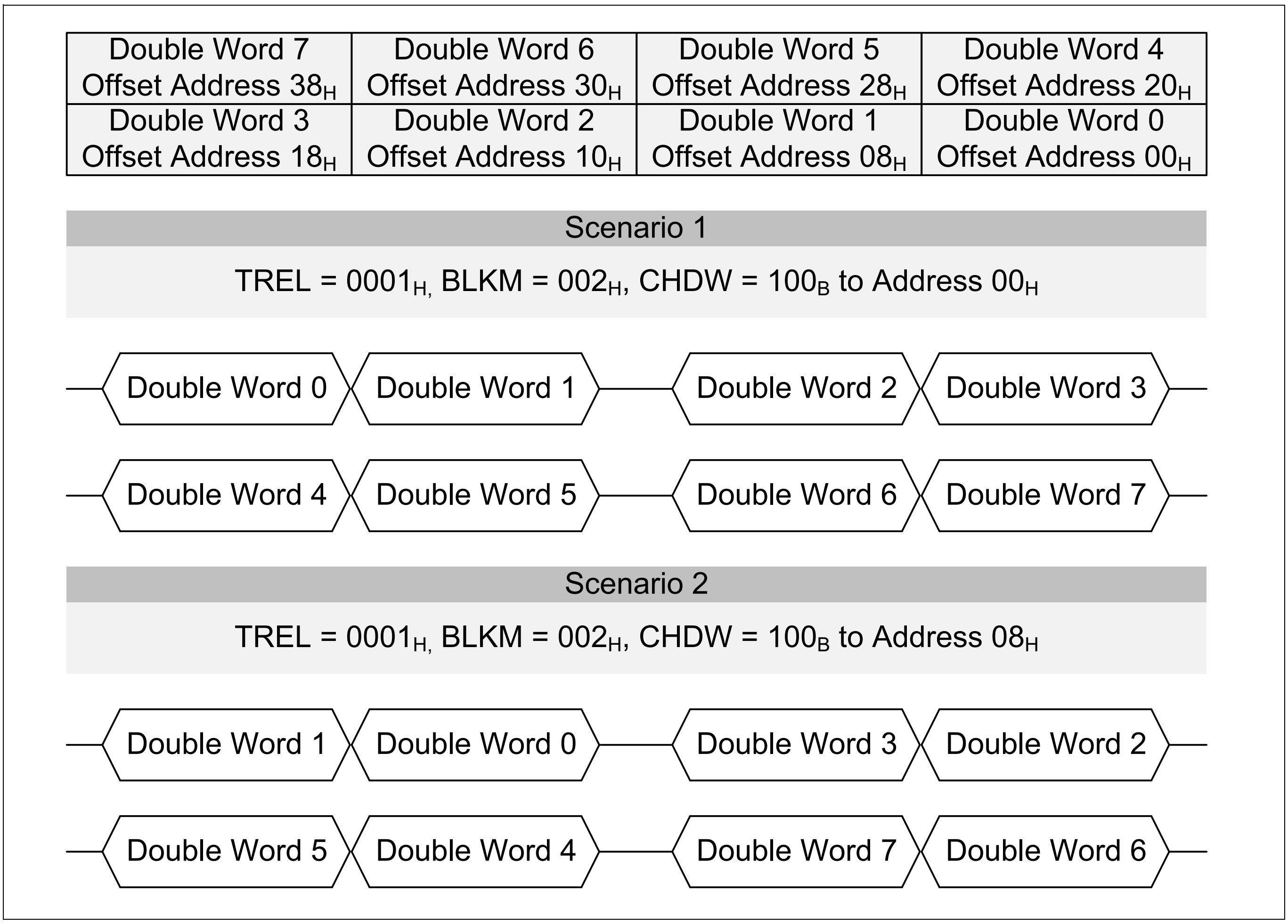

Figure 202 SRI Bus Protocol Block Transfer Request - 2 Transfers

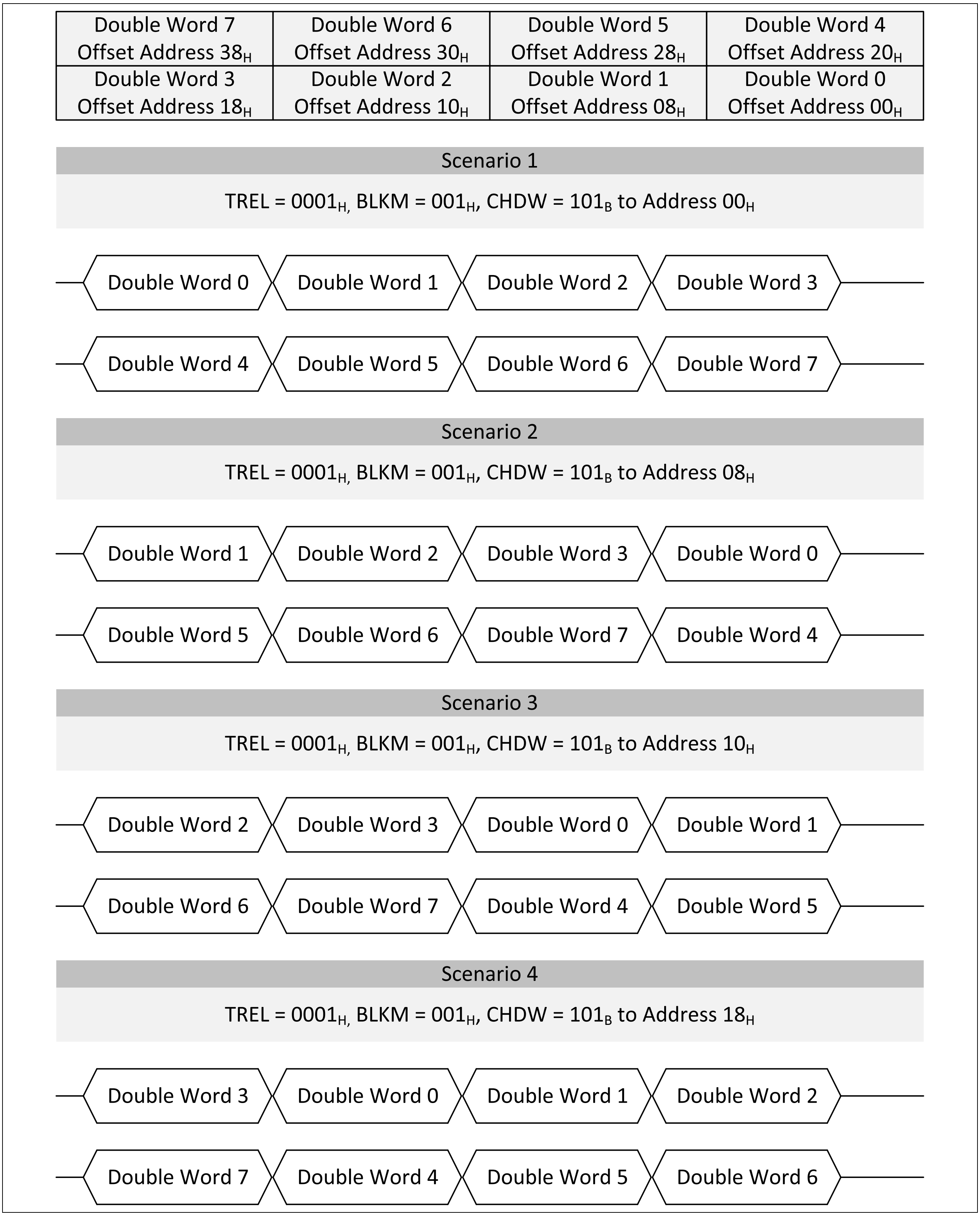

Figure 203 SRI Bus Protocol Block Transfer Request - 4 Transfers

18.3.4.4.5 Address Counter

If the source/destination circular buffer is not enabled (ADICR.SCBE/DCBE = 0_B) then the source/destination address will increment or decrement across the entire 32-bit address field. The address offset is determined by the DMA channel CHCFGR.CHDW. The address will wrap around on the 32-bit address boundary:

- If the address is incrementing then will wrap from $FFFFFFFF_H$ to 00000000_H.
- If the address is decrementing then will wrap from 00000000_H to $FFFFFFFF_H$.

18.3.4.4.6 DMA Address Checksum

The DMA shall calculate a **SDCRC** checksum in accordance with the IEEE 802.3 standard using the source and destination addresses presented to the on chip bus as input data. Software shall compare the calculated **DMA Address Checksum** (**SDCRC**) with an expected **DMA Address Checksum** stored in memory to validate the address generation during a DMA transaction. To check the DMA channel address generation use the following steps:

- Software shall initialize the DMA channel **DMA Address Checksum** stored in DAMRAM to a known initial value.
- The DMA channel receives a DMA request.
- If the DMA channel is in the **Active State** and no error or retry event is reported, the ME shall calculate an updated **DMA Address Checksum** for each DMA read move and DMA write move.
- On completion of the DMA transaction, the DMA shall store the final **DMA Address Checksum** in DMARAM.
- Software shall compare the calculated **DMA Address Checksum** and expected **DMA Address Checksum**.
  - If the calculated and expected **DMA Address Checksum match**, then the DMA has correctly calculated the source and destination address. If the checksums do not match, then software shall report a fault.

The **DMA Address Checksum** is not available for the following configurations of **DMA Channel Operation**:

- **DMA Double Source Buffering Software Switch** Only
- **DMA Double Source Buffering Software Switch** and **Automatic Hardware Switch**
- **DMA Double Destination Buffering Software Switch** Only
- **DMA Double Destination Buffering Software Switch** and **Automatic Hardware Switch**
- **Conditional Linked List (CONLL)**

18.3.4.4.7 DMA Channel Interrupt Service Request

Each DMA channel generates an interrupt service request (at **Figure 204**) to report DMA channel events:

- **DMA Channel Transfer Interrupt Service Request**
- **DMA Channel Pattern Match Interrupt Service Request**
- **DMA Channel Wrap Buffer Interrupt Service Request**

18.3.4.4.8 DMA Channel Transfer Interrupt Service Request

If a DMA channel is in the **Active State** then a DMA channel transfer interrupt service request may be activated on completion of a DMA transfer, or when ME CHSR.TCOUNT matches with the value of bit field ADICR.IRDV after it has been decremented after a DMA transfer. An interrupt service request from a DMA channel is indicated when status flag CSR.ICH is set. The status flag shall be reset by software setting DMA channel CHCSR.CICH = 1_B (or TSR.RST = 1_B). The DMA channel interrupt service request is enabled when DMA channel ADICR.INTCT[1] is set.

Bit ADICR.INTCT[0] selects one of two types of interrupt source. For the compare operation, bit field ADICR.IRDV (4-bit) is zero-extended to 14-bit and then compared with the 14-bit TCOUNT value. This means that a TCOUNT match interrupt may be generated after one of the last 16 DMA transfers of a DMA transaction. Note that with IRDV = 0000_B, the match interrupt is generated at the end of a DMA transaction (after the last DMA transfer).

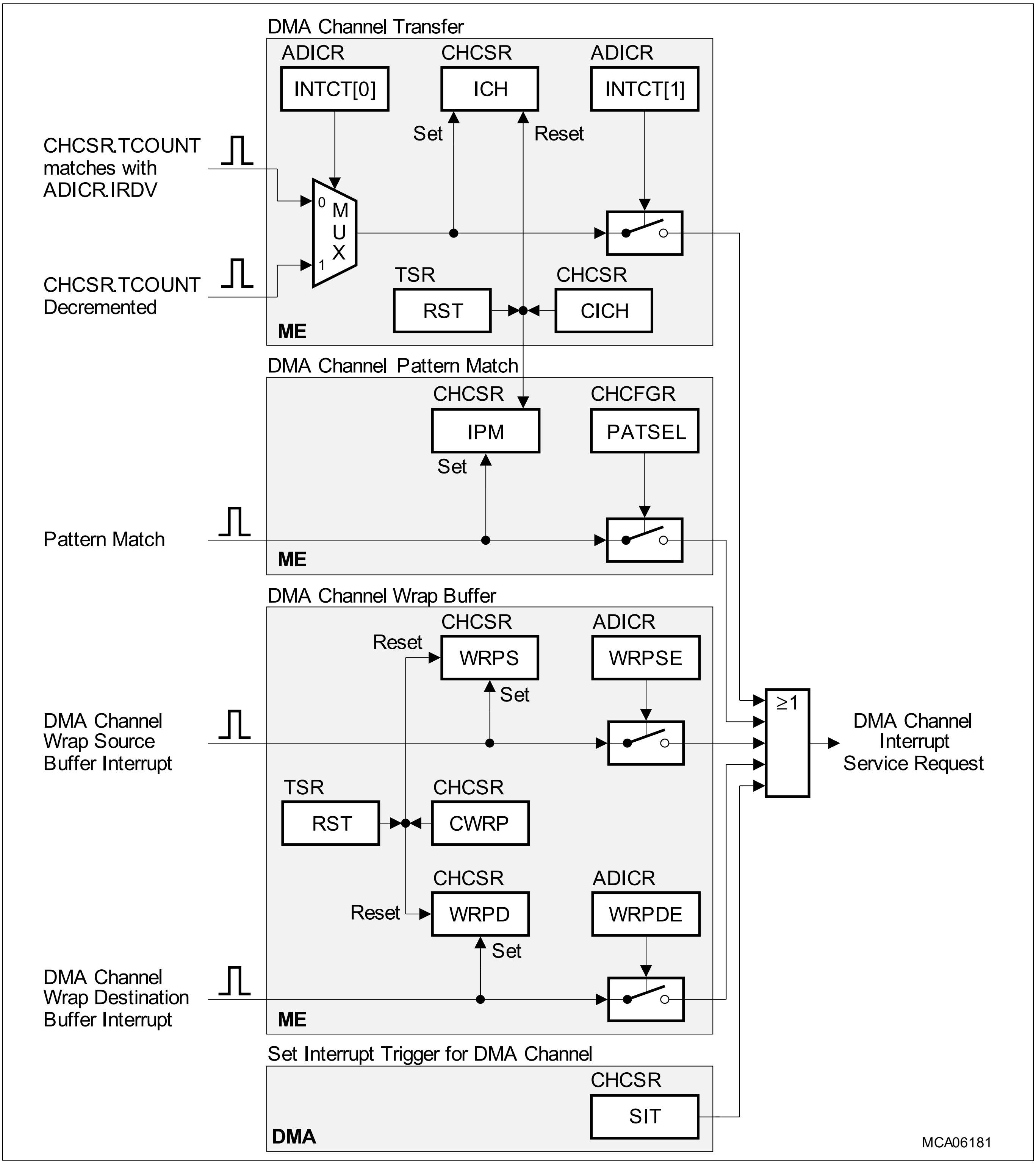

Figure 204 DMA Channel Interrupt Service Request

18.3.4.4.9 DMA Channel Pattern Match Interrupt Service Request

The DMA channel pattern detection interrupt service request is enabled when CHCFGR.PATSEL[1:0] is not equal to 00_B. The pattern detection interrupt is indicated when status flag CHCSR.IPM is set. The status flag CHCSR.IPM shall be reset by software setting bit DMA channel CHCSR.CICH = 1_B (or TSR.RST = 1_B).

18.3.4.4.10 DMA Channel Wrap Buffer Interrupt Service Request

Each DMA channel reports a source buffer wraparound and/or a destination buffer wraparound as follows:

- A wrap source buffer wraparound is indicated by status flag CHCSR.WRPS.
- A wrap destination buffer wraparound is indicated by the status flag CHCSR.WRPD.

Software may reset both buffer wraparound status flags by setting CHCSR.CWRP = 1_B (or TSR.RST = 1_B).

A DMA channel wrap buffer interrupt service request is enabled as follows:

- A DMA channel wrap source buffer interrupt service request is enabled when bit ADICR.WRPSE is set.
- A DMA channel wrap destination buffer interrupt service request is enabled when bit ADICR.WRPDE is set.

The DMA channel wrap source buffer wraparound interrupt service request and the DMA channel wrap destination buffer wraparound interrupt service request are OR-ed together with other DMA channel interrupt service requests to form one DMA channel interrupt service request.

18.3.4.5 Shadow Operation

If a DMA channel is configured for shadow operation, the DMA buffers a variable source or destination address.

18.3.4.5.1 Application of Shadow Operation

In a typical application, an ASC module that receives data (fixed DMA channel source address) has to deliver the data to a memory buffer using a DMA transaction (variable DMA channel destination address). After a certain amount of data has been transferred, a new DMA transaction should be initiated to deliver further ASC data into another memory buffer. While the destination address register is updated during a running DMA transaction with the actual destination address, a shadow mechanism allows programming of a new destination address without disturbing the content of the destination address register. In this case, the new destination address is written and buffered in the shadow address register. At the start of the next DMA transaction, the destination address register shall use the new address without CPU intervention. The shadow operation avoids the CPU having to check for the end of a DMA transaction before reprogramming address registers.

18.3.4.5.2 Shadowed Address Register

The shadow address register stores either a new source address, or a new destination address. If both the source address and destination address have to be updated for the next DMA transaction, a running DMA transaction for this DMA channel must be finished. After that, source and destination address registers may be written before the next DMA transaction is started.

Only one address register may be shadowed while a DMA transaction is running, because the shadow register can only be assigned either to the source address or to the destination address. Note that the shadow address transfer mechanism has the same behavior in **Single Mode** and **Continuous Mode**.

The function of the shadow address register shall be defined by the mode of the shadow operation:

- **Read Only Mode**: if software writes to the shadow address register, the DMA shall return a bus error.
- **Direct Write Mode**: if software writes to the shadow address register, the DMA shall store a new shadow address in the shadow address register.

18.3.4.5.3 Read Only Mode

If software writes a new DMA channel source or destination address value, the DMA shall store the new address value as follows:

- **Idle State** (DMA channel CHCSR.TCOUNT=0)

  - If software writes a new DMA channel source address value, the DMA shall store the new source address value in the DMA channel SADR.
  - If software writes a new DMA channel destination address value, the DMA shall store the new destination address value in the DMA channel DADR.
- **Idle State** (DMA channel CHCSR.TCOUNT!=0) or **Pending State**
  - If software writes a new DMA channel source or destination address value, the DMA shall store the new address value in the DMA channel SHADR.
- **Active State**
  - If software writes a new DMA channel source or destination address value, the DMA shall store the new address value in the ME SHADR.

End of DMA Transaction

As soon as the DMA completes a DMA transaction, the DMA shall check the value of ME SHADR.

If ME SHADR is not equal to 0000 0000_H, the ME shall write back data to the DMARAM as follows:

- Shadow Operation Read Only Mode Source Address
  - The DMA shall store the ME SHADR value into DMA channel SADR.
  - The DMA shall store 0000 0000_H into DMA channel SHADR.
- Shadow Operation Read Only Mode Destination Address
  - The DMA shall store the ME SHADR value into DMA channel DADR.
  - The DMA shall store 0000 0000_H into DMA channel SHADR.

If software has not executed a shadow address update, ME SHADR is equal to 0000 $0000_{H.}$ The ME shall write back data to the DMARAM as follows:

- Shadow Operation Read Only Mode Source Address
  - The DMA shall store 0000 0000_H into DMA channel SADR (see **Error Conditions**).
- Shadow Operation Read Only Mode Destination Address
  - The DMA shall store 0000 0000_H into DMA channel DADR (see **Error Conditions**).

18.3.4.5.4 Direct Write Mode

If software writes a new DMA channel shadow address value, the DMA shall store the new shadow address value as follows:

- **Idle State** or **Pending State**
  - If software writes a new DMA channel shadow address value, the DMA shall store the new shadow address value in the DMA channel SHADR.
- **Active State**
  - If software writes a new DMA channel shadow address value, the DMA shall store the new shadow address value in the ME SHADR. As soon as the ME shall write back data to the DMA RAM, the DMA shall store the ME SHADR value in the DMA channel SHADR word.

Start of DMA Transaction

As soon as the DMA services a DMA request, the DMA shall perform a shadow address transfer as follows:

- Shadow Operation Direct Write Mode Source Address
  - If DMA channel SHADR is not 0000 0000_H, the DMA shall store the DMA channel SHADR value into ME SADR.
- Shadow Operation Direct Write Mode Destination Address

– If DMA channel SHADR is not 0000 0000_H, the DMA shall store the DMA channel SHADR value into ME DADR.

End of Shadow Operation

If software writes a new DMA channel shadow address value of 0000 0000_H or applies a **DMA Channel Reset**, the DMA shall not perform a shadow address update at the start of the next DMA transaction.

18.3.4.5.5 Error Conditions

If a DMA channel is configured for **Shadow Operation** and a new DMA request is serviced before software updates the shadow address, the DMA shall perform a DMA move to address 0000 0000_H resulting in a bus error.

18.3.4.5.6 Transfer Count Update

The transfer count of a DMA transaction, stored in the DMA channel bit field CHCFGR.TREL, may be programmed if a DMA transaction is running. At the start of a DMA transaction, CHCFGR.TREL is transferred to the ME CHSR.TCOUNT. No reload of address or counter will be done if ME CHSR.TCOUNT is not equal to 0.

The reprogramming of channel specific values (except for the selected shadow address register) must be avoided while a DMA channel is active as the data transfer maybe corrupted.

Transfer Count and Source Address Update - Example

The contents of the ME transfer count and the ME source address register are updated (at **Figure 205**) during two DMA transactions with a transfer count and a shadowed source address update.

At reference point 2) the DMA transaction 1 is finished and DMA transaction 2 is started. At 1) the DMA channel is reprogrammed with two new parameters for the next DMA transaction: Transfer count tc2 and source address sa2. Source address sa2 is buffered in ME SHADR and transferred to ME SADR when the new DMA transaction is started at 2). At this time, transfer count tc2 is also transferred to ME CHSR.TCOUNT. Note that the shadow address register is only reset by hardware to 0000 0000_H as shown in this example, if Read Only Mode is selected.

In the event that DMA channel CHCFGR.TREL is written while DMA channel is active:

- A write to DMA channel CHCFGR.TREL will update the ME CHCR.TREL value.
- On write back DMA channel CHCFGR.TREL is updated with the latest ME CHCR.TREL value.
- ME CHSR.TCOUNT is updated to the new DMA channel CHCFGR.TREL value at the start of the next DMA transaction.

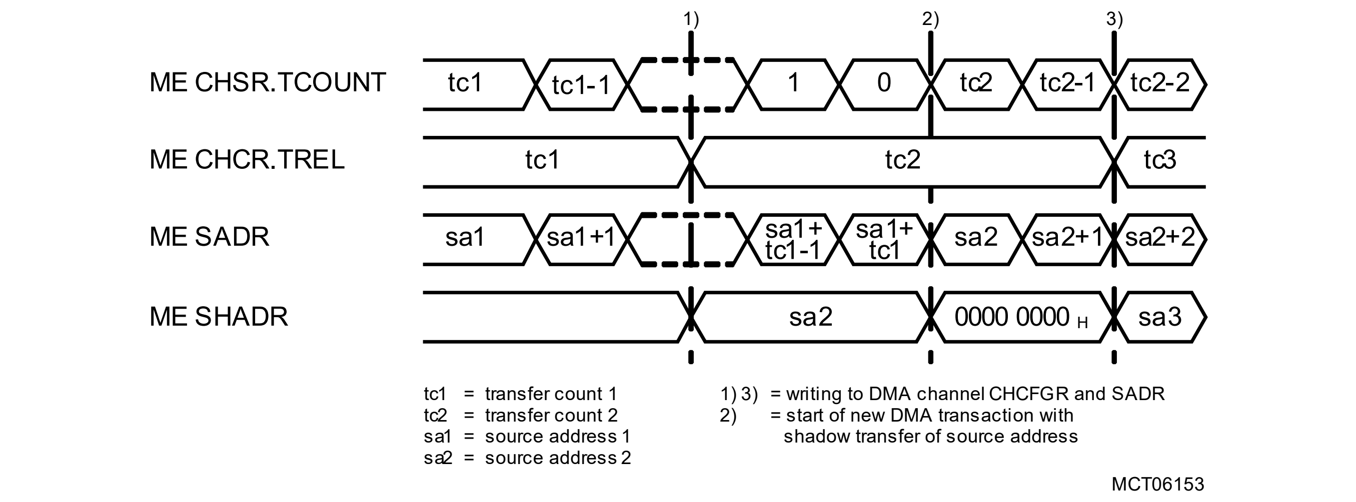

Figure 205 Shadow Operation Read Only Mode Source Address: Transfer Count and Source Address Update

18.3.4.6 DMA Timestamp

As soon as the last DMA transfer has completed, a 32-bit DMA timestamp may be appended to record the occurrence of a DMA operation. The DMA timestamp is the time at which the event was recorded by the DMA and not the real time. The DMA transaction completes after the DMA timestamp is written.

18.3.4.6.1 Generation of DMA Timestamp

The DMA timestamp is generated from a 32-bit binary upwards synchronous counter clocked by the SPB clock divided by 8 as shown in **Figure 206**. The counting starts automatically after the application reset is released. It is not possible to affect the counter during normal DMA operation. The current 32-bit timestamp value may be read through the TIME register.

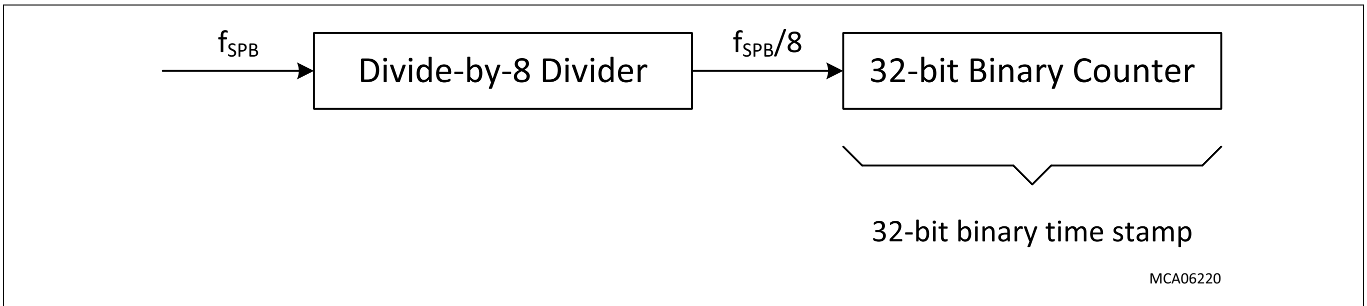

Figure 206 Generation of DMA Timestamp

18.3.4.6.2 Appendage of DMA Timestamp to Non Destination Circular Buffer

The appendage of a DMA timestamp to the destination data is controlled by the DMA channel ADICR.STAMP bit.

If a DMA channel is configured for the appendage of a DMA timestamp and not destination circular buffer operation (ADICR.DCBE = 0_B), the ME shall write a DMA timestamp at a destination address defined by:

- If ADICR.INCD = 1_B, the address of the DMA timestamp is the next higher word aligned destination address.
- If ADICR.INCD = 0_B, the address of the DMA timestamp is the next lower word aligned destination address.

Appendage of DMA Timestamp - Example 1

16-bit half-words are moved from a source memory with an incrementing source address offset of 10_H to a destination memory with an incrementing destination address offset of 08_H (at **Figure 207**).

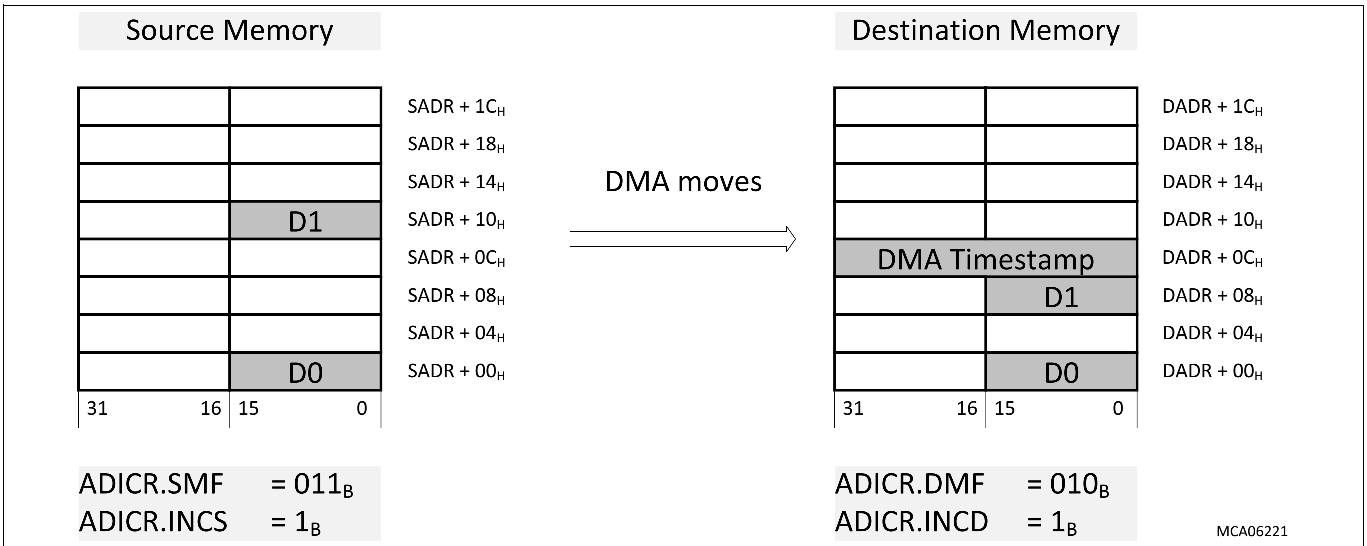

Figure 207 Incrementing Destination Address: DMA Timestamp appended above DMA Write Move Data

Appendage of DMA Timestamp - Example 2

16-bit half-words are moved from a source memory with an incrementing source address offset of 02_H to a destination memory with a decrementing destination address offset of 04_H (at **Figure 208**).

Figure 208 Decrementing Destination Address: DMA Timestamp appended below DMA Write Move Data

Destination Address Generation

The destination address of the next DMA write move (of the next DMA transaction) shall be calculated using:

- The destination address of the last DMA write move (of the current DMA transaction).
- The destination address offset defined by ADICR.INCD, ADICR.DMF and CHCFGR.CHDW.

The ME checks the destination address of the next DMA write move.

If the currently calculated destination address of the next DMA write move shall result in DMA write move data overwriting the DMA timestamp of the current DMA transaction, the ME shall calculate a new destination address:

- If DMA channel ADICR.INCD = 1_B, the ME shall select the greater value of the following:
  - Add (2_D x destination address offset) to the destination address of last DMA write move.
  - Add 8_D to the destination address of last DMA write move (of the current DMA transaction).
- If DMA channel ADICR.INCD = 0_B, the ME shall select the lesser value of the following:
  - Subtract (2_D x destination address offset) from the destination address of last DMA write move.
  - Subtract 8_D from the destination address of last DMA write move (of the current DMA transaction).

The ME shall write back the DMA channel TCS (including updated destination address) to the DMARAM.

18.3.4.6.3 Appendage of DMA Timestamp to Destination Circular Buffer

If a DMA channel is configured for the appendage of a DMA timestamp and destination circular buffer operation (ADICR.DCBE = 1_B), the ME shall write a DMA timestamp at a destination address defined by:

- If ADICR.INCD = 1_B AND ADICR.CBLD = 1 or 0, the address of the DMA timestamp = (DADR[31:2] + 1) << 2
- If ADICR.INCD = 1_B AND ADICR.CBLD > 1, the address of the DMA timestamp = (DADR[31:CBLD] + 1) << CBLD
- If ADICR.INCD = 0_B AND ADICR.CBLD = 1 or 0, the address of the DMA timestamp = (DADR[31:2] << 2) - 4
- If ADICR.INCD = 0_B AND ADICR.CBLD > 1, the address of the DMA timestamp = (DADR[31:CBLD]) << CBLD) - 4

The ME writes back to the DMARAM an incremented and wrapped destination address.

18.3.4.6.4 Application of DMA Timestamp

The appendage of a DMA timestamp may be used by software to detect DMA failure modes:

- **Lost DMA operation**: DMA request is not serviced (i.e. no data).
- **Unintended DMA operation**: unintentional DMA request results in unexpected DMA transaction (i.e. unexpected data).
- **Delayed DMA transaction**: expected DMA transaction is completed too late.

Software may analyze relative DMA timestamp values to determine the correct sequencing of DMA transactions.

18.3.4.7 Pattern Detection

If a DMA channel is configured for **PATDET** then after each DMA read move, the ME compares move data with a value stored in a Pattern Read Register (PRR). The PATDET operation depends on the channel data width (CHCFGR.CHDW). The control field CHCFGR.PATSEL selects different PATDET operations for a specific value of CHDW.

The **Pattern Compare Logic** compares the move data stored in an ME read register to the selected PRR. If the bytes of move data to be compared are spread across different words of move data, the ME may combine the pattern match result of the current **DMA Move** and the pattern match result (stored in ME CHSR.LXO) of the previous **DMA Move**.

Configuration

If a DMA channel is configured for **PATDET**:

- The channel data width (CHCFGR.CHDW) shall be configured to 8-bit, 16-bit or 32-bit[1].
- The source address shall not be configured to a cached address (segments 8 and 9). A non-cached address (segments A and B) shall be configured to prevent the **ME Read Buffer** re-aligning the move data.

Interrupt Service Request

If a DMA channel is configured for PATDET and a pattern match is detected:

- The DMA transaction ends with the current DMA move.
- DMA channel TSR.HTRE is cleared to stop DMA transfers in the current active DMA channel.
- DMA channel TSR.CH will clear when the DMA write move has completed.
- The ME shall write back the next DMA move values of SADR and DADR.
  - If the DMA channel is configured for **Shadow Operation Read Only Mode**, the ME shall not perform a shadow address update. The ME write backs the current value of SHADR is to the DMARAM.
- A PATDET interrupt service request shall be triggered.

Software may read the DMA channel status to identify the location of the pattern match in the data stream.

Errors

If a **SER** is reported during the current DMA read move, the ME shall not perform a pattern match.

18.3.4.7.1 Pattern Compare Logic

The ME COMPare (COMP) logic (at **Figure 209**) compares move data against a reference pattern at a bit-wise level. One COMP block controls either 8 bits or 16 bits of data. The COMP block may be configured to mask each data bit for the pattern compare.

1) If a DMA channel is configured for PATDET, the ME shall only store move data in MEm1R and/or MEm0R.

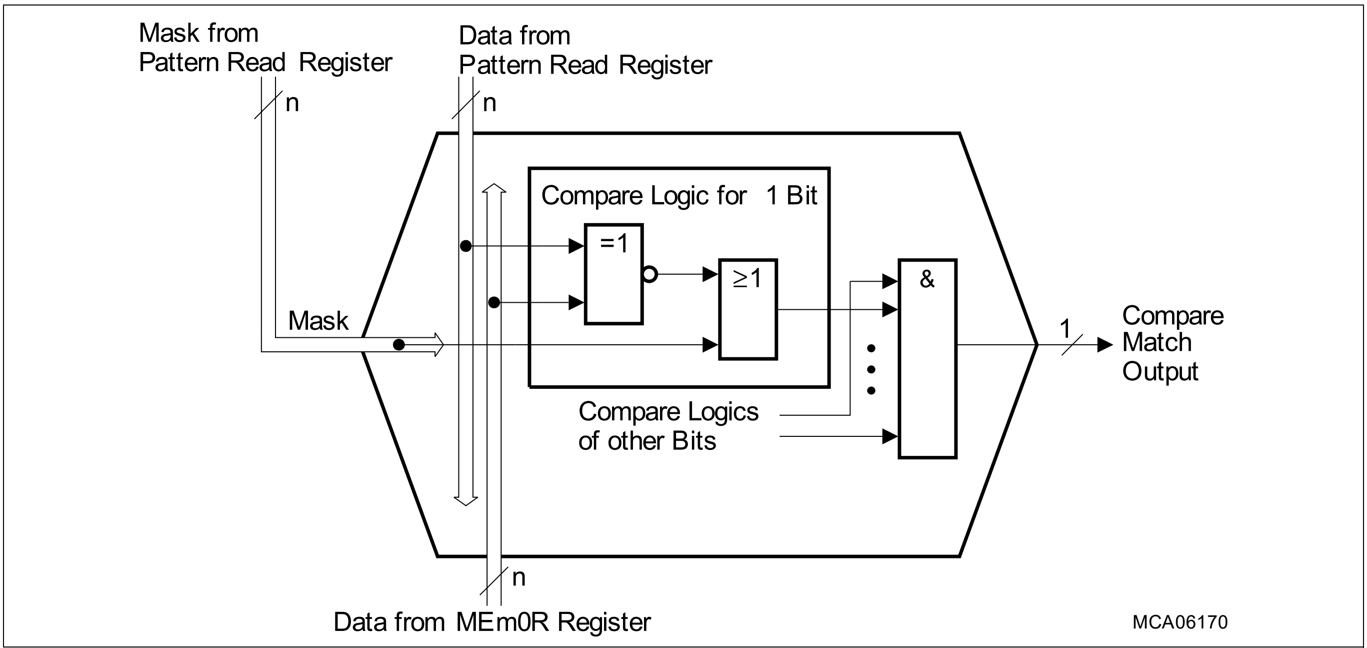

Figure 209 Pattern Compare Logic (COMP Block)

In the COMP logic, one data bit from a ME read register is compared to the corresponding pattern bit stored in the selected PRR. If both bits are equal and a pattern mask bit stored in another PRR bit field is 0_B then the compare matched condition becomes active. When the pattern mask bit is set to 1_B, the compare matched condition is always active (set) for the related bit. When the compare matched conditions for each bit within the COMP logic are true, the compare match output line of the COMP block becomes active.

18.3.4.7.2 Pattern Detection for 8-bit Channel Data Width

If a DMA channel is configured for **Pattern Detection for 8-bit Channel Data Width**, the **ME** shall use the source address to select byte(s) of move data to be compared with patterns stored in PRR0 or PRR1. Three pattern compare configurations are possible defined by the DMA channel pattern select (CHCFGFR.PATSEL[1:0]).

The pattern compare logic allows a byte of DMA move data to be compared with two different patterns.

A mask operation of each compared bit is possible.

Example of PATDET for 8-bit Channel Data Width

The DMA channel (at **Figure 210**) is configured for PATDET for 8-bit channel data width (CHCFGR.CHDW = 000_B).

If the current DMA read move stores the move data byte in MEm0R.RD00 then

- The **ME** uses the source address to select the move data byte stored in MEm0R.RD00.
- The **ME** compares move data byte stored in MEm0R.RD00 to PRR0.

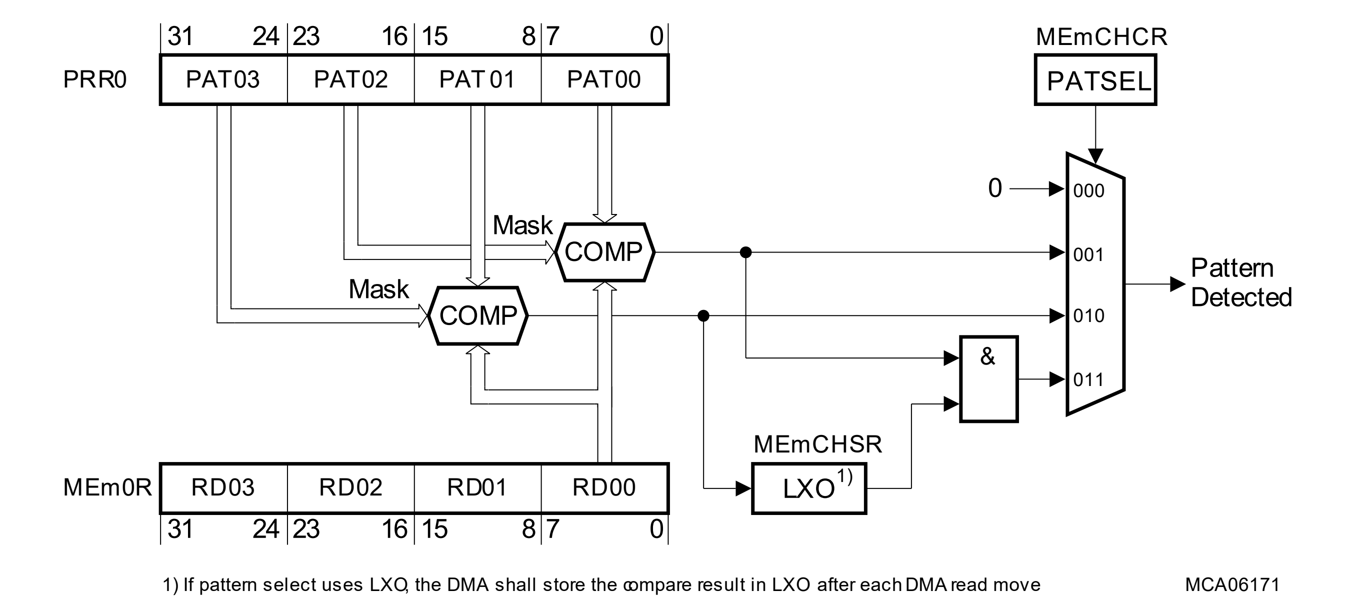

Figure 210 PATDET for 8-bit Channel Data Width and Move Data Byte stored in MEm0R.RD00

The DMA channel may be configured for one of three pattern compare operations (at **Table 574**).

Table 574 PATDET for 8-bit Channel Data Width and Move Data Byte stored in MEm0R.RD00

PATSEL	Pattern Detection Operating Modes
000_B	Pattern detection disabled
001_B	Pattern compare of MEm0R.RD00 to PRR0.PAT00, masked by PRR0.PAT02
010_B	Pattern compare of MEm0R.RD00 to PRR0.PAT01, masked by PRR0.PAT03
011_B	Pattern compare of MEm0R.RD00 to PRR0.PAT00, masked by PRR0.PAT02 of the current DMA read move **and** Pattern compare of MEm0R.RD00 to PRR0.PAT01, masked by PRR0.PAT03 of the previous DMA read move
100_B	Pattern detection disabled
101_B	Pattern compare of MEm0R.RD00 to PRR1.PAT10, masked by PRR1.PAT12
110_B	Pattern compare of MEm0R.RD00 to PRR1.PAT11, masked by PRR1.PAT13
111_B	Pattern compare of MEm0R.RD00 to PRR1.PAT10, masked by PRR1.PAT12 of the current DMA read move **and** Pattern compare of MEm0R.RD00 to PRR1.PAT11, masked by PRR1.PAT13 of the previous DMA read move

Example of Two Byte PATDET Sequence

If a DMA channel is configured for a two byte PATDET sequence compared to PRR0 (CHCFGR.PATSEL[2:0] = 011_B) then after each DMA move the pattern match result of move data compared with PRR0.PAT01, masked by PRR0.PAT03 is stored in bit ME CHSR.LXO.

This DMA channel configuration allows, for example, two-byte sequences to be detected in an 8-bit data stream coming from a serial peripheral unit with 8-bit data width (e.g. recognition of carriage-return, line-feed characters, etc.).

18.3.4.7.3 Pattern Detection for 16-bit Channel Data Width

If a DMA channel is configured for **Pattern Detection for 16-bit Channel Data Width**, the **ME** shall use the source address to select bytes of move data to be compared with patterns stored in PRR0 or PRR1. Three pattern match configurations are possible defined by the DMA channel pattern select (CHCFGFR.PATSEL[1:0]):

- Aligned Mode: compare complete half-word of one DMA move only.
- Unaligned Mode: compare upper and lower byte of two consecutive DMA moves.
- Combined Mode: combine Aligned Mode and Unaligned Mode.

A mask operation of each compared bit is possible.

Example of PATDET for 16-bit Channel Data Width

The DMA channel (at **Figure 211**) is configured for PATDET for 16-bit channel data width (CHCFGR.CHDW = 001_B).

If the current DMA read move stores the move data half word in MEm0R.RD01||RD00 then

- The **ME** uses the source address to select the move data half word stored in MEm0R.RD01||RD00.
- The **ME** compares move data half word stored in MEm0R.RD01||RD00 to PRR0.

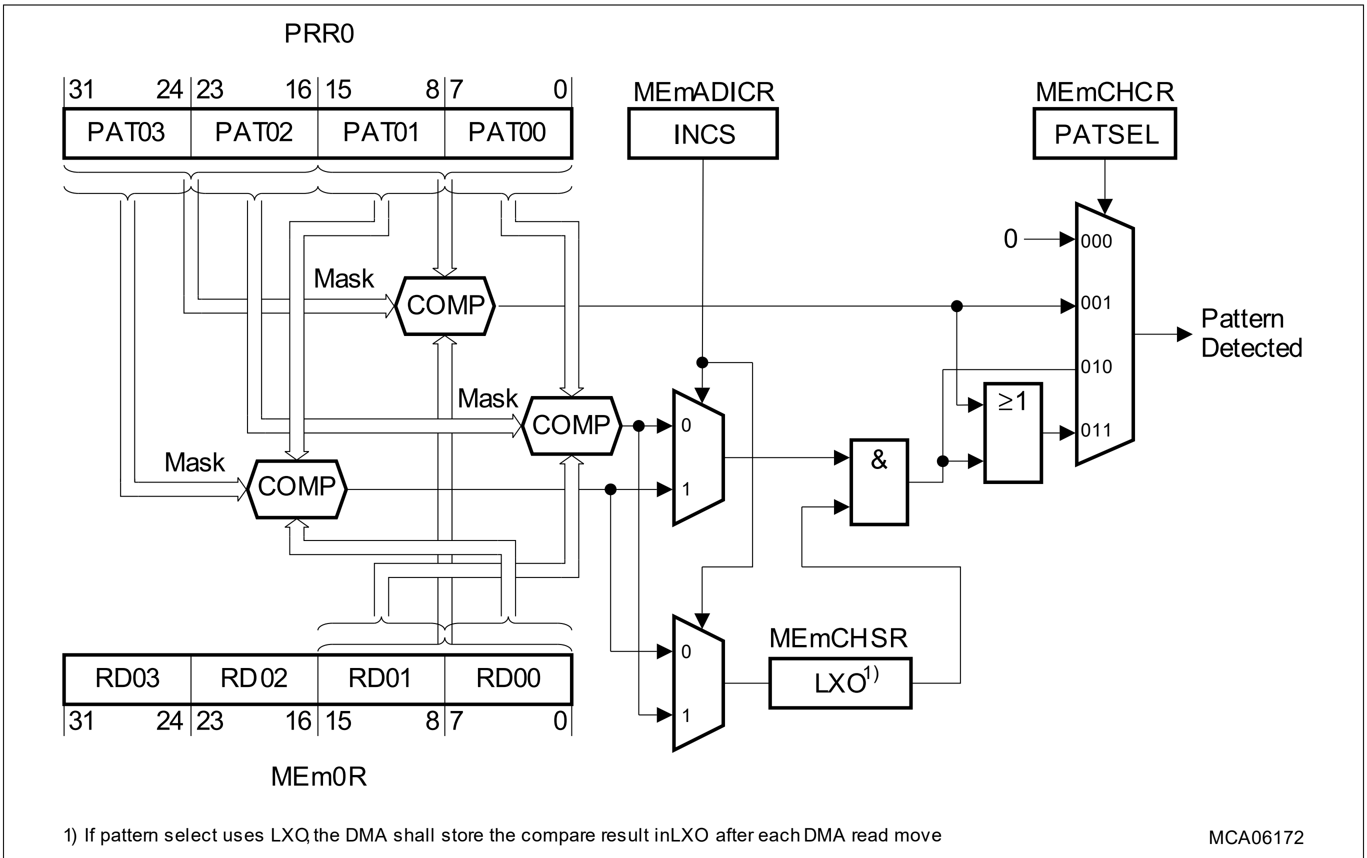

Figure 211 PATDET for 16-bit Channel Data Width and Move Data stored in MEm0R.RD01||RD00

The DMA channel may be configured for one of three pattern compare operations (at **Table 575**).

- Aligned Mode
- Unaligned Mode 1 (Source Address Decrement)
  - The high byte of the current 16-bit DMA read move and the low byte of the previous 16-bit DMA read move are compared.
- Unaligned Mode 2 (Source Address Increment)

- The low byte of the current 16-bit DMA read move and the high byte of the previous 16-bit DMA read move are compared.

- Combined Mode
  - If it is not known on which byte boundary (even or odd address) the 16-bit pattern to be detected is located, the DMA channel should be configured for combined mode.
  - The combined mode is the most flexible mode that combines the pattern search capability for aligned and unaligned 16-bit data searches.

Table 575 PATDET for 16-bit Channel Data Width and Move Data stored in MEm0R.RD01||RD00

PATSEL	INCS	Pattern Detection Operating Modes
000_B	–	Pattern detection disabled
001_B	–	**Aligned Mode** Pattern compare of MEm0R.RD01||RD00 to PRR0.PAT01||PAT00, masked by PRR0.PAT03||PAT02
010_B	0	**Unaligned Mode 1 (Source Address Decrement)** Pattern compare of MEm0R.RD01 to PRR0.PAT00, masked by PRR0.PAT02 of the current DMA read move **and** Pattern compare of MEm0R.RD00 to PRR0.PAT01, masked by PRR0.PAT03 (LXO) of the previous DMA read move
	1	**Unaligned Mode 2 (Source Address Increment)** Pattern compare of MEm0R.RD00 to PRR0.PAT01, masked by PRR0.PAT03 of the current DMA read move **and** Pattern compare of MEm0R.RD01 to PRR0.PAT00, masked by PRR0.PAT02 (LXO) of the previous DMA read move
011_B	0 or 1	**Combined Mode** Pattern compare for aligned mode (PATSEL = 001_B) **or** unaligned modes (PATSEL = 010_B)
100_B	–	Pattern detection disabled
101_B	–	**Aligned Mode** Pattern compare of MEm0R.RD01||RD00 to PRR1.PAT11||PAT10, masked by PRR1.PAT13||PAT12
110_B	0	**Unaligned Mode 1 (Source Address Decrement)** Pattern compare of MEm0R.RD01 to PRR1.PAT10, masked by PRR1.PAT12 of the current DMA read move **and** Pattern compare of MEm0R.RD00 to PRR1.PAT11, masked by PRR1.PAT13 (LXO) of the previous DMA read move
	1	**Unaligned Mode 2 (Source Address Increment)** Pattern compare of MEm0R.RD00 to PRR1.PAT11, masked by PRR1.PAT13 of the current DMA read move **and** Pattern compare of MEm0R.RD01 to PRR1.PAT10, masked by PRR1.PAT12 (LXO) of the previous DMA read move
111_B	0 or 1	**Combined Mode** Pattern compare for aligned mode (PATSEL = 101_B) **or** unaligned modes (PATSEL = 110_B)

18.3.4.7.4 Pattern Detection for 32-bit Channel Data Width

If a DMA channel is configured for **Pattern Detection for 32-bit Channel Data Width**, the **ME** shall compare a move data word with patterns stored in PRR0 or PRR1. Three pattern compare configurations are possible defined by the DMA channel pattern select (CHCFGFR.PATSEL[1:0]). A mask operation is not possible.

Example of PATDET for 32-bit Channel Data Width

The DMA channel (at **Figure 212**) is configured for PATDET and 32-bit channel data width (CHCFGR.CHDW = 010_B).

If the current DMA read move stores the move data word in MEm0R then

- The **ME** uses the source address to select the move data word stored in MEm0R.
- The **ME** compares move data word stored in MEm0R to PRR0.

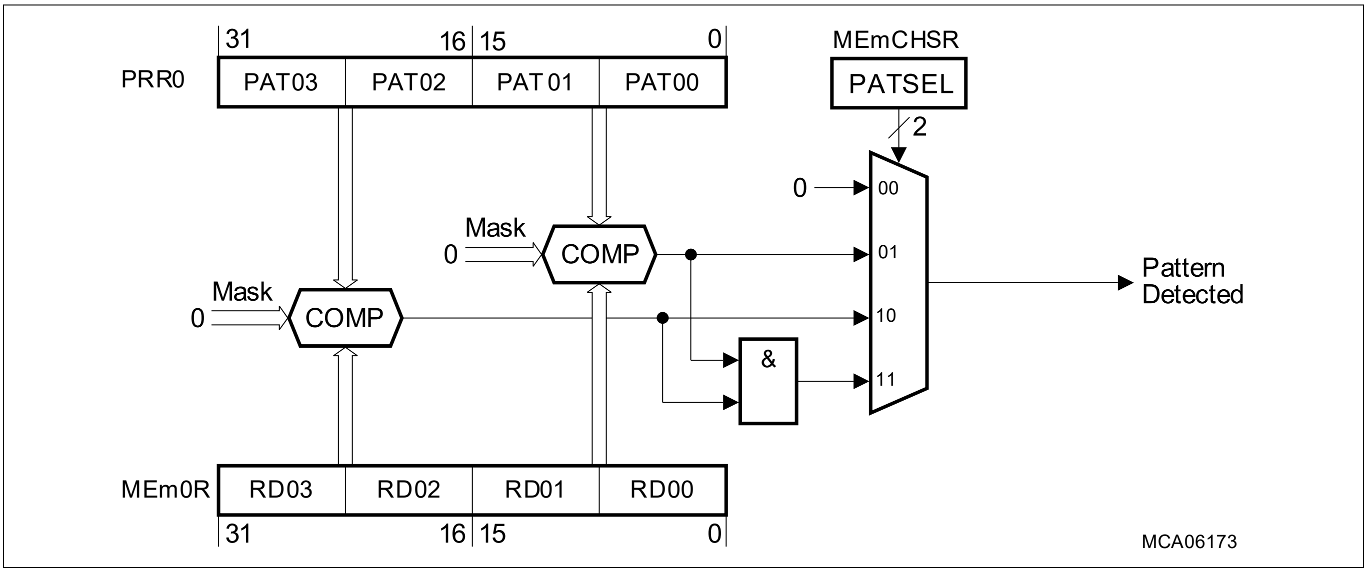

Figure 212 PATDET for 32-bit Channel Data Width and Move Data stored in MEm0R

The DMA channel may be configured for one of three pattern compare operations (at **Table 576**).

- Lower half-word only compared to pattern stored in PRR0 or PRR1.
- Upper half-word only compared to pattern stored in PRR0 or PRR1.
- Complete 32-bit word compared to pattern stored in PRR0 or PRR1.

Table 576 PATDET for 32-bit Channel Data Width and Move Data stored in MEm0R

PATSEL	Pattern Detection Operating Modes
000_B	Pattern detection disabled
001_B	Unmasked pattern compare of MEm0R.RD01||RD00 to PRR0.PAT01||PAT00
010_B	Unmasked pattern compare of MEm0R.RD03||RD02 to PRR0.PAT03||PAT02
011_B	Unmasked pattern compare of MEm0R.RD03||RD02||RD01||RD00 to PRR0.PAT03||PAT02||PAT01||PAT00
100_B	Pattern detection disabled
101_B	Unmasked pattern compare of MEm0R.RD01||RD00 to PRR1.PAT11||PAT10
110_B	Unmasked pattern compare of MEm0R.RD03||RD02 to PRR1.PAT13||PAT12
111_B	Unmasked pattern compare of MEm0R.RD03||RD02||RD01||RD00 to PRR1.PAT13||PAT12||PAT11||PAT10

18.3.4.8 Double Buffering Operations

The DMA supports double buffering. For example, Double Destination Buffering (at **Figure 213**) is as follows:

- DMA read moves transfer a continuous data stream from a peripheral to the DMA.
- DMA write moves transfer the read data from the DMA to one of two destination buffers stored in memory.

The dormant buffer is frozen and available for cyclic software tasks while the other buffer continues to be filled.

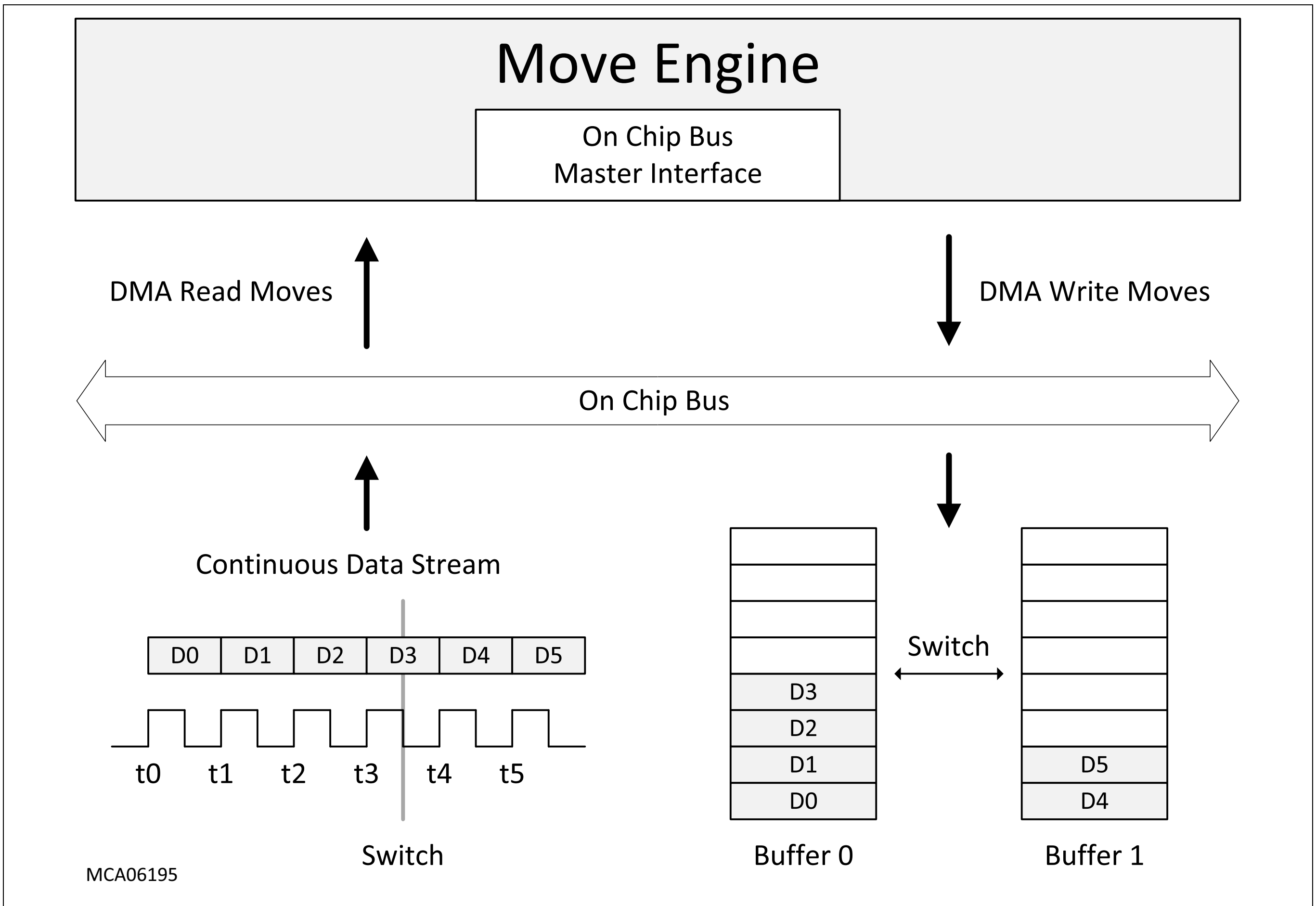

Figure 213 DMA Double Buffering

If a DMA channel is configured for one of the **Double Buffering Operations** AND is triggered by **DMA Hardware Request**, software should configure the DMA channel for **Continuous Mode** (DMA channel CHCFGR.CHMODE = 1_B).

18.3.4.8.1 DMA Double Source Buffering

Data is streamed by the DMA from one of two data buffers (buffer 0 and buffer 1) stored in memory to a continuous output data stream. The Double Source Buffering addresses are:

- The source address SADR shall address source buffer 0.
- The shadow address SHADR shall address source buffer 1.

18.3.4.8.2 DMA Double Destination Buffering

A continuous input data stream is directed by the DMA to one of two data buffers (buffer 0 and buffer 1) stored in memory. The Double Destination Buffering addresses are:

- The destination address DADR shall address destination buffer 0.

- The shadow address SHADR shall address destination buffer 1.

18.3.4.8.3 Size of Buffer

The data size of the two source/destination buffers is equal and determined by the following TCS parameters:

- The channel data width (CHCFGR.CHDW).
- The block mode value (CHCFGR.BLKM).
- The transfer reload value (CHCFGR.TREL).

The memory size of the two source/destination buffers is determined by the additional TCS parameters:

- The source/destination circular buffer enable/disable control (ADICR.SCBE/DCBE).
- The source/destination address modification factor (ADICR.DMF/SMF).
- The increment of the source/destination address (ADICR.INCD/INCS).

The size of one buffer is equal to the size of one DMA transaction.

18.3.4.8.4 Buffer Switch

The re-direction of the data stream from one buffer to the other buffer shall be controlled by a **Software Switch** and additionally **Automatic Hardware Switch**. During a buffer switch:

- No DMA requests shall be lost by the DMA.
- There shall be no loss, duplication or split of data across the two buffers.

Active Buffer Status

During DMA double buffering the DMA channel CHCSR.BUFFER bit is used to indicate which buffer is active.

- DMA channel CHCSR.BUFFER = 0_B: buffer 0 is read or filled by the DMA.
- DMA channel CHCSR.BUFFER = 1_B: buffer 1 is read or filled by the DMA.

Frozen Buffer Status

When the DMA switches buffers the DMA channel frozen bit is set (CHSR.FROZEN= 1_B). The frozen buffer is then available for a cyclic software task. On completion of the software task the status bit CHSR.FROZEN is cleared by software and the buffer address pointer is re-programmed to the start address ready for the next DMA transaction that reads/fills the buffer.

Active Buffer Empty or Full

If the active buffer is emptied (in the case of DMA double source buffering) or filled (in the case of DMA double destination buffering) and no automatic hardware switch occurs before a software buffer switch is received then the DMA channel will stop and no more DMA transfers are made. When the DMA channel stops, the transfer count (CHCSR.TCOUNT) is equal to 0000_H.

A buffer empty or full event may be signalled by a **DMA Channel Interrupt Service Request**. The DMA channel interrupt control ADICR.INTCT should be configured to 10_B and the interrupt raise detect value ADICR.IRDV to 0000_B. When the transfer count equals zero then the DMA channel will raise a DMA channel interrupt service request.

The DMA channel interrupt handler can interrogate the DMA to identify which DMA channel generated the interrupt service request. Software may initialize the DMA channel ready to restart double buffering operations.

Active Buffer Overflow

If the data rate is faster than the DMA is able to transfer the data to one of the buffers then a **TRL** event shall occur.

18.3.4.8.5 Software Switch

Before a DMA transaction completes and the active buffer is full software shall re-direct the data stream from the active buffer to the other buffer by setting the DMA channel CHCSR.SWB. If a ME is actively servicing a DMA channel configured for double buffering then the current DMA transfer shall complete before the buffer switch is made. On completion of the DMA transfer the DMA shall:

- Automatically re-load DMA channel CHCSR.TCOUNT with CHCFGR.TREL
- Switch the address pointer:
  - Buffer 0 address pointer (source address or destination address).
  - Buffer 1 address pointer (shadow address).

The TCS address control factors remain the same. The next DMA transaction controlling the filling or reading of the new buffer will start when a DMA request is received.

If software re-directs the data stream by setting DMA channel CHCSR.SWB = 1_B, software shall poll DMA channel CHCSR.FROZEN AND DMA channel CHCSR.BUFFER. If software reads DMA channel CHCSR.FROZEN = 1_B and an active buffer transition, software may start to analyze the frozen buffer.

If a DMA channel is configured for Software Switch Only AND the DMA transaction is completed before the DMA receives a Software Switch, the DMA shall report a **TRL** event (DMA channel TSR.TRL = 1_B). If the DMA channel enable TRL bit is set (TSR.ETRL = 1_B), the DMA shall trigger a **DMA RP Error Interrupt Service Request**.

18.3.4.8.6 Automatic Hardware Switch

The DMA will automatically switch from the active buffer to the other buffer when the DMA transaction reading or filling the active buffer completes. On switching buffers the DMA channel CHCSR.FROZEN bit is set to indicate that one buffer is frozen and available for cyclic software tasks. The completion of a DMA transaction may be signalled by a **DMA Channel Interrupt Service Request**. On completion of the DMA transaction when ME CHSR.TCOUNT equals 0000_H the DMA shall:

- Automatically re-load DMA channel CHCSR.TCOUNT with CHCFGR.TREL
- Switch the address pointer to select the other buffer:
  - Buffer 0 address pointer (source address or destination address).
  - Buffer 1 address pointer (shadow address).

The DMA channel TCS address control factors remain the same. The next DMA transaction controlling the filling or reading of the new buffer will start when a DMA request is received.

If DMA channel CHCSR.FROZEN is equal to $1_{B,}$ the automatic hardware switch will not occur and the DMA shall report a **TRL** event (DMA channel TSR.TRL = 1_B). If the DMA channel enable TRL bit is set (TSR.ETRL = 1_B), the DMA shall trigger a **DMA RP Error Interrupt Service Request**.

18.3.4.8.7 Application of Double Buffering

A typical DMA double buffer application is shown in **Figure 214**:

- DMA channel m is configured for **DMA Double Destination Buffering**. The DMA channel moves the input data generated by a sensor into the active destination buffer.
- A cyclic software task processes the input data stored in the frozen destination buffer and writes the output into the frozen source buffer. The CPU controls both the source and destination buffer switching.
- DMA channel n is configured for **DMA Double Source Buffering**. The DMA channel moves the processed data from the frozen source buffer to the actuator.

The system configuration shall balance the input data rate, the duration of cyclic software tasks and the buffer size in order to prevent **TRL** events.

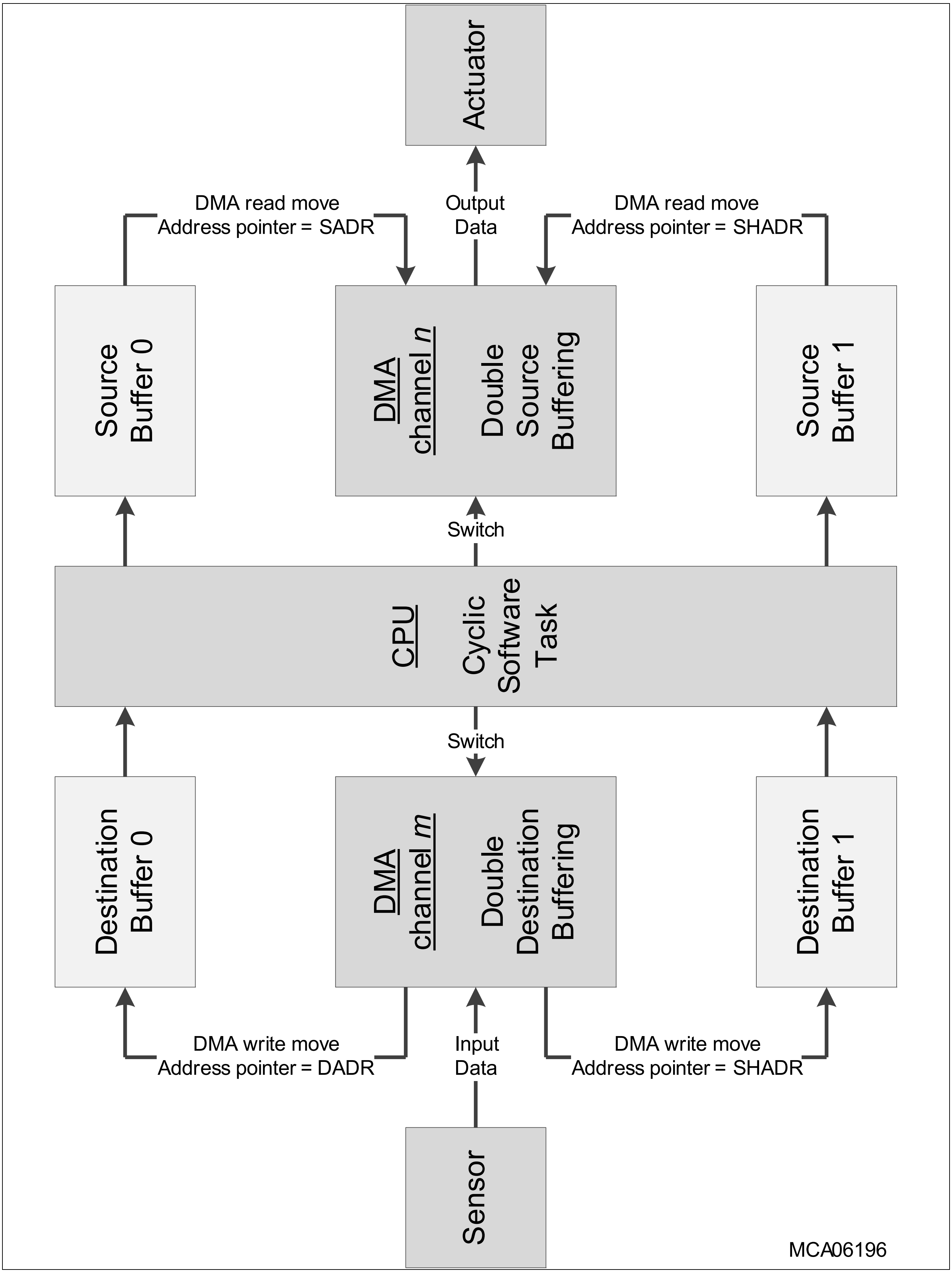

Figure 214 Application of Double Buffering

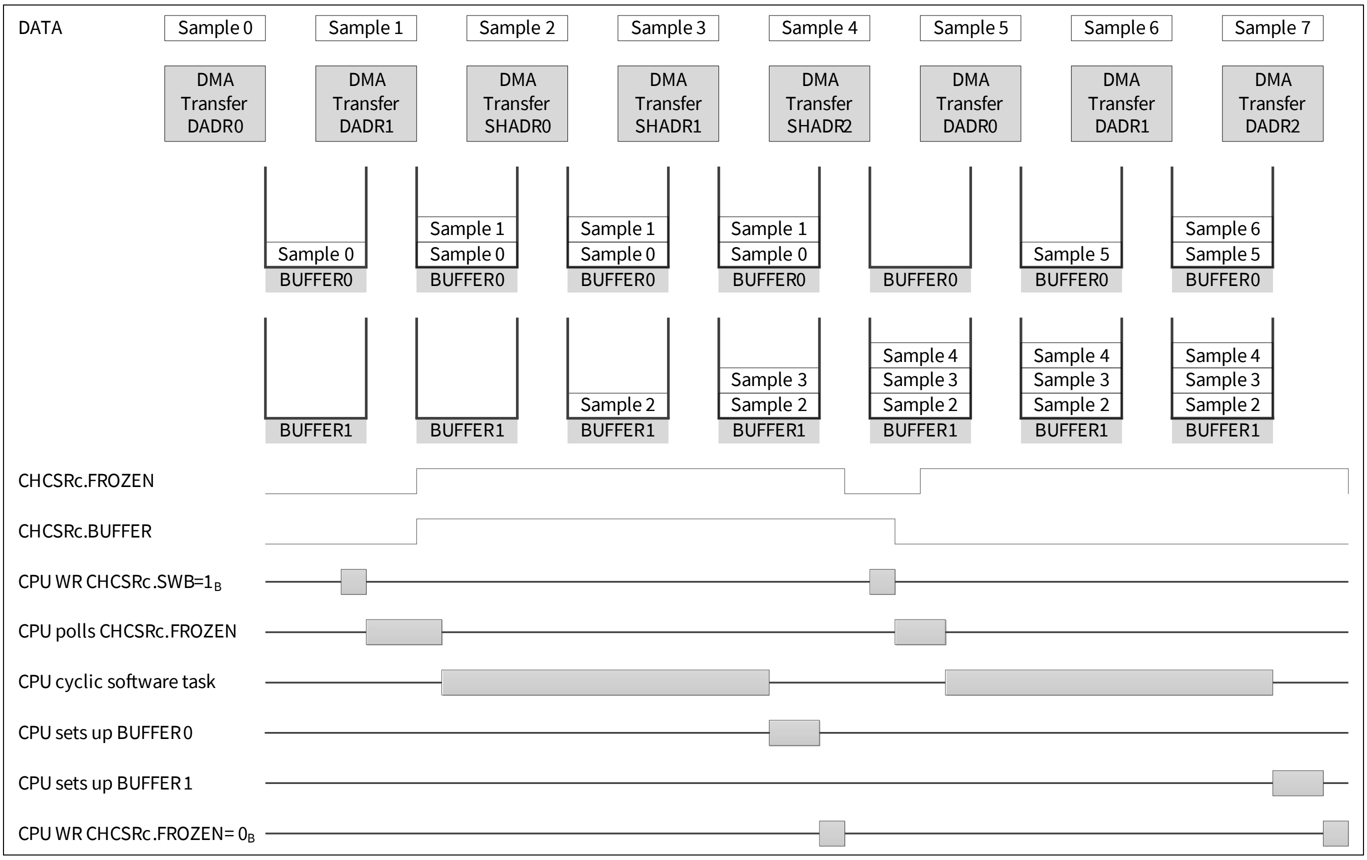

Figure 215 DMA Double Destination Buffering Software Switch Only

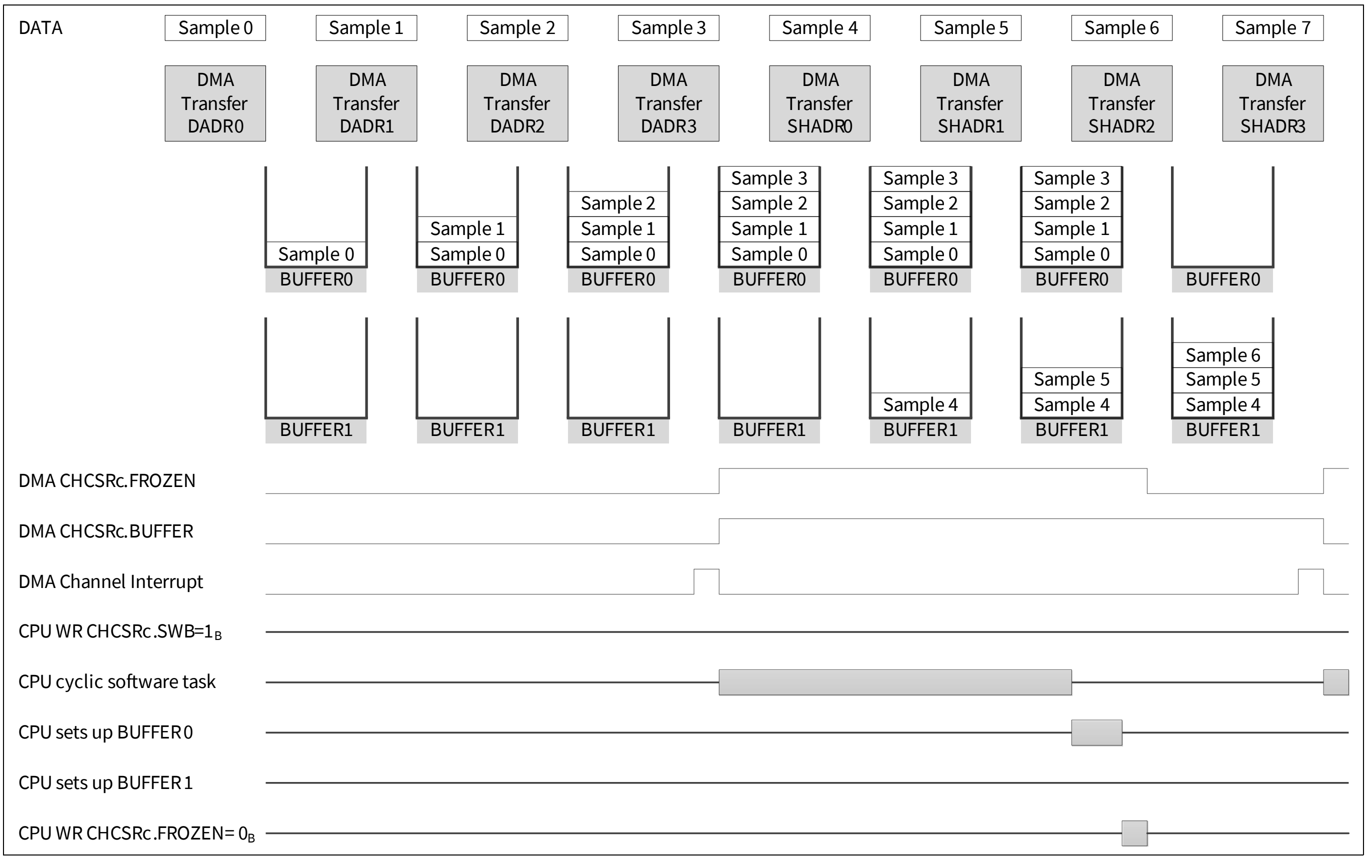

Figure 216 DMA Double Destination Buffering Software Switch and Automatic Hardware Switch

18.3.4.9 Linked List Operations

Linked list operations are an extension of DMA channel functionality to support the more flexible use of the DMA.

A linked list operation shall consist of a series of DMA transactions executed by the same DMA channel. Each DMA transaction shall have a unique TCS. The source and destination areas shall not have to exist in contiguous areas of memory.

If a DMA channel is configured for linked list operation then as soon as the ME completes the current DMA transaction, the ME shall read the next TCS from memory and overwrite the current TCS at the DMA channel location in DMARAM. The current DMA transaction uses a 32-byte aligned address pointer to point to the next TCS stored in either internal or external memory. The DMA shall start reading the next TCS from word 0 upwards. There is no limit to the number of DMA transactions in a linked list operation. The first DMA transaction in a linked list operation shall be initiated by a DMA hardware request or a DMA software request. Subsequent DMA transactions may additionally be initiated by a **DMA Auto Start Request**.

Each DMA channel supports the following types of linked list operation:

- **DMA Linked List (DMALL)**
- **Accumulated Linked List (ACCLL)**
- **Safe Linked List (SAFLL)**
- **Conditional Linked List (CONLL)**

18.3.4.9.1 DMA Auto Start Request

If a **DMA Auto Start Request** is selected in the next TCS (CHCSR.SCH = 1_B) then the next DMA transaction shall be initiated by the ME. The access pending bit TSR.CH shall be set. The DMA shall perform an **Arbitration Sequence** to determine the highest priority DMA request. A **DMA Auto Start Request** shall bypass the Interrupt Router so reducing the cumulative latency over a number of DMA transactions.

18.3.4.9.2 Non Linked List Operation

If the next TCS is not configured for linked list operation, the next TCS should be configured for a **Move Operation**. The DMA channel exits linked list operation. A **DMA Auto Start Request** shall not initiate the next DMA transaction.

18.3.4.9.3 Last DMA Transaction

A **DMA Channel Interrupt Service Request** may be used to signal the completion of the last DMA transaction in a sequence of linked list operations. The last DMA transaction will load the next TCS. **DMA Auto Start Request** shall not be selected.

18.3.4.9.4 Circular Linked List Operations

A sequence of linked list operations may be configured for circular operation. The same series of DMA transactions shall be repeated.

18.3.4.9.5 DMA Linked List (DMALL)

The DMA channel (at **Figure 217**) is configured for DMALL operation. The DMA channel shadow address register (SHADR) stores the 32-bit address pointer to the next TCS. As soon as the current DMA transaction completes, the ME reads the next TCS and stores it in DMARAM.

DMA Address Checksum & DMA Data Checksum

Software shall configure the initial value of **DMA Address Checksum** and **DMA Data Checksum** used during the first DMA transaction. When the ME reads the next TCS, the **DMA Address Checksum** and **DMA Data Checksum** shall be initialized to 00000000_H.

18.3.4.9.6 Accumulated Linked List (ACCLL)

ACCLL is a variant of DMALL and has an identical footprint in memory (size and structure). The SDCRC and RDCRC checksums are accumulated across DMA transactions.

DMA Address Checksum & DMA Data Checksum

The **DMA Address Checksum** and **DMA Data Checksum** are not overwritten when the next TCS is loaded into the DMA channel. As long as the DMA channel is configured for ACCLL, the **DMA Address Checksum** and **DMA Data Checksum** are calculated across all DMA transactions.

18.3.4.9.7 Safe Linked List (SAFLL)

The DMA channel (at **Figure 218**) is configured for SAFLL operation. SAFLL provides protection against software errors. If the address pointer in a sequence of linked list operations is incorrectly written then the address pointer may point to any random area of memory and load the next TCS from the wrong location and execute it. As long as a DMA channel is configured for SAFLL, the ME checks that the calculated **DMA Address Checksum** matches an expected **DMA Address Checksum** stored in the next TCS.

DMA Address Checksum

The user is required to load the SDCRC word with an expected **DMA Address Checksum** value. As soon as the ME has read the next TCS from memory, the ME compares the **DMA Address Checksum** calculated by the current DMA transaction against the expected **DMA Address Checksum** stored in the next TCS. If the checksums match then the DMA proceeds with the execution of the next TCS. If the checksums do not match then the ME records a **SAFLL DMA Address Checksum Error** and triggers a **DMA RP Error Interrupt Service Request**. The DMA stops the execution of the linked list operation. Assuming all the **DMA Address Checksum** values match during the sequence of linked list operations then the **DMA Address Checksum** is calculated across all DMA transactions.

DMA Data Checksum

The **DMA Data Checksum** is not overwritten when the new TCS is loaded into the DMA channel. The **DMA Data Checksum** is calculated across all DMA transactions.

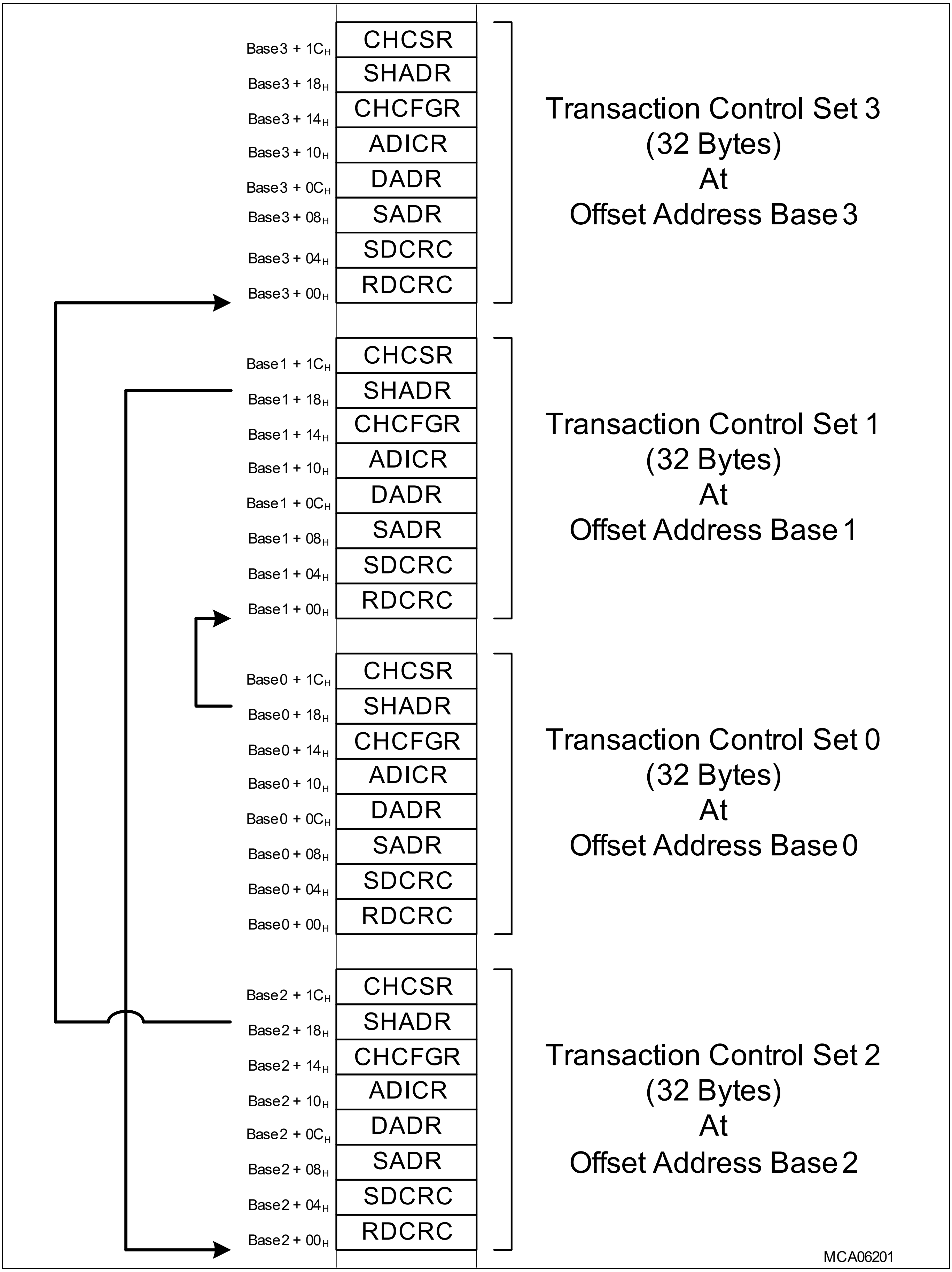

Figure 217 DMA Linked List

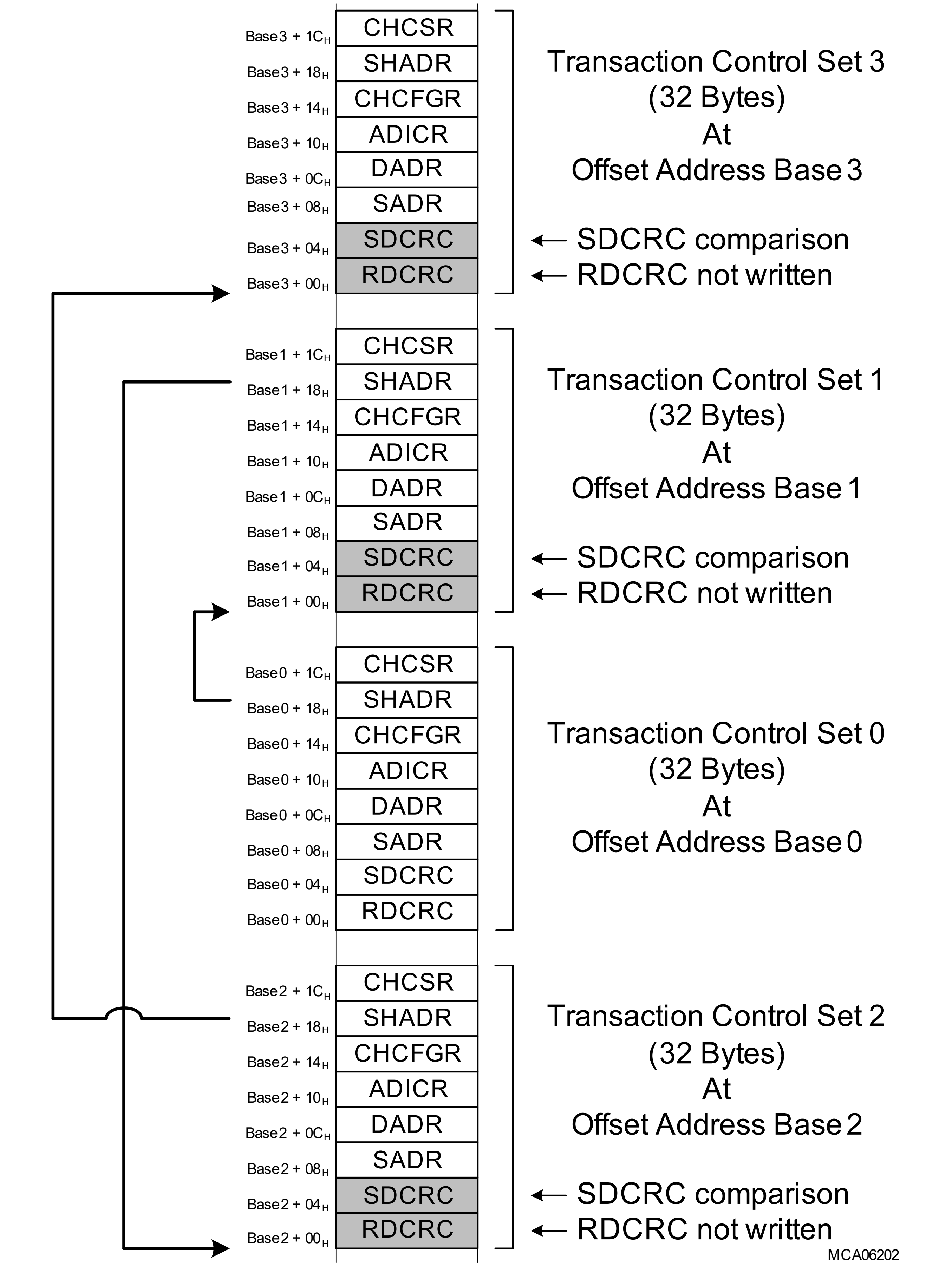

Figure 218 Safe Linked List

18.3.4.9.8 Conditional Linked List (CONLL)

A special use of a linked list is CONLL (at **Figure 219**). Selection of the address pointer to the next TCS is determined by a conditional state. CONLL uses the shadow address (SHADR) and the source and destination address CRC registers (SDCRCR) as address pointers.

A CONLL operation is limited to 8-bit DMA moves (CHCFGR.CHDW = 000_B). The source address shall not be configured to a cached address (segments 8 and 9). A non-cached address (segments A and B) shall be configured to prevent the **ME Read Buffer** re-aligning the move data.

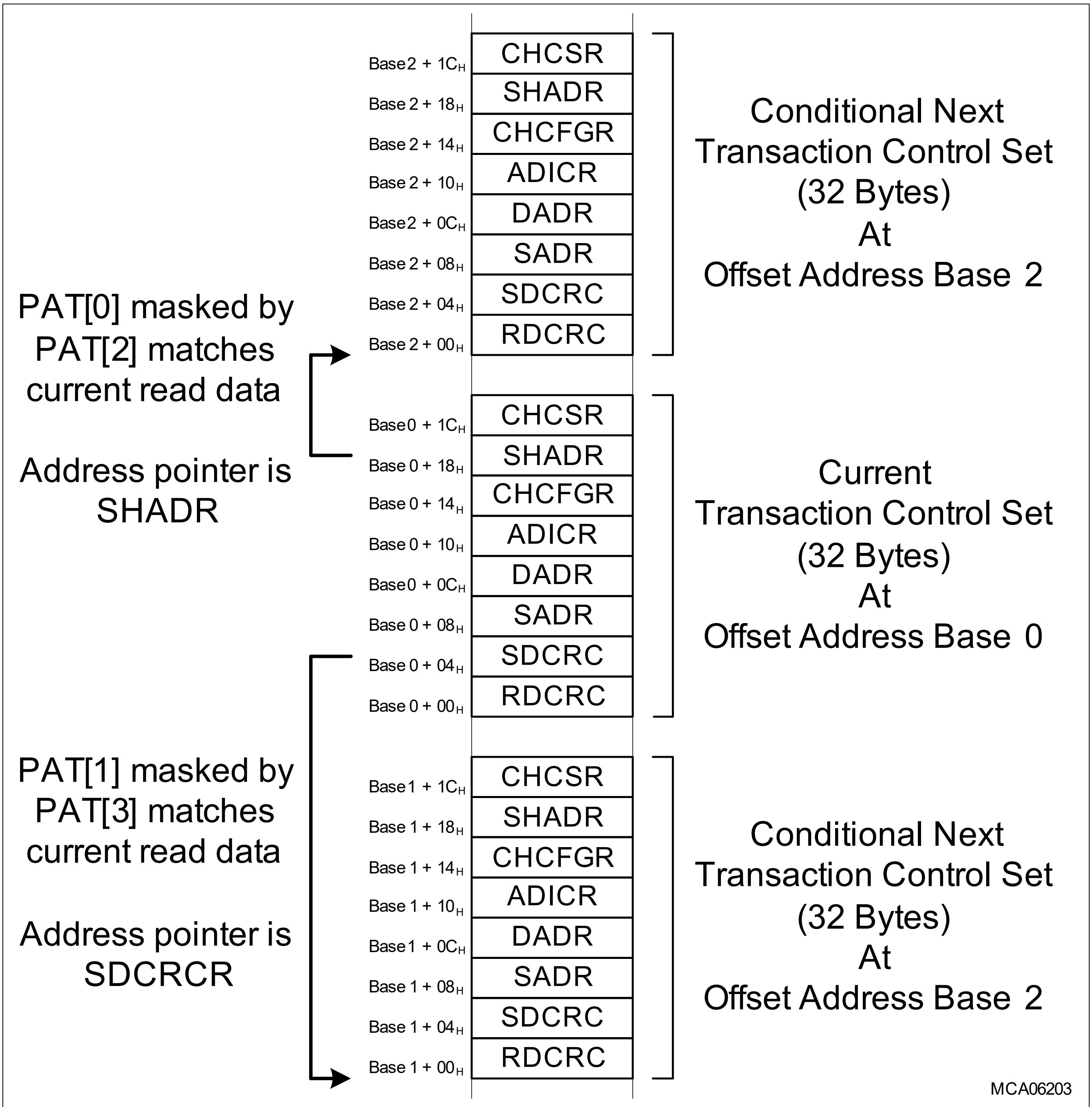

Figure 219 Conditional Linked List

CONLL Pattern Detection

The DMA channel (at **Figure 220**) is configured for CONLL. The TCS selects PRR0 for a pattern compare.

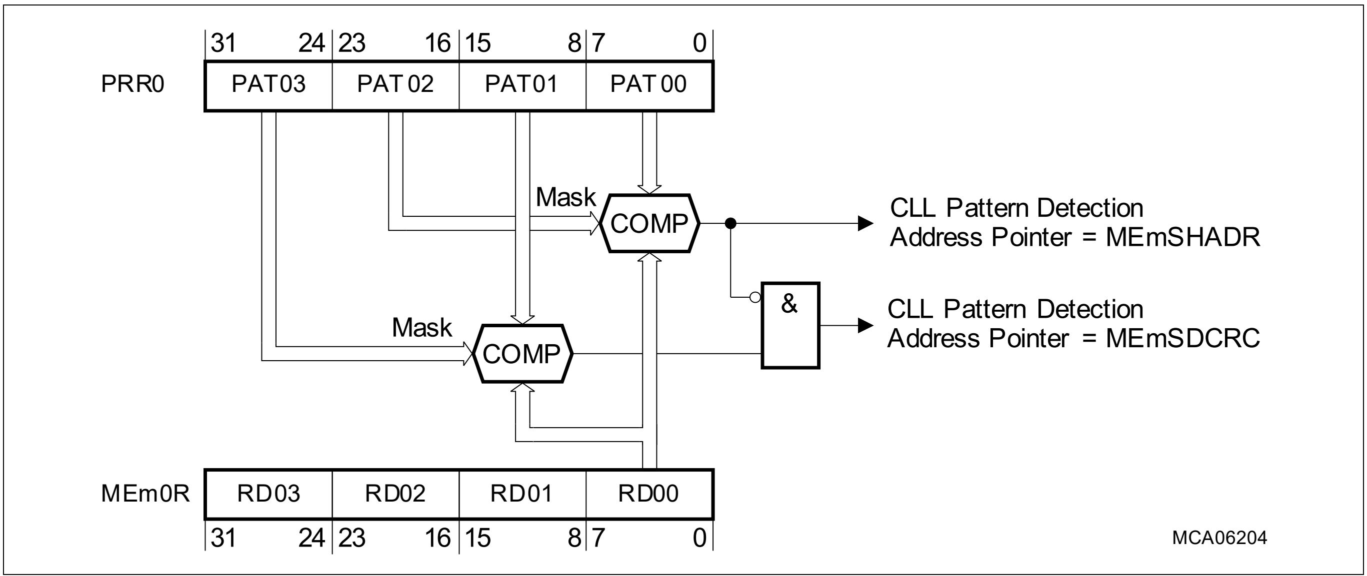

Figure 220 Conditional Linked List and Pattern Compare Logic

During each DMA move the pattern detection logic determines the selection of the address pointer:

- If PAT00 masked by PAT02 matches the DMA move data stored in MEm0R RD00 then as soon as the DMA move completes the ME shall load the DMA channel with the next TCS read from an address stored in SHADR.
- If PAT01 masked by PAT03 matches the DMA move data stored in MEm0R RD00 then as soon as the DMA move completes the ME shall load the DMA channel with the next TCS read from an address stored in SDCRCR.
- If both PAT00 masked by PAT02 and PAT01 masked by PAT03 match the DMA move data stored in MEm0R RD00 then as soon as the ME shall load the DMA channel with the next TCS read from an address stored in SHADR.
- If there is no pattern match then the ME continues with the DMA transaction.

If a pattern match is detected then:

- The DMA transaction ends with the current DMA move.
- The DMA channel TSR.CH will be cleared when the DMA write move has completed.

Completion of Current DMA Transaction

If the current DMA transaction completes when ME CHSR.TCOUNT = 0_D and there is no pattern match then the CONLL will load the DMA channel with the next TCS read from an address stored in SHADR.

Multiple Pattern Detection Conditions

The user may intentionally program PAT00 masked by PAT02 shall not match with the DMA move data stored in MEm0R RD00 and transition across a series of DMA transactions in the CONLL. For each DMA transaction the user may programme a series of different PAT01 masked by PAT03 values and test if each value matches with the current DMA move data stored in MEm0R RD00. If PAT01 masked by PAT03 matches the current DMA move data stored in MEm0R RD00, then as soon as the DMA move is completed, the ME shall load the DMA channel with the next TCS read from an address stored in SDCRCR.

18.3.4.10 DMA Data Checksum

To support enhanced data integrity checking the DMA calculates a 32-bit Read Data Cyclic Redundancy Checksum (**RDCRC**) in accordance with the IEEE 802.3 standard on DMA move data as it is passes through the DMA.

Move Operation or Shadow Operation

To calculate a **DMA Data Checksum** for DMA move data the DMA channel RDCRCR must be initialized (e.g. written with 00000000_H or another desired initial value) in addition to the standard DMA transaction configuration (source address, etc.) for the size of DMA move data.

If no error or retry event is reported during DMA move then the ME calculates an updated **DMA Data Checksum** value for each DMA move. On completion of the DMA transaction the ME stores the **DMA Data Checksum** in the DMA channel RDCRCR. Software may compare the calculated **DMA Data Checksum** with an expected **DMA Data Checksum** to verify the integrity of DMA move data.

Swap Byte

The **DMA Data Checksum** shall be calculated for all DMA channel data widths and may be configured to swap the byte order (at **Figure 221**). If DMA channel swap byte is selected (CHCFGR.SWAP = 1_B) then the order of bytes are swapped before **DMA Data Checksum** computation. Big-endian input is converted to little-endian and vice versa. If the DMA channel is configured for 8-bit channel data width (CHCFGR.CHDW = 000_B) then swap byte has no effect.

Swap byte does not change the byte order of data between a DMA read move and a DMA write move.

DMA Timestamp

If the DMA channel is configured to append a DMA timestamp then the DMA timestamp is not part of the **DMA Data Checksum** calculation.

Double Buffering Operations

The **DMA Data Checksum** is accumulated across all the DMA move data moved into both buffers. Software may re-initialize the **DMA Data Checksum** after a **Software Switch** and before the next data sample is received.

DMA Linked List (DMALL)

The **DMA Data Checksum** is initialized to 00000000_H at the start of each DMA transaction.

Accumulated Linked List (ACCLL), Safe Linked List (SAFLL) and Conditional Linked List (CONLL)

The **DMA Data Checksum** is accumulated across all the DMA transactions in the linked list.

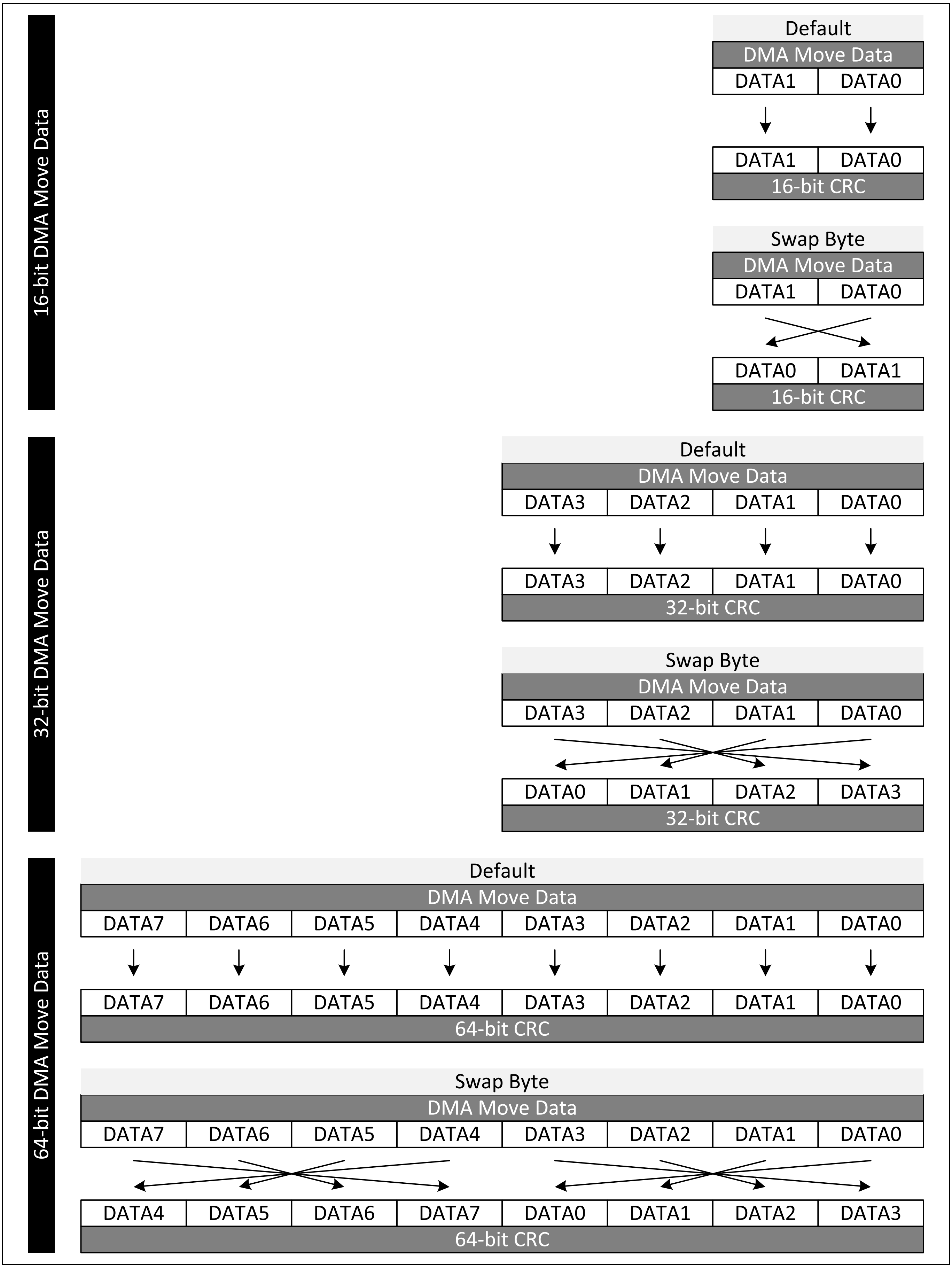

Figure 221 DMA Data Checksum - Swap Byte Order

18.3.4.11 DMARAM Initialization

On the assertion of an application reset, the DMA shall initialize all DMARAM data to all zeros to prevent the triggering of DMARAM data integrity errors in the application.

18.3.5 Move Engine

Any ME shall service a DMA request from any DMA channel. As soon as a DMA channel wins arbitration, the TCS is copied from the DMARAM to the ME active channel registers and the DMA request is serviced. The ME requests the required buses and loads or stores data according to the TCS parameters of the active DMA channel. The ME is able to wait if the source or destination target is not available.

The processing of a DMA transfer (composed of several DMA moves) by the ME cannot be interrupted and is always completed. A DMA channel interrupt, reset, halt request or debug suspend only becomes active when the current DMA transfer is complete. Error conditions are reported. On completion of a DMA transaction or when a DMA channel loses channel arbitration the ME shall write the TCS back to the DMARAM.

18.3.5.1 ME Read Buffer

The ME stores the DMA read move data in eight 32-bit read registers. If a DMA channel is configured for 8-bit, 16-bit, 32-bit, 64-bit or 128-bit channel data width and a DMA read move is to a cached address[1)], the ME shall translate the read access to the on chip bus into a BTR4 access to a 32-byte aligned address. The ME stores the DMA read move data in the eight ME read registers together with the 32-byte aligned address to function as a ME read buffer as follows:

- **Hit**, if the 32-byte aligned address of the next DMA read move matches the 32-byte aligned address stored in the ME read buffer then the DMA read move data is read from the ME read buffer. No read access to the on chip bus occurs.
- **Miss**, if the 32-byte aligned address of the next DMA read move does not match the 32-byte aligned address stored in the ME read buffer then the ME invalidates the contents of the ME buffer. The ME executes the DMA read move by performing a read access to the on chip bus.

The ME invalidates the ME read buffer at the start of a DMA transfer.

18.3.5.1.1 DMA Address Checksum

If a DMA read move is to a cached address, then the ME shall calculate the **DMA Address Checksum** using the 32-byte aligned address source address.

18.3.5.2 ME Error Conditions

The ME reports TCS and source/destination error status. In the case of multiple errors, the error bits are set according to the error conditions (i.e. more than one error flag may be set). For all **ME Error Conditions**, the DMA shall perform the following actions:

- The ME shall record the number of the DMA channel in the ME last error channel bit field ERRSR.LEC.
- If the DMA channel is enabled for DMA channel hardware request then DMA channel TSR.HTRE is cleared.

DMARAM Integrity Error

When a DMA channel wins arbitration, a ME accesses the DMARAM to read the TCS. If an ECC error is detected, the ME shall set the error flag ERRSRm.RAMER. The DMA transaction shall not take place.

1) Segments 8 and 9 (see MEMMAP for details).

Source and Destination Errors

SER and **DER** include access protection errors and unsupported types of bus transaction.

- ME SER flag ERRSR.SER indicates an error occurred during a DMA read move from a source address.
- ME DER error flag ERRSR.DER indicates an error occurred during a DMA write move to a destination address.

If a SER or DER is reported then the ME completes the DMA transaction. If a SER is reported during a DMA read move then the DMA write move is not executed, but the destination address is updated.

- The ME bus error flag ERRSR.SPBER indicates an SPB bus error occurred during a DMA move.
- The ME bus error flag ERRSR.SRIER indicates an SRI bus error occurred during a DMA move.

Linked List Operation TCS Load Error

During DMALL, ACCL, SAFLL and CONLL operations if an error is reported during the loading of the next TCS from the on chip bus, the ME shall set the ME error flag ERRSR.DLLER. The linked list operation shall be aborted and the DMA channel transaction request state bit TSR.CH bit cleared.

SAFLL DMA Address Checksum Error

The ME compares the **DMA Address Checksum** calculated by the current DMA transaction against the expected **DMA Address Checksum** stored in the next TCS. If the checksums do not match then the ME shall set the ME error flag ERRSR.SLLER.

18.3.5.3 Error Interrupt Service Request

18.3.5.3.1 DMARAM Integrity Error Interrupt Service Request

A **RAMER** indicates that the ME detected a DMARAM integrity error:

- A RAMER is indicated by the ME error status flag ERRSRm.RAMER
- The DMA shall trigger a **DMA RP Error Interrupt Service Request**
- If software sets CLREm.CRAMER, the DMA shall clear error status fag ERRSRm.RAMER

18.3.5.3.2 Source and Destination Error Interrupt Service Request

The ME shall detect the following error conditions (see **Figure 222**):

- A **SER** indicates a bus error occurred during a DMA read move from a source address.
  - A SER is indicated by the ME error status flag ERRSRm.SER
  - If EERm.ESER is set, the DMA shall trigger a **DMA RP Error Interrupt Service Request**.
  - If software sets CLREm.CSER, the DMA shall clear error status flag ERRSRm.SER
- A **DER** indicates a bus error that occurred during a DMA write move to a destination address.
  - A DER is indicated by the ME error status flag ERRSRm.DER
  - If EERm.EDER is set, the DMA shall trigger a **DMA RP Error Interrupt Service Request**.
  - If software sets CLREm.CDER, the DMA shall clear error status flag ERRSRm.DER

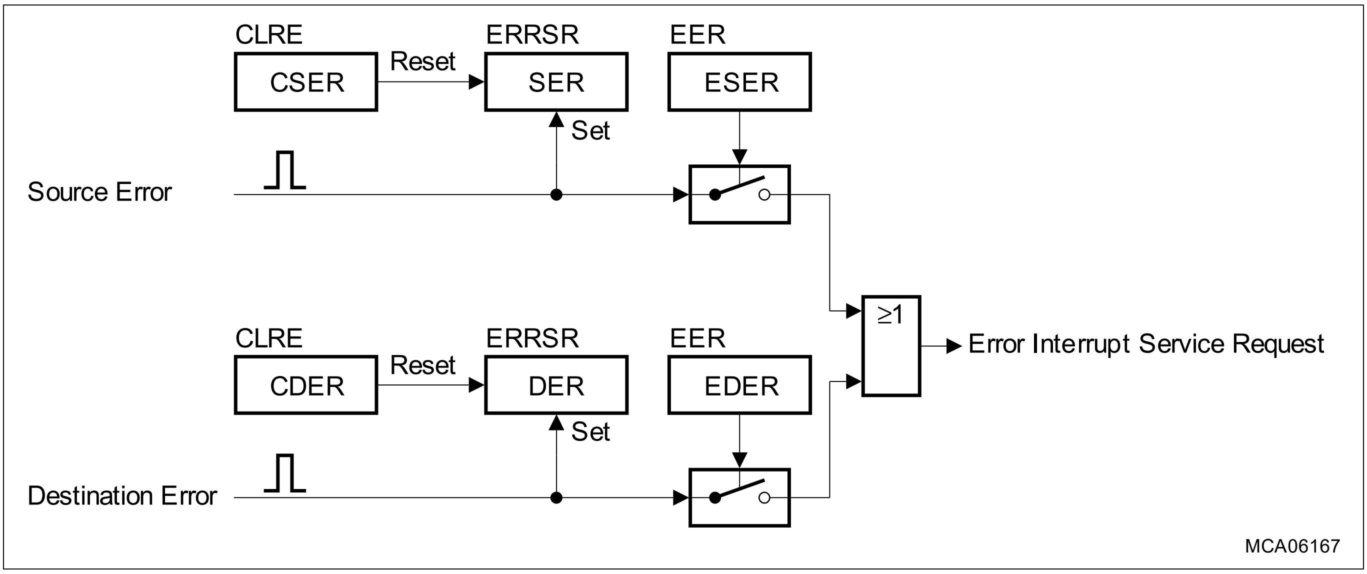

Figure 222 Source and Destination Error Interrupt Service Request

18.3.5.3.3 Linked List Operation TCS Error Interrupt Service Request

A **DLLER** indicates the ME detects an error when loading the next TCS from the on chip bus.

- A DLLER is indicated by the ME error status flag ERRSRm.DLLER
- The DMA shall trigger a **DMA RP Error Interrupt Service Request**.
- If software sets CLREm.CDLLER, the DMA shall clear error status flag ERRSRm.DLLER

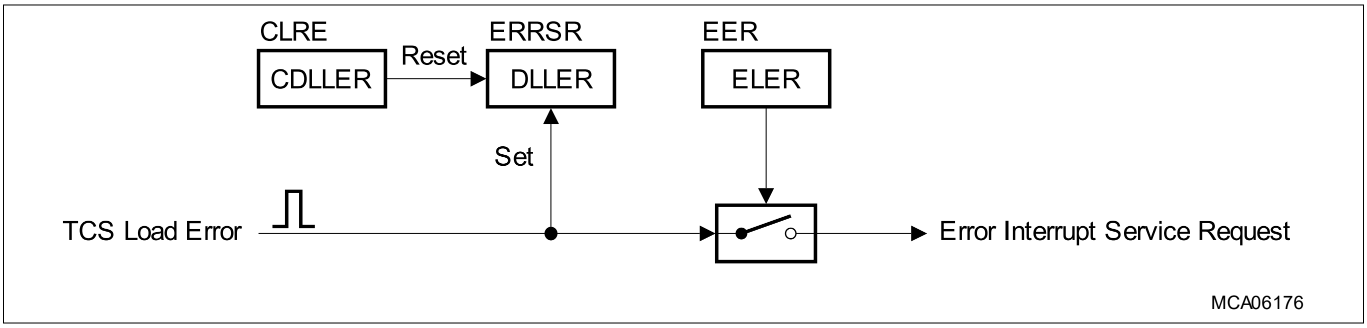

Figure 223 Linked List Operation TCS Error Interrupt Service Request

18.3.5.3.4 SAFLL DMA Address Checksum Error Interrupt Service Request

A **SLLER** indicates the ME detected a difference between the calculated and expected **DMA Address Checksum**.

- A SLLER is indicated by the ME error status flag ERRSRm.SLLER
- The DMA shall trigger a **DMA RP Error Interrupt Service Request**.
- If software sets CLREm.CSLLER, the DMA shall clear error status flag ERRSRm.SLLER

18.3.6 DMA On Chip Bus

18.3.6.1 DMA On Chip Bus Switch

The DMA on chip bus switch (at **Figure 224**) arbitrates between each on chip bus request from the ME and Cerberus. Address routing across the whole memory map directs a request to an on chip bus master interface.

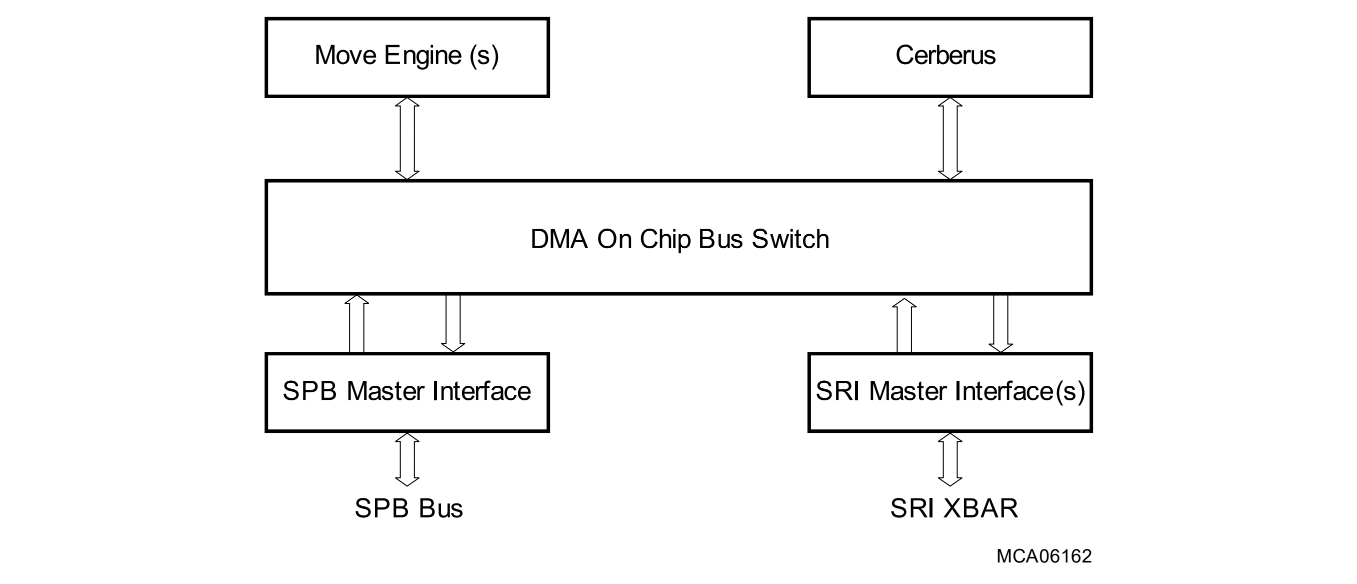

Figure 224 DMA On Chip Bus Switch

18.3.6.1.1 SRI Master Interfaces

For production devices the DMA shall instance one SRI master interface (MIF0). For emulation devices the DMA may instance additional SRI master interfaces (MIF1 and MIF2)[1] to be used as follows:

- DMA moves to EMEM Memory Banks 0 and 2 shall be routed through SRI master interface MIF1.
- DMA moves to EMEM Memory Banks 1 and 3 shall be routed through SRI master interface MIF2.

18.3.6.1.2 DMA On Chip Bus Switch Arbitration

If a ME and Cerberus request collide, the DMA on chip bus switch priorities (at **Table 577**) determine arbitration.

Table 577 DMA On Chip Bus Switch Priorities

Priority	Agent Requests	Comment
Highest	Cerberus to On Chip Bus High Priority.	Priority selection by software in Cerberus.
	ME write	For concurrent requests, the highest DMA channel wins.
	ME read	For concurrent requests, the highest DMA channel wins.
Lowest	Cerberus to On Chip Bus Low Priority.	Priority selection by software in Cerberus.

The on chip bus supports pipelining. High and low priority ME0, ME1 and Cerberus requests may be pipelined.

18.3.6.2 On Chip Bus Master Interfaces

The DMA instances on chip bus master interfaces to the SPB Bus and to the SRI Bus.

The DMA SPB master interface supports

- Single data read and write transactions (8-bit, 16-bit, 32-bit).
- Generation of interleaved FPI transactions from different access requesters (ME and Cerberus).
- De-assertion of request after retry in order to prevent bus blocking.

1) See MEMMAP and On-Chip System Connectivity for details.

- Out of order transactions from different sources in order to avoid side effects (blocking) between the different access requesters (ME and Cerberus).

The DMA SRI master interface supports

- Single data read and write transactions (8-bit, 16-bit, 32-bit, 64-bit)[1].
- Block transfer read and write transactions (128-bit, 256-bit)[2].
- Generation of interleaved SRI transactions from different access requesters (ME and Cerberus).

Read Modify Write (RMW) Support

The DMA does not support RMW accesses to the on chip buses.

18.3.6.3 SRI Alarm

The SRI bus protocol supports the reliable delivery of data between sources and destinations by extending the protocol to include ECC protection. During a DMA read move a ME checks the integrity of data received from SRI bus sources. A ME stores the DMA move data with ECC protection in the ME registers. During a DMA write move a ME generates ECC information compliant with the SRI bus protocol for write accesses to SRI destinations. A ME shall check the data integrity of DMA move data at the following locations:

- During a DMA read move the ME checks the integrity of data received from SRI source addresses.
- During a DMA write move the ME checks the integrity of data stored in the ME read register(s).

If a ME detects an ECC error then the DMA shall trigger an **SRI Alarm** to the SMU.

System Peripheral Bus

DMA read move data received from an SPB source is not ECC protected. A ME shall generate ECC protection before the ME stores the data and ECC in the ME read register(s).

Data Integrity Testing

Software may trigger an **SRI Alarm**. The CPU (CPUx_SEGEN.ADFLIP = 01_B and CPUx_SEGEN.ADTYPE = 10_B) may be used to generate an ECC error condition in a CPU slave interface used as a DMA read move source. When the data passes through the DMA it will generate an ECC error condition.

18.3.7 Power Modes

The DMA disable status bit (DMA_CLC.DISS) reports the state of the internal DMA clock signal (f_{DMA}).

After the assertion of an application reset, the DMA is enabled (DMA_CLC.DISS = 0_B) i.e. f_{DMA} is enabled.

18.3.7.1 Sleep Mode

The DMA shall enter **Sleep Mode** as follows:

- If the sleep mode enable control bit is enabled (DMA_CLC.EDIS = 0_B) AND the DMA sleep control is activated, the DMA shall enter **Sleep Mode** when the ME(s) have completed any pending DMA transfers.
- If software writes a DMA Disable Request (DMA_CLC.DISR = 1_B), the DMA shall enter **Sleep Mode** when the ME(s) have completed any pending DMA transfers.

If the DMA is in **Sleep Mode** (DMA_CLC.DISS = 1_B):

- The DMA shall disable f_{DMA} to minimize DMA power consumption.

1) 64-bit DMA move supported for DMA read move from SRI source to DMA write move to SRI destination.

2) Block transfer DMA move supported for DMA read move from SRI source to DMA write move to SRI destination.

- DMA SFRs
  - If the DMA is in **Sleep Mode** AND software writes to DMA_CLC, the DMA shall update DMA_CLC.
  - If the DMA is in **Sleep Mode** AND software writes to other DMA SFRs, the DMA shall return a bus error.
  - If the DMA is in **Sleep Mode** AND software reads a DMA SFR, the DMA shall complete the read access.
- DMARAM
  - If the DMA is in **Sleep Mode** AND software writes the DMARAM, the DMA shall return a bus error.
  - If the DMA is in **Sleep Mode** AND software reads the DMARAM, the DMA shall complete the read access.

If the DMA sleep control is de-activated AND software clears the DMA disable request (DMA_CLC.DISR = 0_B):

- The DMA shall enable f_{DMA}.
- The DMA shall resume the servicing of DMA requests.

18.4 Register

The DMA registers are accessed through the DMA slave interface.

Table 578 Register Address Space - DMA

Module	Base Address	End Address	Note
	$F0010000_H$	$F0013FFF_H$	FPI slave interface

Table 579 Register Overview - DMA (ascending Offset Address)

Short Name	Long Name	Offset Address	Access Mode		Reset	Page Number
			Read	Write		
CLC	DMA Clock Control Register	0000_H	U,SV	SV,E,P00,P01	Application Reset	60
ID	DMA Identification Register	0008_H	U,SV	BE	Application Reset	60
ACCENr0	RP r Access Enable Register 0	0040_H+r*8	U,SV	SV,SE	Application Reset	64
ACCENr1	RP r Access Enable Register 1	0044_H+r*8	U,SV	nBE	Application Reset	65
EERm	ME m Enable Error Register	0120_H+m*1000_H	U,SV	SV	Application Reset	80
ERRSRm	ME m Error Status Register	0124_H+m*1000_H	U,SV	BE	Application Reset	80
CLREm	ME m Clear Error Register	0128_H+m*1000_H	U,SV	SV	Application Reset	82
MEmSR	ME m Status Register	0130_H+m*1000_H	U,SV	BE	Application Reset	83
MEm0R	ME m Read Register 0	0140_H+m*1000_H	U,SV	BE	Application Reset	84
MEm1R	ME m Read Register 1	0144_H+m*1000_H	U,SV	BE	Application Reset	84
MEm2R	ME m Read Register 2	0148_H+m*1000_H	U,SV	BE	Application Reset	85
MEm3R	ME m Read Register 3	$014C_H$+m*1000_H	U,SV	BE	Application Reset	85
MEm4R	ME m Read Register 4	0150_H+m*1000_H	U,SV	BE	Application Reset	86
MEm5R	ME m Read Register 5	0154_H+m*1000_H	U,SV	BE	Application Reset	86
MEm6R	ME m Read Register 6	0158_H+m*1000_H	U,SV	BE	Application Reset	87
MEm7R	ME m Read Register 7	$015C_H$+m*1000_H	U,SV	BE	Application Reset	87

Table 579 Register Overview - DMA (ascending Offset Address) (cont'd)

Short Name	Long Name	Offset Address	Access Mode		Reset	Page Number
			Read	Write		
MEmRDCRC	ME m Channel Read Data CRC Register	0180_H+m *1000_H	U,SV	BE	Application Reset	87
MEmSDCRC	ME m Channel Source and Destination Address CRC Register	0184_H+m *1000_H	U,SV	BE	Application Reset	88
MEmSADR	ME m Channel Source Address Register	0188_H+m *1000_H	U,SV	BE	Application Reset	88
MEmDADR	ME m Channel Destination Address Register	$018C_H$+ m*1000_H	U,SV	BE	Application Reset	89
MEmADICR	ME m Channel Address and Interrupt Control Register	0190_H+m *1000_H	U,SV	BE	Application Reset	89
MEmCHCR	ME m Channel Control Register	0194_H+m *1000_H	U,SV	BE	Application Reset	91
MEmSHADR	ME m Channel Shadow Address Register	0198_H+m *1000_H	U,SV	BE	Application Reset	91
MEmCHSR	ME m Channel Status Register	$019C_H$+ m*1000_H	U,SV	BE	Application Reset	92
OTSS	DMA OCDS Trigger Set Select	1200_H	U,SV	SV	See page 61	61
PRR0	DMA Pattern Read Register 0	1208_H	U,SV	SV	Application Reset	62
PRR1	DMA Pattern Read Register 1	$120C_H$	U,SV	SV	Application Reset	62
TIME	DMA Time Register	1210_H	U,SV	BE	Application Reset	63
MODEr	RP r Mode Register	1300_H+r* 4	U,SV	SV,SE,P00, P01	Application Reset	63
ERRINTRr	RP r Error Interrupt Set Register	1320_H+r* 4	U,SV	SV,Pr	Application Reset	64
HRRc	DMA Channel c Resource Partition Register	1800_H+c *4	U,SV	SV,SE,P00, P01	Application Reset	65
SUSENRc	DMA Channel c Suspend Enable Register	$1A00_H$+c *4	U,SV	SV,E,Pr	See page 65	65
SUSACRc	DMA Channel c Suspend Acknowledge Register	$1C00_H$+c *4	U,SV	BE	See page 66	66
TSRc	DMA Channel c Transaction State Register	$1E00_H$+c *4	U,SV	SV,Pr	Application Reset	67
RDCRCRc	DMARAM Channel c Read Data CRC Register	2000_H+c *20_H	U,SV	SV,Pr	Application Reset	69

Table 579 Register Overview - DMA (ascending Offset Address) (cont'd)

Short Name	Long Name	Offset Address	Access Mode		Reset	Page Number
			Read	Write		
SDCRCRc	DMARAM Channel c Source and Destination Address CRC Register	2004_H+c *20_H	U,SV	SV,Pr	Application Reset	69
SADRc	DMARAM Channel c Source Address Register	2008_H+c *20_H	U,SV	SV,Pr	Application Reset	70
DADRc	DMARAM Channel c Destination Address Register	$200C_H$+c *20_H	U,SV	SV,Pr	Application Reset	70
ADICRc	DMARAM Channel c Address and Interrupt Control Register	2010_H+c *20_H	U,SV	SV,Pr	Application Reset	71
CHCFGRc	DMARAM Channel c Configuration Register	2014_H+c *20_H	U,SV	SV,Pr	Application Reset	75
SHADRc	DMARAM Channel c Shadow Address Register	2018_H+c *20_H	U,SV	SV,Pr	Application Reset	77
CHCSRc	DMARAM Channel c Control and Status Register	$201C_H$+c *20_H	U,SV	SV,Pr	Application Reset	77

18.4.1 Safety Flip-Flops

Safety flip-flops are special flip-flops that implement a hardware mechanism capable to detect single event effects that may lead to single event upsets (bit flip). The configuration and control registers that are implemented with safety flip-flops are:

- TSRc.CH

18.4.2 Register

DMA Clock Control Register

The Clock Control Register controls the internal f_{DMA} clock signal and sleep mode in order to control the power consumption. After the application of a reset, the DMA is enabled.

CLC

DMA Clock Control Register **(0000_H)** **Application Reset Value: $0000\ 0008_H$**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0												**EDIS**	**0**	**DISS**	**DISR**
r												rw	r	rh	rw

Field	Bits	Type	Description
DISR	0	rw	**Module Disable Request Bit** Used for enable/disable control of the DMA 0_B DMA enable is requested. 1_B DMA disable is requested.
DISS	1	rh	**Module Disable Status Bit** Bit indicates the current status of the DMA 0_B DMA is enabled. 1_B DMA is disabled.
EDIS	3	rw	**Sleep Mode Enable Control** Used for DMA sleep mode control. 0_B Sleep control enabled. DMA may enter sleep mode. 1_B Sleep control disabled. DMA shall not enter sleep mode.
0	2, 31:4	r	**Reserved** Read as 0; must be written with 0.

DMA Identification Register

ID

DMA Identification Register **(0008_H)** **Application Reset Value: 0087 C0XX$_H$**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MOD_NUMBER															
r															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MOD_TYPE								**MOD_REV**							
r								r							

Field	Bits	Type	Description
MOD_REV	7:0	r	**Module Revision Number** This bit field defines the module revision number.
MOD_TYPE	15:8	r	**Module Type** The bit field is set to $C0_H$ which defines the module as a 32-bit module.
MOD_NUMBER	31:16	r	**Module Number Value** This bit field defines a module identification number.

DMA OCDS Trigger Set Select

The OTSS register controls the selection of the OCDS Trigger Bus (OTGB).

The SCU controls the assertion of Power-on Reset to the OTSS register when OCDS is disabled or enabled. If OCDS is disabled then the OTSS register is reset by Power-on Reset else if OCDS is enabled then the OTSS register is not touched by Power-on Reset.

Write access requires OCDS to be enabled and Supervisor mode.

OTSS

DMA OCDS Trigger Set Select **(1200_H)** **Reset Value: Table 580**

31–16
0
r

15–8	7	6–4	3–0
0	**BS**	**0**	**TGS**
r	rw	r	rw

Field	Bits	Type	Description
TGS	3:0	rw	**Trigger Set for OTGB0 or OTGB1** 0_H No Trigger Set selected 1_H Trigger Set 1 2_H Trigger Set 2 3_H Reserved … 7_H Reserved 8_H Trigger Set 8 9_H Trigger Set 9 A_H Trigger Set 10 B_H Trigger Set 11 C_H Trigger Set 12 D_H Trigger Set 13 E_H Trigger Set 14 F_H Trigger Set 15
BS	7	rw	**OTGB0 or OTGB1 Bus Select** 0_B Trigger Set is output on OTGB0 1_B Trigger Set is output on OTGB1

Field	Bits	Type	Description
0	6:4, 31:8	r	**Reserved** Read as 0; must be written with 0.

Table 580 Reset Values of OTSS

Reset Type	Reset Value	Note
PowerOn Reset	0000 0000_H	
Debug Reset	0000 0000_H	

DMA Pattern Read Register 0

The Pattern Read Register 0 stores the pattern data (mask and/or compare bits) to be processed by the ME pattern compare logic.

PRR0

DMA Pattern Read Register 0 (1208_H) Application Reset Value: 0000 0000_H

31–24	23–16
PAT03	**PAT02**
rw	rw

15–8	7–0
PAT01	**PAT00**
rw	rw

Field	Bits	Type	Description
PAT00	7:0	rw	**Pattern Data Byte**
PAT01	15:8	rw	**Pattern Data Byte**
PAT02	23:16	rw	**Pattern Data Byte**
PAT03	31:24	rw	**Pattern Data Byte**

DMA Pattern Read Register 1

The Pattern Read Register 1 stores the pattern data (mask and/or compare bits) to be processed by the ME pattern compare logic.

PRR1

DMA Pattern Read Register 1 ($120C_H$) Application Reset Value: 0000 0000_H

31–24	23–16
PAT13	**PAT12**
rw	rw

15–8	7–0
PAT11	**PAT10**
rw	rw

Field	Bits	Type	Description
PAT10	7:0	rw	**Pattern Data Byte**
PAT11	15:8	rw	**Pattern Data Byte**
PAT12	23:16	rw	**Pattern Data Byte**
PAT13	31:24	rw	**Pattern Data Byte**

DMA Time Register

The Time Register stores the 32-bit count value used for the appendage of DMA timestamps.

TIME

DMA Time Register **(1210_H)** **Application Reset Value: $0000\ 0000_H$**

31–16
COUNT
r

15–0
COUNT
r

Field	Bits	Type	Description
COUNT	31:0	r	**Timestamp Count** The count value used during the appendage of DMA timestamps.

18.4.3 DMA Resource Partition Registers

RP r Mode Register

The Mode Register defines whether an RP accesses the on chip buses in supervisor mode or user mode.

MODEr (r=0-3)

RP r Mode Register **(1300_H+r*4)** **Application Reset Value: $0000\ 0001_H$**

31–16
0
r

15–1	0
0	MODE
r	rw

Field	Bits	Type	Description
MODE	0	rw	**Resource Partition Supervisor Mode** 0_B Bus master interface accesses on chip bus in user mode. 1_B Bus master interface accesses on chip bus in supervisor mode.

Field	Bits	Type	Description
0	31:1	r	**Reserved** Read as 0; must be written with 0.

RP r Error Interrupt Set Register

ERRINTRr (r=0-3)

RP r Error Interrupt Set Register **(1320_H+r*4)** **Application Reset Value: 0000 0000_H**

31–1	0
0	**SIT**
r	w

Field	Bits	Type	Description
SIT	0	w	**Set Error Interrupt Service Request** Reading this bit returns a 0. 0_B No action. 1_B DMA error interrupt service request will be activated.
0	31:1	r	**Reserved** Read as 0; must be written with 0.

RP r Access Enable Register 0

ACCENr0 (r=0-3)

RP r Access Enable Register 0 **(0040_H+r*8)** **Application Reset Value: FFFF $FFFF_H$**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
EN31	**EN30**	**EN29**	**EN28**	**EN27**	**EN26**	**EN25**	**EN24**	**EN23**	**EN22**	**EN21**	**EN20**	**EN19**	**EN18**	**EN17**	**EN16**
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EN15	**EN14**	**EN13**	**EN12**	**EN11**	**EN10**	**EN9**	**EN8**	**EN7**	**EN6**	**EN5**	**EN4**	**EN3**	**EN2**	**EN1**	**EN0**
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Field	Bits	Type	Description
ENq (q=0-31)	q	rw	**Access Enable for Master TAG ID q** This bit enables write access to the module kernel addresses for transactions with the Master TAG ID q 0_B Write access will not be executed 1_B Write access will be executed

RP r Access Enable Register 1

ACCENr1 (r=0-3)
RP r Access Enable Register 1 **(0044_H+r*8)** **Application Reset Value: 0000 0000_H**

31 … 16
0
r

15 … 0
0
r

Field	Bits	Type	Description
0	31:0	r	**Reserved** Read as 0; should be written with 0.

18.4.4 DMA Channel Registers

DMA Channel c Resource Partition Register

The Resource Partition Register assigns a DMA channel to a Resource Partition.

HRRc (c=000-127)
DMA Channel c Resource Partition Register **(1800_H+c*4)** **Application Reset Value: 0000 0000_H**

31 … 16
0
r

15 … 2	1 … 0
0	**HRP**
r	rw

Field	Bits	Type	Description
HRP	1:0	rw	**DMA Channel Resource Partition** 00_B Resource Partition 0 (RP0). 01_B Resource Partition 1 (RP1). 10_B Resource Partition 2 (RP2). 11_B Resource Partition 3 (RP3).
0	31:2	r	**Reserved** Read as 0; must be written with 0.

DMA Channel c Suspend Enable Register

The Suspend Enable Register enables/disables soft suspend mode capability.

SUSENRc (c=000-127)

DMA Channel c Suspend Enable Register (1A00$_H$+c*4) Reset Value: Table 581

31-16
0
r

15-1	0
0	SUSEN
r	rw

Field	Bits	Type	Description
SUSEN	0	rw	**Channel Suspend Enable for DMA Channel** Enables the DMA channel suspend capability. Channel suspend mode shall be terminated when SUSEN is written with 0. 0_B DMA channel is disabled for DMA channel suspend. The DMA channel does not react on an active suspend request signal SUSREQ. 1_B DMA channel is enabled for DMA channel suspend. If the suspend request signal SUSREQ becomes active, a DMA transaction of the DMA channel is stopped after the current DMA transfer has completed.
0	31:1	r	**Reserved** Read as 0; must be written with 0.

Table 581 Reset Values of SUSENRc (c=000-127)

Reset Type	Reset Value	Note
PowerOn Reset	0000 0000$_H$	
Debug Reset	0000 0000$_H$	

DMA Channel c Suspend Acknowledge Register

The Suspend Acknowledge Register indicates the DMA Channel soft suspend status.

SUSACRc (c=000-127)

DMA Channel c Suspend Acknowledge Register(1C00$_H$+c*4) Reset Value: Table 582

31-16
0
r

15-1	0
0	SUSAC
r	rh

Field	Bits	Type	Description
SUSAC	0	rh	**DMA Channel Suspend State or Frozen State Active for DMA Channel** Status bit indicates whether or not a DMA channel is in channel suspend state or in the frozen state. 0_B DMA channel is not in channel suspend state, frozen state or internal actions are not completed after the channel suspend state or frozen state was requested. 1_B DMA channel is in channel suspend state or frozen state.
0	31:1	r	**Reserved** Read as 0; must be written with 0.

Table 582 Reset Values of SUSACRc (c=000-127)

Reset Type	Reset Value	Note
PowerOn Reset	$0000\ 0000_H$	
Debug Reset	$0000\ 0000_H$	

DMA Channel c Transaction State Register

If an application reset is asserted, the DMA shall set DMA channel reset to all DMA channels. As soon as a DMA channel has entered the reset state, the DMA shall clear the DMA channel bit.

TSRc (c=000-127)

DMA Channel c Transaction State Register (1E00$_H$+c*4) Application Reset Value: 0000 0000$_H$

31–25	24	23–19	18	17	16
0	HLTCLR	0	CTL	DCH	ECH
r	w	r	w	w	w

15–10	9	8	7–5	4	3	2	1	0
0	HLTACK	HLTREQ	0	ETRL	CH	TRL	HTRE	RST
r	rh	rwh	r	rw	rh	rh	rh	rwh

Field	Bits	Type	Description
RST	0	rwh	**DMA Channel Reset** The DMA channel reset bit is set by software (DMA channel TSR.RST = 1) and cleared by hardware when the DMA channel has been reset. 0_B After the application of a DMA channel reset, the DMA channel is in the DMA channel reset state. Software write to 0 has no effect. 1_B A DMA channel reset is pending. See Functional Description.

Field	Bits	Type	Description
HTRE	1	rh	**DMA Channel Hardware Request Enable** *Note: See Functional Description for enable and disable. When a DMA channel is configured for single mode, HTRE is reset when ME CHSR.TCOUNT is decremented and ME CHSR.TCOUNT = 0. When a DMA channel error is reported or a pattern match is detected, DMA channel HTRE is reset.* 0_B DMA hardware request is disabled for DMA channel. 1_B DMA hardware request is enabled for DMA channel.
TRL	2	rh	**DMA Channel Transaction/Transfer Request Lost** This bit is reset by software clearing TRL (writing DMA channel TSR.CTL = 1) or resetting the DMA channel (writing DMA channel TSR.RST = 1). 0_B No TRL event has been detected for DMA channel. 1_B TRL event has been detected for DMA channel.
CH	3	rh	**DMA Channel Transaction Request State** CH is reset when a pattern match is detected. 0_B No DMA request is pending for DMA channel. 1_B DMA request is pending for DMA channel.
ETRL	4	rw	**Enable DMA Channel Transaction/Transfer Request Lost Interrupt** DMA channel control bit to enable the generation of an error interrupt service request when DMA channel TSR.TRL is set. 0_B Interrupt generation for DMA channel TRL event is disabled. 1_B Interrupt generation for DMA channel TRL event is enabled.
HLTREQ	8	rwh	**DMA Channel Halt Request** The DMA channel halt request bit is set by software (writing DMA channel TSR.HLTREQ = 1) and cleared by software (writing DMA channel TSR.HLTCLR = 1). 0_B No action. 1_B Halt request.
HLTACK	9	rh	**DMA Channel Halt Acknowledge** 0_B DMA channel is not halted. 1_B DMA channel is halted.
ECH	16	w	**Enable DMA Channel Hardware Transaction Request** See Functional Description. Reading this bit returns a 0.
DCH	17	w	**Disable DMA Channel Hardware Transaction Request** See Functional Description. Reading this bit returns a 0.
CTL	18	w	**Clear DMA Channel Transaction/Transfer Request Lost** Software clear of the DMA channel TRL status flag. Reading this bit returns a 0. 0_B No action. 1_B Clear DMA channel TRL flag (TSR.TRL).

Field	Bits	Type	Description
HLTCLR	24	w	**Clear DMA Channel Halt Request and Acknowledge** Reading this bit returns a 0. 0_B No action. 1_B Clear DMA channel halt request (TSR.HLTREQ) and halt acknowledge (TSR.HLTACK).
0	7:5, 15:10, 23:19, 31:25	r	**Reserved** Read as 0; must be written with 0.

18.4.5 DMARAM Channel Registers

DMARAM Channel c Read Data CRC Register

RDCRCRc (c=000-127)

DMARAM Channel c Read Data CRC Register (2000$_H$+c*20$_H$) Application Reset Value: 0000 0000$_H$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RDCRC															
rwh															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDCRC															
rwh															

Field	Bits	Type	Description
RDCRC	31:0	rwh	**Read Data CRC** Checksum calculated for DMA read move data.

DMARAM Channel c Source and Destination Address CRC Register

SDCRCRc (c=000-127)

DMARAM Channel c Source and Destination Address CRC Register(2004$_H$+c*20$_H$) Application Reset Value: 0000 0000$_H$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SDCRC															
rwh															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SDCRC															
rwh															

Field	Bits	Type	Description
SDCRC	31:0	rwh	**Source and Destination Address CRC** Checksum calculated for DMA move source and destination addresses.

DMARAM Channel c Source Address Register

SADRc (c=000-127)
DMARAM Channel c Source Address Register (2008_H+c*20_H) **Application Reset Value: 0000 0000_H**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SADR															
rwh															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SADR															
rwh															

Field	Bits	Type	Description
SADR	31:0	rwh	**Source Address** 32-bit source address.

DMARAM Channel c Destination Address Register

DADRc (c=000-127)
DMARAM Channel c Destination Address Register($200C_H$+c*20_H) **Application Reset Value: 0000 0000_H**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DADR															
rwh															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DADR															
rwh															

Field	Bits	Type	Description
DADR	31:0	rwh	**Destination Address** 32-bit destination address.

DMARAM Channel c Address and Interrupt Control Register

ADICRc (c=000-127)

DMARAM Channel c Address and Interrupt Control Register(2010_H+c*20_H) Application Reset Value: 0000 0000_H

31-28	27-26	25	24	23	22	21	20	19-16
IRDV	INTCT	WRPDE	WRPSE	0	STAMP	DCBE	SCBE	SHCT
rwh	rwh	rwh	rwh	r	rwh	rwh	rwh	rwh

15-12	11-8	7	6-4	3	2-0
CBLD	CBLS	INCD	DMF	INCS	SMF
rw	rw	rwh	rwh	rwh	rwh

Field	Bits	Type	Description
SMF	2:0	rwh	**Source Address Modification Factor** DMA channel TCS 32-bit source address modification factor and the channel data width CHDW determines an address offset value by which the source address is modified after each DMA move. If SCBE = 1_B and CBLS = 0000_B then the source address is not modified. 000_B Address offset is 1 x CHDW 001_B Address offset is 2 x CHDW 010_B Address offset is 4 x CHDW 011_B Address offset is 8 x CHDW 100_B Address offset is 16 x CHDW 101_B Address offset is 32 x CHDW 110_B Address offset is 64 x CHDW 111_B Address offset is 128 x CHDW
INCS	3	rwh	**Increment of Source Address** DMA channel TCS control bit to determine if the address offset selected by SMF will be added to or subtracted from the source address after each DMA move. If SCBE = 1_B and CBLS = 0000_B then the source address is not modified. 0_B Address offset will be subtracted. 1_B Address offset will be added.

Field	Bits	Type	Description
DMF	6:4	rwh	**Destination Address Modification Factor** DMA channel TCS 32-bit destination address modification factor and the channel data width CHDW determines an address offset value by which the destination address is modified after each DMA move. If DCBE = 1_B and CBLD = 0000_B then the destination address is not modified. 000_B Address offset is 1 x CHDW 001_B Address offset is 2 x CHDW 010_B Address offset is 4 x CHDW 011_B Address offset is 8 x CHDW 100_B Address offset is 16 x CHDW 101_B Address offset is 32 x CHDW 110_B Address offset is 64 x CHDW 111_B Address offset is 128 x CHDW
INCD	7	rwh	**Increment of Destination Address** DMA channel TCS control bit to determine if the address offset selected by DMF will be added to or subtracted from the destination address after each DMA move. If DCBE = 1_B and CBLD = 0000_B the destination address is not modified. 0_B Address offset will be subtracted. 1_B Address offset will be added.
CBLS	11:8	rw	**Circular Buffer Length Source** DMA channel TCS circular buffer source address update control bit determines which part of the 32-bit source address register remains unchanged and is not updated after a DMA move operation. *Note: CBLS determines the size of the circular source buffer.* 0_H Source address SADR[31:0] is not updated 1_H Source address SADR[31:1] is not updated 2_H Source address SADR[31:2] is not updated 3_H Source address SADR[31:3] is not updated 4_H Source address SADR[31:4] is not updated 5_H Source address SADR[31:5] is not updated 6_H Source address SADR[31:6] is not updated 7_H Source address SADR[31:7] is not updated 8_H Source address SADR[31:8] is not updated 9_H Source address SADR[31:9] is not updated A_H Source address SADR[31:10] is not updated B_H Source address SADR[31:11] is not updated C_H Source address SADR[31:12] is not updated D_H Source address SADR[31:13] is not updated E_H Source address SADR[31:14] is not updated F_H Source address SADR[31:15] is not updated

Field	Bits	Type	Description
CBLD	15:12	rw	**Circular Buffer Length Destination** DMA channel TCS circular buffer destination address update control bit determines which part of the 32-bit destination address register remains unchanged and is not updated after a DMA move operation. *Note: CBLD determines the size of the circular destination buffer.* 0_H Destination address DADR[31:0] is not updated 1_H Destination address DADR[31:1] is not updated 2_H Destination address DADR[31:2] is not updated 3_H Destination address DADR[31:3] is not updated 4_H Destination address DADR[31:4] is not updated 5_H Destination address DADR[31:5] is not updated 6_H Destination address DADR[31:6] is not updated 7_H Destination address DADR[31:7] is not updated 8_H Destination address DADR[31:8] is not updated 9_H Destination address DADR[31:9] is not updated A_H Destination address DADR[31:10] is not updated B_H Destination address DADR[31:11] is not updated C_H Destination address DADR[31:12] is not updated D_H Destination address DADR[31:13] is not updated E_H Destination address DADR[31:14] is not updated F_H Destination address DADR[31:15] is not updated
SHCT	19:16	rwh	**Shadow Control** DMA channel TCS shadow control determines the function of the shadow address register. 0_H Move Operation. 1_H Shadow Operation Read Only Mode Source Address. 2_H Shadow Operation Read Only Mode Destination Address. 3_H Reserved. 4_H Reserved. 5_H Shadow Operation Direct Write Mode Source Address. 6_H Shadow Operation Direct Write Mode Destination Address. 7_H Reserved. 8_H DMA Double Source Buffering with Software Switch Only. 9_H DMA Double Source Buffering with Software Switch and Automatic Hardware Switch. A_H DMA Double Destination Buffering with Software Switch Only. B_H DMA Double Destination Buffering with Software Switch and Automatic Hardware Switch. C_H DMA Linked List (DMALL). D_H Accumulated Linked List (ACCLL). E_H Safe Linked List (SAFLL). F_H Conditional Linked List (CONLL).
SCBE	20	rwh	**Source Circular Buffer Enable** DMA channel TCS source circular buffer enable. 0_B Source circular buffer disabled. 1_B Source circular buffer enabled.

Field	Bits	Type	Description
DCBE	21	rwh	**Destination Circular Buffer Enable** DMA channel TCS destination circular buffer enable. 0_B Destination circular buffer disabled. 1_B Destination circular buffer enabled.
STAMP	22	rwh	**Time Stamp** DMA channel TCS control bit to enable the appendage of a timestamp after the end of the last DMA Move during a DMA transaction. 0_B No action. 1_B DMA timestamp is appended.
WRPSE	24	rwh	**Wrap Source Enable** DMA channel TCS source buffer interrupt trigger enable/disable. 0_B Wrap source buffer interrupt trigger disabled. 1_B Wrap source buffer interrupt trigger enabled.
WRPDE	25	rwh	**Wrap Destination Enable** DMA channel TCS destination buffer interrupt trigger enable/disable. 0_B Wrap destination buffer interrupt trigger disabled. 1_B Wrap destination buffer interrupt trigger enabled.
INTCT	27:26	rwh	**Interrupt Control** DMA channel TCS interrupt control. *Note: If DMA channel CHCFGR.PRSEL = 1_B for the next lower priority channel then the channel transfer trigger interrupt is disabled.* 00_B No interrupt trigger will be generated on changing the TCOUNT value. The DMA channel CHSR.ICH is set when TCOUNT equals IRDV. 01_B No interrupt trigger will be generated on changing the TCOUNT value. The DMA channel CHSR.ICH is set when TCOUNT is decremented 10_B Interrupt trigger is generated and DMA channel CHSR.ICH is set on changing the TCOUNT value and TCOUNT equals IRDV 11_B Interrupt trigger is generated and DMA channel CHSR.ICH is set each time TCOUNT is decremented
IRDV	31:28	rwh	**Interrupt Raise Detect Value** DMA channel TCS interrupt threshold value defines the Threshold Limit of CHSR.TCOUNT for which a channel interrupt trigger will be raised.
0	23	r	**Reserved** Read as 0; must be written with 0.

DMARAM Channel c Configuration Register

CHCFGRc (c=000-127)

DMARAM Channel c Configuration Register (2014$_H$+c*20$_H$) Application Reset Value: 0000 0000$_H$

31–29	28	27	26–24	23–21	20	19	18–16
0	PRSEL	SWAP	PATSEL	CHDW	CHMODE	RROAT	BLKM
r	rwh	rw	rwh	rwh	rwh	rwh	rwh

15–14	13–0
0	TREL
r	rwh

Field	Bits	Type	Description
TREL	13:0	rwh	**Transfer Reload Value** DMA channel TCS transfer reload value to control the number of DMA transfers in a DMA transaction. The 14-bit transfer count value is loaded into ME CHSR.TCOUNT at the start of a DMA transaction (when TSR.CH becomes set and CHSR.TCOUNT = 0). A write to CHCFGR.TREL during a running DMA transaction has no influence on the running DMA transaction. If CHCFGR.TREL = 0 or if CHCFGR.TREL = 1 then ME CHSR.TCOUNT will be loaded with 1 when a new DMA transaction is started (at least one DMA transfer must be executed per DMA transaction).
BLKM	18:16	rwh	**Block Mode** Defines the number of DMA moves executed during one DMA transfer. 000$_B$ One DMA transfer has 1 DMA move 001$_B$ One DMA transfer has 2 DMA moves 010$_B$ One DMA transfer has 4 DMA moves 011$_B$ One DMA transfer has 8 DMA moves 100$_B$ One DMA transfer has 16 DMA moves 101$_B$ One DMA transfer has 3 DMA moves 110$_B$ One DMA transfer has 5 DMA moves 111$_B$ One DMA transfer has 9 DMA moves
RROAT	19	rwh	**Reset Request Only After Transaction** DMA channel control bit to determine if the DMA request state flag (DMA channel TSR.CH) is reset after each DMA transfer. 0$_B$ DMA channel TSR.CH is reset after the start of each DMA transfer. A DMA request is required for each DMA transfer. 1$_B$ DMA channel TSR.CH is reset when CHSR.TCOUNT = 0 and after the completion of the last DMA transfer (i.e. on completion of the DMA transaction). One DMA request starts a complete DMA transaction.

Field	Bits	Type	Description
CHMODE	20	rwh	**Channel Operation Mode** DMA channel TCS control to determine TSR.HTRE reset condition. 0_B **Single_Mode**, is selected for DMA channel. After a DMA transaction, DMA channel is disabled for further hardware requests (TSR.HTRE is reset by hardware) TSR.HTRE must be set again by software for starting a new transaction. 1_B **Continuous_Mode**, is selected for DMA channel. After a DMA transaction, bit TSR.HTRE remains set.
CHDW	23:21	rwh	**Channel Data Width** DMA channel TCS data width for DMA read moves and DMA write moves. 000_B 8-bit data width for moves selected *Note: Single Data Transfer Byte (SDTB)* 001_B 16-bit data width for moves selected *Note: Single Data Transfer Half-Word (SDTH)* 010_B 32-bit data width for moves selected *Note: Single Data Transfer Word (SDTW)* 011_B 64-bit data width transaction selected *Note: SRI-Bus: Single Data Transfer Double Word (SDTD)* *Note: SPB-Bus: not supported.* 100_B 128-bit data width transaction selected *Note: SRI-Bus: Block Transfer Request - 2 Transfers (BTR2)* *Note: SPB-Bus: not supported.* 101_B 256-bit data width transaction selected *Note: SRI-Bus: Block Transfer Request - 4 Transfers (BTR4)* *Note: SPB-Bus: not supported.* 110_B Reserved. 111_B Reserved.
PATSEL	26:24	rwh	**Pattern Select** DMA channel TCS bit field to select the pattern detection operation (see Functional Description). If PATSEL[1:0] is not equal to 00_B then a ME pattern detection operation defined by the channel data width (CHDW) will be performed. PATSEL[2] selects the pattern read register. If a pattern match is detected then a DMA channel interrupt shall be triggered. 000_B No pattern compare operation. 001_B DMA read move data compared with PRR0. … 011_B DMA read move data compared with PRR0. 100_B No pattern compare operation. 101_B DMA read move data compared with PRR1. … 111_B DMA read move data compared with PRR1.
SWAP	27	rw	**Swap Data CRC Byte Order** DMA channel TCS swap data CRC byte order. 0_B Byte order is not swapped 1_B Byte order is swapped.

Field	Bits	Type	Description
PRSEL	28	rwh	**Peripheral Request Select** DMA channel TCS control bit field to select the source of a DMA request. 0_B DMA hardware request selected. 1_B DMA daisy chain request selected.
0	15:14, 31:29	r	**Reserved** Read as 0; must be written with 0.

DMARAM Channel c Shadow Address Register

SHADRc (c=000-127)
DMARAM Channel c Shadow Address Register(2018_H+c*20_H) Application Reset Value: 0000 0000_H

31–16
SHADR
rwh

15–0
SHADR
rwh

Field	Bits	Type	Description
SHADR	31:0	rwh	**Shadowed Address** 32-bit shadow address.

DMARAM Channel c Control and Status Register

The CHCSR bit fields are used for two functions:

1. Storing DMA channel status bits.
2. Write only triggers to set and clear DMA channel configuration and status.

CHCSRc (c=000-127)
DMARAM Channel c Control and Status Register($201C_H$+c*20_H) Application Reset Value: 0000 0000_H

31	30–28	27	26	25	24	23	22	21–20	19	18	17	16
SCH	**0**	**SIT**	**CICH**	**CWRP**	**SWB**	**FROZEN**	**BUFFER**	**0**	**IPM**	**ICH**	**WRPD**	**WRPS**
w	r	w	w	w	w	rwh	rh	r	rh	rh	rh	rh

15	14	13–0
LXO	**0**	**TCOUNT**
rh	r	rh

Field	Bits	Type	Description
TCOUNT	13:0	rh	**Transfer Count** DMA channel status transfer count updated after DMARAM write back.

Field	Bits	Type	Description
LXO	15	rh	**Old Value of Pattern Detection** DMA channel status bit to store the result of a pattern detection operation when 8-bit or 16-bit data width is selected. 0_B The corresponding pattern compare operation did not find a pattern match during the previous DMA read move. 1_B The corresponding pattern compare operation found a pattern match during the previous DMA read move.
WRPS	16	rh	**Wrap Source Buffer** Status bit indicates that a DMA channel has reached a wrap source buffer boundary. Bit is reset by software (DMA channel CHCSR.CWRP = 1_B or DMA channel reset TSR.RST = 1_B). 0_B No wrap source buffer occurred. 1_B Wrap source buffer occurred.
WRPD	17	rh	**Wrap Destination Buffer** Status bit indicates that a DMA channel has reached a wrap destination buffer boundary. Bit is reset by software (DMA channel CHCSR.CWRP = 1_B or DMA channel reset TSR.RST = 1_B). 0_B No wrap destination buffer occurred. 1_B Wrap destination buffer occurred.
ICH	18	rh	**Interrupt from Channel** As soon as ME CHSR.TCOUNT decrements, if ADICR.INTCT[0] = 0 and CHSR.COUNT = IRDV then ME CHSR.ICH is set else ME CHSR.ICH is set for each decrement of CSR.TCOUNT. Bit is reset by software (DMA channel CHCSR.CICH = 1_B or by a DMA channel reset TSR.RST = 1_B). 0_B An interrupt from channel has not been detected. 1_B An interrupt from channel has been detected.
IPM	19	rh	**Pattern Detection from Channel** Status bit indicates that a pattern match has been detected for the DMA channel when pattern detection is enabled. This bit is reset by software (DMA channel CHCSR.CICH = 1_B or DMA channel reset TSR.RST = 1_B). 0_B A pattern match has not been detected. 1_B A pattern match has been detected.
BUFFER	22	rh	**DMA Double Buffering Active Buffer** During a DMA double buffering operation, the status bit indicates which buffer is read or filled. 0_B Buffer 0 read or filled by DMA. 1_B Buffer 1 read or filled by DMA.
FROZEN	23	rwh	**DMA Double Buffering Frozen Buffer** If a DMA channel is configured for double buffering operation, the FROZEN bit indicates that one of the buffers is frozen and available for processing by a cyclic software task. *Note: FROZEN bit shall only be set by the DMA and shall only be cleared by software.* 0_B Buffer is not frozen. 1_B Buffer is frozen.

Field	Bits	Type	Description
SWB	24	w	**DMA Double Buffering Switch Buffer** When DMA double buffering is configured, the control bit is used to re-direct data from one buffer to the other buffer. *Note: If a DMALL, ACCLL, SAFLL or CONLL operation is configured then SWB shall be 0_B.* 0_B No action. 1_B Switch from buffer.
CWRP	25	w	**Clear Wrap Buffer Interrupt** Software clear of the DMA channel source and destination wrap buffer flags stored at CHCSR.WRPS and CHCSR.WRPD. If the DMA channel is active in a ME then clear ME bit fields CHSR.WRPS and CHSR.WRPD. Reading this bit returns a 0. 0_B No action. 1_B Clear DMA channel bits CSR.WRPS and CSR.WRPD.
CICH	26	w	**Clear Interrupt for DMA Channel** Software clear of the DMA channel flags stored at CHCSR.ICH and CHCSR.IPM. If the DMA channel is active in a ME then clear ME bit fields CHSR.ICH and CHSR.IPM. Reading this bit returns a 0. 0_B No action. 1_B Clear DMA channel bits CSR.ICH and CSR.IPM.
SIT	27	w	**Set Interrupt Trigger for DMA Channel** Reading this bit returns a 0. *Note: If a DMALL, ACCLL, SAFLL or CONLL operation is configured then SIT must be 0_B.* 0_B No action. 1_B DMA channel interrupt trigger will be activated.
SCH	31	w	**Set Transaction Request** Reading this bit returns a 0. 0_B No action. 1_B DMA software request initiated for DMA channel. When SCH is set, DMA channel TSR.CH is set to indicate a DMA request is pending.
0	14, 21:20, 30:28	r	**Reserved** Read as 0; must be written with 0.

18.4.6 ME Registers

ME m Enable Error Register

The Enable Error Register enables an error interrupt service request for ME errors.

EERm (m=0-1)
ME m Enable Error Register **(0120_H+m*1000_H)** **Application Reset Value: 0403 0000_H**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0					**ELER**	**0**								**EDER**	**ESER**
r					rw	r								rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0															
r															

Field	Bits	Type	Description
ESER	16	rw	**Enable ME Source Error** This bit enables the generation of a ME source error interrupt. 0_B ME source error interrupt is disabled. 1_B ME source error interrupt is enabled.
EDER	17	rw	**Enable ME Destination Error** This bit enables the generation of a ME destination error interrupt. 0_B ME destination error interrupt is disabled. 1_B ME destination error interrupt is enabled.
ELER	26	rw	**Enable ME DMA Linked List Error** This bit enables the generation of a ME DMA Linked List error interrupt. 0_B ME DMA Linked List error interrupt is disabled. 1_B ME DMA Linked List error interrupt is enabled.
0	15:0, 25:18, 31:27	r	**Reserved** Read as 0; must be written with 0.

ME m Error Status Register

The Error Status Register indicates if the DMA controller could not service a DMA request. It indicates that an SPB Bus access or an SRI Bus access has terminated with errors.

ERRSRm (m=0-1)

ME m Error Status Register **(0124_H+m*1000_H)** **Application Reset Value: 0000 0000_H**

31–27	26	25	24	23–22	21	20	19–18	17	16
0	DLLER	SLLER	RAMER	0	SRIER	SPBER	0	DER	SER
r	rh	rh	rh	r	rh	rh	r	rh	rh

15–7	6–0
0	LEC
r	rh

Field	Bits	Type	Description
LEC	6:0	rh	**ME Last Error Channel** This bit field indicates the DMA channel number of the last DMA channel of ME generating an error i.e. RAM error DMA_ERRSRm.RAMER, Safe Linked List error DMA_ERRSRm.SLLER, DMA Linked List error DMA_ERRSRm.DLLER and all on chip bus errors.
SER	16	rh	**ME Source Error** This bit is set whenever a ME error occurred during a source (read) move of a DMA transfer, or a request could not been serviced due to the access protection. 0_B No ME source error has occurred. 1_B ME source error has occurred.
DER	17	rh	**ME Destination Error** This bit is set whenever a ME error occurred during a destination (write) move of a DMA transfer, or a request could not been serviced due to the access protection. 0_B No ME destination error has occurred. 1_B ME destination error has occurred.
SPBER	20	rh	**ME SPB Bus Error** This bit is set when a ME DMA Move that has been started by the DMA SPB master interface leads to an error on the SPB Bus. 0_B No error occurred. 1_B An error occurred on SPB Bus interface.
SRIER	21	rh	**ME SRI Bus Error** This bit is set when a ME DMA Move that has been started by the DMA SRI master interface leads to an error on the SRI Bus. 0_B No error occurred. 1_B An error occurred on SRI Bus interface.
RAMER	24	rh	**ME RAM Error** This bit is set whenever a ME error occurred during the loading of a TCS from the DMARAM to the ME channel registers. 0_B No error occurred. 1_B An error occurred during the load of a TCS.

Field	Bits	Type	Description
SLLER	25	rh	**ME Safe Linked List Error** This bit is set when a ME error occurred during the comparison of a SDCRC checksums during a safe linked list. 0_B No error occurred. 1_B An error occurred during the SDCRC checksum comparison.
DLLER	26	rh	**ME DMA Linked List Error** This bit is set when a ME error occurred during a DMALL, ACCLL, SAFLL or CONLL operation when a new TCS is loaded from anywhere in memory to overwrite the current TCS stored in the DMARAM. 0_B No error occurred. 1_B An error occurred during the loading of a new TCS.
0	15:7, 19:18, 23:22, 31:27	r	**Reserved** Read as 0; must be written with 0.

ME m Clear Error Register

The Clear Error Register contains bits to clear the corresponding ME error flags.

CLREm (m=0-1)

ME m Clear Error Register **(0128_H+m*1000_H)** **Application Reset Value: 0000 0000_H**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
		0			**CDLLER**	**CSLLER**	**CRAMER**	**0**		**CSRIER**	**CSPBER**	**0**		**CDER**	**CSER**
		r			w	w	w	r		w	w	r		w	w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
							0								
							r								

Field	Bits	Type	Description
CSER	16	w	**Clear ME Source Error** 0_B No action 1_B Clear source error flag DMA_ERRSRm.SER.
CDER	17	w	**Clear ME Destination Error** 0_B No action 1_B Clear destination error flag DMA_ERRSRm.DER.
CSPBER	20	w	**Clear SPB Error** 0_B No action 1_B Clear error flag DMA_ERRSRm.SPBER.
CSRIER	21	w	**Clear SRI Error** 0_B No action 1_B Clear error flag DMA_ERRSRm.SRIER.

Field	Bits	Type	Description
CRAMER	24	w	**Clear RAM Error** 0_B No action 1_B Clear error flag DMA_ERRSRm.RAMER.
CSLLER	25	w	**Clear SLL Error** 0_B No action 1_B Clear error flag DMA_ERRSRm.SLLER.
CDLLER	26	w	**Clear DLL Error** 0_B No action 1_B Clear error flag DMA_ERRSRm.DLLER.
0	15:0, 19:18, 23:22, 31:27	r	**Reserved** Read as 0; must be written with 0.

ME m Status Register

The ME Status Register holds status information about the DMA transaction handled by the ME.

MEmSR (m=0-1)

ME m Status Register **(0130_H+m*1000_H)** **Application Reset Value: 0000 0000_H**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0									**CH**						
r									rh						

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0											**WS**	**0**			**RS**
r											rh	r			rh

Field	Bits	Type	Description
RS	0	rh	**ME Read Status** 0_B ME is not performing a DMA read move. 1_B ME is performing a DMA read move.
WS	4	rh	**ME Write Status** 0_B ME is not performing a DMA write move. 1_B ME is performing a DMA write move.
CH	22:16	rh	**ME Active Channel** Indicates the number of the DMA Channel currently processed by the ME.
0	3:1, 15:5, 31:23	r	**Reserved** Read as 0; must be written with 0.

ME m Read Register 0

MEm0R (m=0-1)
ME m Read Register 0 **(0140_H+m*1000_H)** **Application Reset Value: 0000 0000_H**

31–24	23–16
RD03	**RD02**
rh	rh

15–8	7–0
RD01	**RD00**
rh	rh

Field	Bits	Type	Description
RD00	7:0	rh	**DMA Read Move Data Byte**
RD01	15:8	rh	**DMA Read Move Data Byte**
RD02	23:16	rh	**DMA Read Move Data Byte**
RD03	31:24	rh	**DMA Read Move Data Byte**

ME m Read Register 1

MEm1R (m=0-1)
ME m Read Register 1 **(0144_H+m*1000_H)** **Application Reset Value: 0000 0000_H**

31–24	23–16
RD13	**RD12**
rh	rh

15–8	7–0
RD11	**RD10**
rh	rh

Field	Bits	Type	Description
RD10	7:0	rh	**DMA Read Move Data Byte**
RD11	15:8	rh	**DMA Read Move Data Byte**
RD12	23:16	rh	**DMA Read Move Data Byte**
RD13	31:24	rh	**DMA Read Move Data Byte**

ME m Read Register 2

MEm2R (m=0-1)
ME m Read Register 2 (0148$_H$+m*1000$_H$) **Application Reset Value: 0000 0000$_H$**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RD23								RD22							
rh								rh							

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RD21								RD20							
rh								rh							

Field	Bits	Type	Description
RD20	7:0	rh	**DMA Read Move Data Byte**
RD21	15:8	rh	**DMA Read Move Data Byte**
RD22	23:16	rh	**DMA Read Move Data Byte**
RD23	31:24	rh	**DMA Read Move Data Byte**

ME m Read Register 3

MEm3R (m=0-1)
ME m Read Register 3 (014C$_H$+m*1000$_H$) **Application Reset Value: 0000 0000$_H$**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RD33								RD32							
rh								rh							

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RD31								RD30							
rh								rh							

Field	Bits	Type	Description
RD30	7:0	rh	**DMA Read Move Data Byte**
RD31	15:8	rh	**DMA Read Move Data Byte**
RD32	23:16	rh	**DMA Read Move Data Byte**
RD33	31:24	rh	**DMA Read Move Data Byte**

ME m Read Register 4

MEm4R (m=0-1)

ME m Read Register 4 (0150$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31-24	23-16
RD43	RD42
rh	rh

15-8	7-0
RD41	RD40
rh	rh

Field	Bits	Type	Description
RD40	7:0	rh	**DMA Read Move Data Byte**
RD41	15:8	rh	**DMA Read Move Data Byte**
RD42	23:16	rh	**DMA Read Move Data Byte**
RD43	31:24	rh	**DMA Read Move Data Byte**

ME m Read Register 5

MEm5R (m=0-1)

ME m Read Register 5 (0154$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31-24	23-16
RD53	RD52
rh	rh

15-8	7-0
RD51	RD50
rh	rh

Field	Bits	Type	Description
RD50	7:0	rh	**DMA Read Move Data Byte**
RD51	15:8	rh	**DMA Read Move Data Byte**
RD52	23:16	rh	**DMA Read Move Data Byte**
RD53	31:24	rh	**DMA Read Move Data Byte**

ME m Read Register 6

MEm6R (m=0-1)
ME m Read Register 6 (0158$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31–24	23–16
RD63	RD62
rh	rh

15–8	7–0
RD61	RD60
rh	rh

Field	Bits	Type	Description
RD60	7:0	rh	**DMA Read Move Data Byte**
RD61	15:8	rh	**DMA Read Move Data Byte**
RD62	23:16	rh	**DMA Read Move Data Byte**
RD63	31:24	rh	**DMA Read Move Data Byte**

ME m Read Register 7

MEm7R (m=0-1)
ME m Read Register 7 (015C$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31–24	23–16
RD73	RD72
rh	rh

15–8	7–0
RD71	RD70
rh	rh

Field	Bits	Type	Description
RD70	7:0	rh	**DMA Read Move Data Byte**
RD71	15:8	rh	**DMA Read Move Data Byte**
RD72	23:16	rh	**DMA Read Move Data Byte**
RD73	31:24	rh	**DMA Read Move Data Byte**

ME m Channel Read Data CRC Register

The read only DMA active channel Read Data CRC Register will store one polynomial calculation during each DMA read move provided that there is no retry or error condition flagged. In order to start a CRC32 sequence the MEmRDCRC must be initialized (e.g. written with 00000000$_H$ or with a desired start value) and an DMA transaction must be set up (start address, length, etc.).

MEmRDCRC (m=0-1)

ME m Channel Read Data CRC Register (0180$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RDCRC															
rh															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDCRC															
rh															

Field	Bits	Type	Description
RDCRC	31:0	rh	**Read Data CRC** Active DMA channel read data CRC32 ethernet polynomial checksum.

ME m Channel Source and Destination Address CRC Register

The read only DMA active channel Source and Destination Address CRC Register will store one polynomial calculation during each DMA read and each DMA write move provided there is no retry or error condition flagged.

MEmSDCRC (m=0-1)

ME m Channel Source and Destination Address CRC Register(0184$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SDCRC															
rh															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SDCRC															
rh															

Field	Bits	Type	Description
SDCRC	31:0	rh	**Source and Destination Address CRC** Active DMA channel source and destination address CRC32 ethernet polynomial checksum.

ME m Channel Source Address Register

The read only DMA active channel Source Address Register holds the 32-bit source address. If a DMA channel is active, MEmSADR is updated continuously and shows the actual source address used for DMA read moves.

MEmSADR (m=0-1)

ME m Channel Source Address Register (0188$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SADR															
rh															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SADR															
rh															

Field	Bits	Type	Description
SADR	31:0	rh	**Source Address** Active DMA channel 32-bit source address used for DMA read moves.

ME m Channel Destination Address Register

The read only DMA active channel Destination Address Register holds the 32-bit destination address. If a DMA channel is active, MEmDADR is updated continuously and shows the actual destination address used for DMA write moves.

MEmDADR (m=0-1)

ME m Channel Destination Address Register(018C$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DADR															
rh															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DADR															
rh															

Field	Bits	Type	Description
DADR	31:0	rh	**Destination Address** Active DMA channel 32-bit destination address used for DMA write moves.

ME m Channel Address and Interrupt Control Register

The read only DMA active channel Address and Interrupt Control Register controls how source and destination addresses are updated after a DMA move. It determines whether or not a source or destination address register update is shadowed. The interrupt control bits enable the generation of DMA channel interrupt triggers.

MEmADICR (m=0-1)

ME m Channel Address and Interrupt Control Register(0190$_H$+m*1000$_H$) Application Reset Value: 0000 0000$_H$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IRDV				INTCT		WRPDE	WRPSE	0	STAMP	DCBE	SCBE	SHCT			
rh				rh		rh	rh	r	rh	rh	rh	rh			

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CBLD				CBLS				INCD	DMF			INCS	SMF		
rh				rh				rh	rh			rh	rh		

Field	Bits	Type	Description
SMF	2:0	rh	**Source Address Modification Factor** Active DMA channel source address modification factor.
INCS	3	rh	**Increment of Source Address** Active DMA channel increment of source address control.
DMF	6:4	rh	**Destination Address Modification Factor** Active DMA channel destination address modification factor.
INCD	7	rh	**Increment of Destination Address** Active DMA channel increment of destination address control.
CBLS	11:8	rh	**Circular Buffer Length Source** Active DMA channel circular source buffer control.
CBLD	15:12	rh	**Circular Buffer Length Destination** Active DMA channel circular destination buffer control.
SHCT	19:16	rh	**Shadow Control** Active DMA channel control of shadow address register function.
SCBE	20	rh	**Source Circular Buffer Enable** Active DMA channel circular source buffer enable/disable.
DCBE	21	rh	**Destination Circular Buffer Enable** Active DMA channel circular destination buffer enable/disable.
STAMP	22	rh	**Time Stamp** Active DMA channel control to enable the appendage of a timestamp after the end of the last DMA Move during a DMA transaction.
WRPSE	24	rh	**Wrap Source Enable** Active DMA channel source buffer interrupt trigger enable/disable.
WRPDE	25	rh	**Wrap Destination Enable** Active DMA channel destination buffer interrupt trigger enable/disable.
INTCT	27:26	rh	**Interrupt Control** Active DMA channel interrupt service request control.
IRDV	31:28	rh	**Interrupt Raise Detect Value** Active DMA channel control of the Threshold Limit of of DMA channel CHSR.TCOUNT for triggering a channel interrupt service request.

Field	Bits	Type	Description
0	23	r	**Reserved** Read as 0; must be written with 0.

ME m Channel Control Register

The read only DMA active channel Channel Control Register contains the configuration and control bits.

MEmCHCR (m=0-1)

ME m Channel Control Register **(0194_H+m*1000_H)** **Application Reset Value: 0000 0000_H**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0			PRSEL	SWAP	PATSEL			CHDW			CHMODE	RROAT	BLKM		
r			rh	rh	rh			rh			rh	rh	rh		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0		TREL													
r		rh													

Field	Bits	Type	Description
TREL	13:0	rh	**Transfer Reload Value** Active DMA channel Transfer Reload Value.
BLKM	18:16	rh	**Block Mode** Active DMA channel Block Mode.
RROAT	19	rh	**Reset Request Only After Transaction** Active DMA channel Reset Request Only After Transaction.
CHMODE	20	rh	**Channel Operation Mode** Active DMA channel Channel Operation Mode.
CHDW	23:21	rh	**Channel Data Width** Active DMA channel DMA move Channel Data Width.
PATSEL	26:24	rh	**Pattern Select** Active DMA channel Pattern Select control.
SWAP	27	rh	**Swap Data CRC byte order** Active DMA channel swap data CRC byte order.
PRSEL	28	rh	**Peripheral Request Select** Active DMA channel Peripheral Request Select.
0	15:14, 31:29	r	**Reserved** Read as 0; must be written with 0.

ME m Channel Shadow Address Register

The read only DMA active channel Shadow Address Register holds the 32-bit shadow address.

MEmSHADR (m=0-1)

ME m Channel Shadow Address Register **(0198_H+m*1000_H)** **Application Reset Value: 0000 0000_H**

31–16
SHADR
rh

15–0
SHADR
rh

Field	Bits	Type	Description
SHADR	31:0	rh	**Shadowed Address** This bit field holds the 32-bit shadow address of the active DMA channel. The function of the shadow address is set by the shadow control settings.

ME m Channel Status Register

The read only DMA active channel Channel Status Register contains the current transfer count, pattern detection compare result and the status of traffic management interrupt triggers.

MEmCHSR (m=0-1)

ME m Channel Status Register **($019C_H$+m*1000_H)** **Application Reset Value: 0000 0000_H**

31–24	23	22	21–20	19	18	17	16
0	FROZEN	BUFFER	0	IPM	ICH	WRPD	WRPS
r	rh	rh	r	rh	rh	rh	rh

15	14	13–0
LXO	0	TCOUNT
rh	r	rh

Field	Bits	Type	Description
TCOUNT	13:0	rh	**Transfer Count Status** Active DMA channel count of the number of DMA transfers. TCOUNT is loaded with the DMA channel value of CHCFGR.TREL when TSR.CH becomes set (and TCOUNT = 0). After each DMA transfer, TCOUNT is decremented by 1.
LXO	15	rh	**Old Value of Pattern Detection** Active DMA channel compare result of a pattern compare operation when 8-bit or 16-bit data width is selected.
WRPS	16	rh	**Wrap Source Buffer** Active DMA channel Wrap Source Buffer status bit.
WRPD	17	rh	**Wrap Destination Buffer** Active DMA channel Wrap Destination Buffer status bit.

Field	Bits	Type	Description
ICH	18	rh	**Interrupt from Channel** Active DMA channel detection of channel interrupt service request.
IPM	19	rh	**Pattern Detection from Channel** Active DMA channel detection of pattern match interrupt service request.
BUFFER	22	rh	**DMA Double Buffering Active Buffer** Active DMA channel DMA Double Buffering Active Buffer status bit. 0_B Buffer 0 read or filled by DMA. 1_B Buffer 1 read or filled by DMA.
FROZEN	23	rh	**DMA Double Buffering Frozen Buffer** Active DMA channel DMA Double Buffering Frozen Buffer status bit.
0	14, 21:20, 31:24	r	**Reserved** Read as 0; must be written with 0.

18.5 Debug

The DMA implements the following debug features:

18.5.1 DMA Channel Suspend

Two DMA channel registers (at **Figure 225**) control and report the channel suspend operation:

- The Suspend Enable Register (SUSENR) enables and disables DMA channel soft suspend mode capability.
- The Suspend Acknowledge Register (SUSACR) indicates the DMA channel soft suspend status.

The On Chip Debug System (OCDS) is able to generate a DMA suspend request (SUSREQ). When the suspend request becomes active, the state of a DMA channel becomes frozen to ensure that the state of the DMA channels can be analyzed by reading the DMA channel register contents. If a DMA channel is active then the current DMA transfer is completed. The DMA signals the suspend mode status back to the OCDS via a suspend acknowledge.

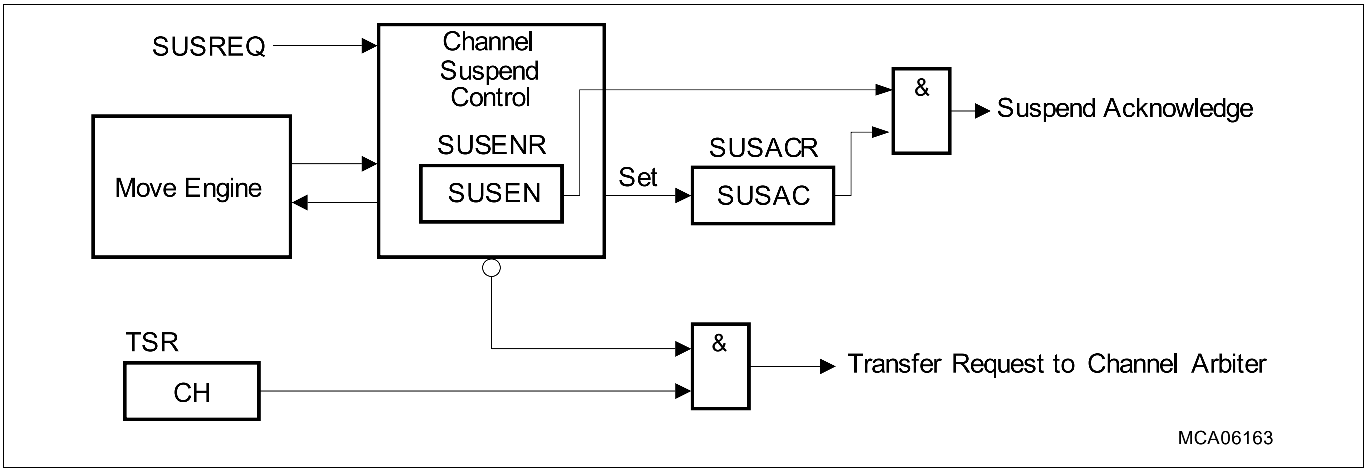

Figure 225 DMA Channel Suspend

Entering Channel Suspend Mode

A DMA channel is configured for Suspend Mode when the DMA channel suspend enable bit in the Suspend Mode Enable Register SUSENR.SUSEN is set. When SUSREQ becomes active, for DMA channels enabled for **DMA Channel Suspend**, the operation is as follows:

- **Idle State**, **Reset State**, **Halt State** and **Pending State**: the DMA channel suspend active status flag SUSACR.SUSAC is set and the DMA channel state is frozen.
- **Active State**: the DMA channel suspend active status flag SUSACR.SUSAC is set on completion of the current DMA transfer and the DMA channel state is frozen.

The DMA suspend acknowledge flag becomes active when all DMA channels that are enabled for **DMA Channel Suspend** have set their DMA channel suspend active status flag SUSACR.SUSAC. In **DMA Channel Suspend**, the DMA channel contents may be modified. These modifications are taken into account for further DMA transfers or DMA transactions of the related DMA channel after **DMA Channel Suspend** has been left.

DMA channels that are disabled for **DMA Channel Suspend** (SUSENR.SUSEN = 0) continue to execute the DMA channel operation.

Exiting Channel Suspend Mode

DMA Channel Suspend mode is left and the DMA channel operation is resumed if either the SUSREQ signal becomes inactive, or if the enable bit SUSENR.SUSEN is reset by software.

18.5.2 Software Activation of DMA Channel Interrupt Service Requests

Each DMA channel interrupt service request shall be activated by programming the DMA channel CHCSR.SIT = 1_B.

18.5.3 Software Activation of DMA RP Error Interrupt Service Requests

Each DMA RP error interrupt service request shall be activated by programming the RP ERRINT.SIT = 1_B.

18.5.4 OCDS Trigger Bus (OTGB) Interface

A **TS** is a collection of signals which supports a specific debug use case. The OCDS Trigger Set Select (OTSS) register controls which TS is applied to one of two DMA OTGB interfaces: OTGB0 or OTGB1. A TS is routed by the OCDS Trigger Multiplexer (OTGM) to the OCDS Trigger Switch (OTGS).

DMA Trigger Sets

The DMA Trigger Sets (at **Table 583**) are selected and routed to the 16 bit OTGB0 or OTGB1 interface by programming the OTSS register. Only one TS shall be output at a time either on OTGB0 or on OTGB1. The OTGB0 and OTGB1 have no dependency on the OCDS enabled state.

DMA TS are unique to each DMA configuration and dependent on the number of DMA channels and ME.

Table 583 DMA Trigger Sets

Index	Description	Details
0	No Trigger Set selected	
1	**Channels (TS16_PF)** Active channels	**Table 584**
2	**Channels (TS16_ERR)** Error channel number and flags	**Table 585**
8	**Transaction Request State (TS16_C15)** Transaction Request State Channel [15:0]	
9	**Transaction Request State (TS16_C31)** Transaction Request State Channel [31:16]	
10	**Transaction Request State (TS16_C47)** Transaction Request State Channel [47:32]	
11	**Transaction Request State (TS16_C63)** Transaction Request State Channel [63:48]	
12	**Transaction Request State (TS16_C79)** Transaction Request State Channel [79:64]	
13	**Transaction Request State (TS16_C95)** Transaction Request State Channel [95:80]	
14	**Transaction Request State (TS16_C111)** Transaction Request State Channel [111:96]	
15	**Transaction Request State (TS16_C127)** Transaction Request State Channel [127:112]	
Other	Reserved. No Trigger Set selected	

Performance Trigger Set

The performance **TS** (TS16_PF) continuously monitors ME activity. While a ME is servicing a **DMA Request**, the ME idle/active bit TS16_PF.ME0/1 is set and the TS16_PF.CH0/1 field is set to the number of the DMA channel. When the ME is idle the idle/active bit TS16_PF.ME0/1 goes low. The TS16_PF.CH0/1 bit field retains the value of the last active DMA channel.

Table 584 TS16_PF Trigger Set Channels

Bits	Name	Description
[6:0]	CH0	Number of DMA channel active in ME0
7	ME0	ME0 idle/active 0_B ME0 is idle 1_B ME0 is active
[14:8]	CH1	Number of DMA channel active in ME1
15	ME1	ME1 active 0_B ME1 is idle 1_B ME1 is active

Error Trigger Set

The error **TS** (TS16_ERR) triggers an event for DMA source errors and DMA destination errors.

If a **ME** detects a **SER** or **DER**, the DMA sets TS16_ERR as follows:

- If **ME**m detects a **SER**, the DMA sets TS16_ERR.MEmSE for one trigger cycle.
- If **ME**m detects a **DER**, the DMA sets TS16_ERR.MEmDE for one trigger cycle.
- The DMA sets TS16_ERR.LEC to the last error channel for one trigger cycle. If both MEs generate a **SER** and/or **DER** in the same clock cycle, the DMA shall set TS16_ERR.LEC to the ME0 last error channel.

Table 585 TS16_ERR Trigger Set Channels

Bits	Name	Description
[6:0]	LEC	Last error channel number
[11:7]		Reserved
12	ME0SE	ME0 Source Error
13	ME0DE	ME0 Destination Error
14	ME1SE	ME1 Source Error
15	ME1DE	ME1 Destination Error

Request Trigger Sets

The request trigger sets (TS16_C15, etc.) continuously monitor the state of the DMA channel TSR.CH bits.

18.5.5 MCDS Trace Interface

Each DMA on chip bus master interface generates a trace vector as follows:

- One 8 bit vector from SPB master interface via BCU_SPB to BAL_SPB inside MCDS (i.e. spb_clk domain).
- One 8 bit vector from SRI master interface via SRI_XBAR to BAL_SRI inside MCDS (i.e. sri_clk domain).

The trace vector (at **Table 586**) identifies the DMA channel and ME making an on chip bus access.

Table 586 Trace Vector Definition

Bits	Trace
[6:0]	DMA Channel
[7]	Move Engine 0_B ME0 1_B ME1

The trace vector is used for OCDS Level 3 and OCDS Level 1 purposes:

DMA Trace Signal Generation for OCDS Level 3

The trace mechanism enables the MCDS system to trace transactions on the on chip bus generated by one or a group of dedicated DMA internal sources.

DMA Trace Signal Generation for OCDS Level 1

For OCDS Level 1 purposes the DMA provides 8 bit trace vectors.

18.6 Use Cases

18.6.1 Move Operation

A code example is provided to give an overview of core DMA functionality. The example realizes DMA channel 000 executing a one word (32 bit-length) single data transfer from a source module to a data module without the intervention of the CPU. The DMA transaction is triggered by a DMA software request in this example. The DMA also supports DMA hardware requests. No interrupt or further functions are created in this example, please see the respective registers for more details.

18.6.1.1 Step Description to Initialize and Trigger a DMA Transaction

(Line 1) The 32-bit source address (DMA_SADR000_ADD) gets stored in the source address register DMA_SADR000.

(Line 2) The 32-bit destination address (DMA_DADR000_ADD) gets stored in the destination address register DMA_DADR000.

(Line 3) The channel data width is defined in the channel configuration register DMA_CHCFGR000. Setting DMA_CHCFGR000.[23:21] = 010_B configures the channel data width to 32 bit.

(Line 4) DMA hardware requests are disabled in the transaction state register DMA_TSR000. Writing DMA_TSR000.[17] = 1 disables the hardware transfer requests.

(Line 5) This line starts the data transfer between the source and destination memory address.

Note: The declaration of DMA_SADR000_ADD and DMA_DADR000_ADD must be done by the user.

DMA channel 000 Code

```
(1)     DMA_SADR000 = DMA_SADR000_ADD; // source address
(2)     DMA_DADR000 = DMA_DADR000_ADD; // destination address
(3)     DMA_CHCFGR000.U = (0x2 << 21); // channel data width
(4)     DMA_TSR000.U |= (1 << 17); //disable DMA hardware requests
(5)     DMA_CHCSR000.U |= (1 << 31); //initiate DMA software request
```

Note: The DMA transaction can be started anywhere in the program.

18.6.2 Error Handler

Each RP generates a single error interrupt trigger to signal a number of different parallel error conditions:

- ME detected errors during the execution of DMA moves.
- TRL errors when a DMA request is not serviced.

The DMA RP error interrupt service request shall be serviced by the CPU supervising the RP.

An EH shall read the following registers to identify the error type and the DMA error channel:

- ME Error Status Registers (ERRSR) to detect the following:
  - Last Error Channel (LEC)
  - Source Error (SER)
  - Destination Error (DER)
  - RAM Error (RAMER)
  - Safe Linked List Error (SLLER)
  - DMA Linked List (DLLER)
- DMA channel Transaction State Registers (TSR) to detect the following:

- Transaction Request State (CH)
- Transaction Request Lost (TRL)

The error routine should disable further DMA requests from the DMA error channel:

- DMA hardware requests: the CPU shall write DMA error channel TSR.DCH = 1_B
- DMA software requests: the CPU shall not write DMA error channel CHCSR.SCH = 1_B

The error routine should clear the error status flag(s) as follows:

- ME error flags:
  - SER: the CPU error routine shall write ME CLRE.CSER = 1_B
  - DER: the CPU error routine shall write ME CLRE.CDER = 1_B
  - SPBER: the CPU error routine shall write ME CLRE.CSPBER = 1_B
  - SRIER: the CPU error routine shall write ME CLRE.CSRIER = 1_B
  - RAMER: the CPU error routine shall write ME CLRE.CRAMER = 1_B
  - SLLER: the CPU error routine shall write ME CLRE.CSLLER = 1_B
  - DLLER: the CPU error routine shall write ME CLRE.CDLLER = 1_B
- DMA channel error flags:
  - TRL: the CPU error routine shall write the DMA error channel TSR.CTL = 1_B

The user shall identify the origin of the error. The DMA configuration shall be changed to prevent a reoccurrence of the error. On completion DMA requests shall be serviced as follows:

- DMA hardware requests: the CPU shall write DMA channel TSR.ECH = 1_B
- DMA software requests: the CPU shall write DMA channel CHCSR.SCH = 1_B

18.6.3 Data Communication

From TC39xB DMA and TC38xA DMA onwards, the DMA has been enhanced to report **TRL** errors if the DMA channel is disabled for hardware requests and a **DMA Hardware Request** is detected.

For example, if the application is transmitting data via a QSPI, the software may perform one write to the TXFIFO or may set the interrupt flag in the TXFIFO interrupt node to initiate data transmission. After each data word is loaded into the TXFIFO, the QSPI triggers a TXFIFO interrupt service request. The system effect is that (n +1) interrupts are generated to move (n) data words. If the application uses the DMA to move data into the TXFIFO and the DMA channel is configured for **Single Mode**, the DMA will disable **DMA Hardware Request** after the DMA has serviced (n) interrupts and report a **TRL** for the last interrupt.

To prevent this **TRL** event, the software should configure the DMA channel as a **DMA Linked List (DMALL)** composed of the following DMA transactions:

- DMA Transaction 1: perform (n) DMA moves to write data to TXFIFO.
- DMA Transaction 2: perform a dummy DMA move to read and write data to an unused address in DSPR.

If the DMA is used for data communication, the application should consider the possibility of **TRL** generation after the last data word.

18.7 Revision History

Table 587 Changes

Reference	Change to Previous Version	Comment
V0.1.15		
Page 59	Safety Flip-Flops **Chapter 18.4.1** added	

Table 587 Changes

Reference	Change to Previous Version	Comment
Page 7	Figure “Software Control” updated	
V0.1.16		
Page 13	Update Resetting a DMA Channel.	
V0.1.17		
Page 39	Updated section “Active Buffer Full or Empty”.	
Page 55	Updated section “Data Integrity Testing”.	
V0.1.18		
Page 23	Updated diagram “SRI Bus Protocol Block Transfer Request - 2 Transfers” regarding BLKM.	